📚 Operating Systems Overview

Brief Overview

This note covers **Operating Systems** and was created from the PDF (273 pages). It covers **distributed file systems**, pipes and IPC, process and thread scheduling, memory management, and synchronization primitives.

Key Points

- Distributed file system request handling and DFS daemon operations.
- Interprocess communication: ordinary and named pipes, message passing, sockets.
- Process and thread scheduling strategies: FCFS, RR, SJF, RMS, EDF, and multilevel feedback.
- Memory management fundamentals: paging, segmentation, virtual memory, swap and COW.

📡 Distributed File System (DFS) Operations

> **Definition:** A DFS request message contains the file operation to be performed on the server's disk (e.g., read, write, rename, delete) and any data needed for the call.
> The DFS daemon on the server executes the operation on behalf of the client and returns a response message with the operation's status and any resulting data.

- **Typical workflow**
  1. Client sends a request message describing the file-related system call.
  2. DFS daemon performs the corresponding disk operation.
  3. Server replies with a status code and, if applicable, the data requested (e.g., file contents).

🔀 Pipes Overview

> **Definition:** A pipe is an inter-process communication (IPC) mechanism that provides a unidirectional or bidirectional conduit for data flow between processes.

Design considerations

Question	Possible Answers
Directionality	Unidirectional vs. bidirectional
Duplex mode	Half-duplex (one-way at a time) or full-duplex
Process relationship	Requires parent–child relationship?
Network scope	Same-machine only or network-transparent?

📂 Ordinary Pipes

UNIX Implementation

> **Definition:** An ordinary (anonymous) pipe is a special file type created by pipe(int fd[]); it yields two file descriptors: fd[0] (read end) and fd[1] (write end).

- **Typical usage pattern**
  1. Parent creates the pipe.
  2. Parent forks a child.
  3. Each process closes the end it does not use.
  4. Parent writes; child reads (or vice-versa).
  5. Closing the write end allows the reader to detect end-of-file (read() returns 0).

```
/* UNIX anonymous pipe example */
#include
#include
#include
#define BUFFER_SIZE 25
#define READ_END 0
#define WRITE_END 1

int main(void) {
    char write_msg[BUFFER_SIZE] = "Greetings";
    char read_msg[BUFFER_SIZE];
    int fd[2];
    pid_t pid;

    if (pipe(fd) == -1) {
        perror("Pipe creation failed");
        return 1;
    }

    pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }

    if (pid > 0) {               /* Parent process */
        close(fd[READ_END]);                    // close unused read end
        write(fd[WRITE_END], write_msg, strlen(write_msg));
        close(fd[WRITE_END]);                   // signal EOF
    } else {                     /* Child process */
        close(fd[WRITE_END]);                   // close unused write end
        read(fd[READ_END], read_msg, BUFFER_SIZE);
        printf("Child read: %s\n", read_msg);
        close(fd[READ_END]);
    }
    return 0;
}
```

Windows Implementation (Anonymous Pipe)

> **Definition:** In Windows, an anonymous pipe is created with CreatePipe, producing separate read and write handles. The pipe is half-duplex and typically used between a parent and a child process.

```
/* Windows anonymous pipe example */
#include
#include
#define BUFFER_SIZE 25

int main(void) {
    HANDLE ReadHandle, WriteHandle;
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    char message[BUFFER_SIZE] = "Greetings";
    DWORD written;

    SECURITY_ATTRIBUTES sa = { sizeof(SECURITY_ATTRIBUTES), NULL, TRUE };
    ZeroMemory(&pi, sizeof(pi));
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);

    if (!CreatePipe(&ReadHandle, &WriteHandle, &sa, 0)) {
        fprintf(stderr, "CreatePipe failed\n");
        return 1;
    }
```

```
    si.hStdOutput = WriteHandle;
    si.hStdInput  = ReadHandle;
    si.dwFlags    = STARTF_USESTDHANDLES;

    // Inherit handles for the child
    if (!CreateProcess(NULL, "child.exe", NULL, NULL, TRUE,
                       0, NULL, NULL, &si, &pi)) {
        fprintf(stderr, "CreateProcess failed\n");
        return 1;
    }

    CloseHandle(ReadHandle);                    // parent does not read
    WriteFile(WriteHandle, message, BUFFER_SIZE, &written, NULL);
    CloseHandle(WriteHandle);                   // signal EOF
    WaitForSingleObject(pi.hProcess, INFINITE);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
    return 0;
}
```

- **Key points**
  - The child inherits the pipe handles via STARTUPINFO.
  - The parent closes the read handle; the child closes the write handle.
  - Half-duplex: only one direction of data flow at a time.

📡 Named Pipes (FIFOs)

UNIX FIFO

> **Definition:** A FIFO (named pipe) appears as a file in the filesystem, created with mkfifo(). It persists beyond the lifetimes of the processes that use it.

- **Characteristics**
  - Can be accessed by unrelated processes.
  - Supports bidirectional communication, but only half-duplex per FIFO; two FIFOs are needed for full duplex.
  - Communication limited to the same machine (network communication requires sockets).

Windows Named Pipe

> **Definition:** Windows named pipes are created with CreateNamedPipe() and support full-duplex, byte- or message-oriented transfer. Processes may reside on the same or different machines.

```
/* Windows named pipe (server) example */
#include
#include
#define BUFFER_SIZE 25

int main(void) {
    HANDLE ReadHandle, WriteHandle;
    SECURITY_ATTRIBUTES sa = { sizeof(SECURITY_ATTRIBUTES), NULL, TRUE };
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    DWORD written;
    char message[BUFFER_SIZE] = "Error writing to pipe.";

    // Create the pipe
    if (!CreatePipe(&ReadHandle, &WriteHandle, &sa, 0)) {
        fprintf(stderr, "CreatePipe failed\n");
        return 1;
    }
```

```
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    si.hStdInput  = ReadHandle;
    si.hStdOutput = WriteHandle;
    si.dwFlags    = STARTF_USESTDHANDLES;

    // Launch child that inherits the handles
    if (!CreateProcess(NULL, "child.exe", NULL, NULL, TRUE,
                       0, NULL, NULL, &si, &pi)) {
        fprintf(stderr, "CreateProcess failed\n");
        return 1;
    }

    CloseHandle(ReadHandle);                    // parent writes only
    WriteFile(WriteHandle, message, BUFFER_SIZE, &written, NULL);
    CloseHandle(WriteHandle);                   // signal EOF
    WaitForSingleObject(pi.hProcess, INFINITE);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
    return 0;
}
```

- **Key differences vs. UNIX FIFO**
  - Full-duplex communication is native.
  - Can operate across networked machines.
  - Supports both byte-oriented and message-oriented data.

🔧 Pipes in Practice (Command-Line)

- **UNIX:** ls | more
  - ls (producer) writes directory listing to the pipe; more (consumer) reads from the pipe and paginates the output.
- **Windows:** dir | more
  - Same concept with dir as the producer and more as the consumer.

The pipe character | connects the standard output of the left command to the standard input of the right command.

🛠️ Process States & Scheduling

> **Definition:** A process is an executing program instance; its state reflects its current activity.

- **Possible states**
  - **new** – just created
  - **ready** – waiting for CPU allocation
  - **running** – currently executing
  - **waiting** – blocked for I/O or other event
  - **terminated** – execution finished
- **Scheduling queues**

Queue type	Contains
Ready queue	Processes ready to run on the CPU
I/O request queue	Processes awaiting I/O completion
Waiting queues	Processes blocked for specific events

- **Scheduler categories**
  - **Long-term (job) scheduler:** selects which processes enter the ready queue (controls degree of multiprogramming).
  - **Short-term (CPU) scheduler:** chooses the next process from the ready queue for execution.

📡 Interprocess Communication (IPC) Mechanisms

Mechanism	Typical use	Responsibility
Shared memory	Large data exchange, low latency	Application programmers allocate and manage the shared region; OS provides mapping facilities
Message passing	Small control messages, synchronization	OS may provide the messaging API (e.g., sockets, pipes)
Sockets	Networked client-server communication	OS handles endpoint creation, routing, and data transport
Remote Procedure Calls (RPCs)	Distributed procedure invocation	OS or middleware implements request/response handling
Pipes	Parent-child or related processes (ordinary) or unrelated local processes (named)	OS creates and manages the pipe objects

---

🧩 Selected Programming Problems (Key Concepts)

Collatz Conjecture with fork()

- Parent creates a child that computes the Collatz sequence for a given integer (passed on the command line).
- Child prints the sequence; parent waits (wait()) for child termination before exiting.
- Demonstrates separate address spaces and synchronization via wait().

PID Manager Design

- Maintain a bitmap for PIDs in the range [MIN_PID, MAX_PID] (300-5000).
- Operations:
  - int allocate_pid(void); → returns an available PID or -1.
  - void release_pid(int pid); → marks the PID as free.
  - void init_pid_manager(void); → allocates and zeroes the bitmap.

File-Copy Program Using Pipes

- Parent opens the source file, writes its contents into an anonymous pipe, then forks a child.
- Child reads from the pipe and writes to the destination file.
- Illustrates producer-consumer pattern with a single pipe.

Echo Server (Java)

- Accepts a client connection with ServerSocket.accept().
- Reads bytes into a buffer; writes the same bytes back to the client.
- Loop terminates when InputStream.read() returns -1 (client closed).
- Demonstrates socket-based IPC and handling of binary data.

🖥️ Simple Shell Design

> **Definition:** A *shell* is a command-line interpreter that reads user input, creates processes to execute commands, and optionally manages background execution.

Main Loop & Process Creation

1. Print prompt osh> and flush stdout.
2. Read a line of input from the user.
3. **Parse** the line into separate tokens (command + arguments) and store them in an args[] array.
4. Call fork() to create a child process.

5. In the **child**: invoke execvp(args[0], args) to replace the child's image with the requested program.
6. In the **parent**:
    - If the command **does not** end with &, call wait() to block until the child terminates.
    - If the command **does** end with &, skip wait() so parent and child run **concurrently**.

```
/* Simplified shell outline (C) */
#include
#include
#define MAX_LINE 80

int main(void) {
    char *args[MAX_LINE/2 + 1];
    int should_run = 1;

    while (should_run) {
        printf("osh> "); fflush(stdout);
        /* (1) read input, (2) tokenize into args[] */
        /* (3) fork child */
        /* (4) child: execvp(args[0], args) */
        /* (5) parent: wait() unless '&' present */
    }
    return 0;
}
```

Handling the Ampersand (&)

- The presence of & tells the shell **not** to wait for the child.
- The parent immediately returns to the prompt, allowing the user to start another command while the first runs in the background.

📜 Shell History Feature

> **Definition:** The *history* buffer records the most recent commands entered, enabling recall and re-execution.

- **Capacity:** Stores the **10 most recent** commands, but numbering continues beyond 10 (e.g., 26–35 for the latest ten).
- **Listing:** Typing history prints the numbered list, most recent first.

Input	Action
!!	Re-execute the **most recent** command.
!N	Re-execute the command with number **N** in the history list.
history	Display the stored commands with their numbers.

- **Echoing:** When a command is invoked via !! or !N, the shell first prints the command line before execution.
- **Error handling:**
    - If no command exists for !!, output: No commands in history.
    - If the requested number does not exist, output: No such history.

The history buffer must be updated **after** a command finishes (including those run in background).

🐧 Linux Kernel Module: Listing Tasks

> **Definition:** A kernel module can traverse the kernel's task structures to report information about all running processes.

Part I – Linear Iteration

- Include to access struct task_struct.
- Use the macro for_each_process(task) to walk the global task list.

```
struct task_struct *task;
for_each_process(task) {
    /* task->comm = name, task->pid = PID, task->state = state */
    printk(KERN_INFO "Task: %s  PID: %d  State: %ld\n",
           task->comm, task->pid, task->state);
}
```

- The printk output appears in the kernel log (dmesg).
- Verify correctness by comparing the output with ps -el; minor differences are expected because kernel threads (e.g., idle threads) may not appear in ps.

Part II – Depth-First Search (DFS) of the Process Tree

- Each task_struct contains list_head children and list_head sibling.
- Use list_for_each and list_entry to traverse children recursively.

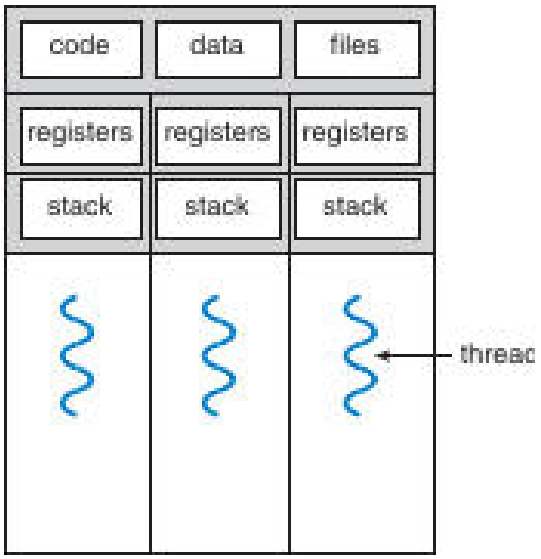
```
void dfs_task(struct task_struct *task) {
    struct list_head *list;
    struct task_struct *child;

    printk(KERN_INFO "Task: %s  PID: %d  State: %ld\n",
           task->comm, task->pid, task->state);

    list_for_each(list, &task->children) {
        child = list_entry(list, struct task_struct, sibling);
        dfs_task(child);   /* recursive depth-first walk */
    }
}
```

- Invoke dfs_task(&init_task) from the module's entry function.
- The DFS order typically differs from the flat list produced by for_each_process, showing the hierarchical parent-child relationships.

🧵 Thread Fundamentals

> **Definition:** A *thread* is the smallest unit of CPU utilization, consisting of its own register set and stack, while sharing the process's code, data, and open resources.

Memory Layout of a Multithreaded Program



*The diagram shows three threads (T1, T2, T3). The **code**, **data**, and **files** regions are shared among all threads, whereas each thread has its own **registers** and **stack**.*

Benefits of Multithreading

Benefit	Explanation
Responsiveness	UI remains interactive while a long-running operation executes in a separate thread.
Resource Sharing	Threads automatically share memory and open file descriptors, avoiding explicit IPC.
Economy	Creating a thread is cheaper than creating a full process; context switches are faster.
Scalability	On multicore CPUs, threads can run in parallel, improving throughput.

Multicore Programming

- **Concurrency vs. Parallelism** – *Concurrency* is the interleaved progress of multiple tasks; *parallelism* requires multiple cores executing tasks simultaneously.
- **Amdahl's Law** quantifies speedup:

  $\text{Speedup} \le \frac{1}{(1 - S) + \frac{S}{N}}$

  where S is the serial fraction and N the number of cores.
  - Example: 25 % serial, 75 % parallel on 4 cores → speedup ≈ 2.28×.
  - As $N \to \infty$, speedup approaches $1/(1-S)$.

Types of Parallelism

Type	Focus
Data parallelism	Same operation applied to different subsets of a data set (e.g., splitting an array between threads).
Task parallelism	Different operations executed concurrently on the same or different data (e.g., one thread sorts, another computes statistics).

Thread Models

Many-to-One

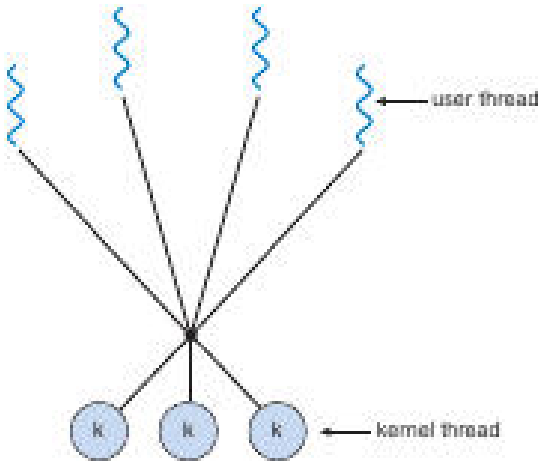
- Multiple **user-level** threads are mapped onto **one** kernel thread.
- Scheduling is done in user space; a blocking system call stalls **all** threads.
- Historically used by “green threads” (early Java, Solaris).

One-to-One

- Each user thread has a dedicated kernel thread.
- Allows true parallelism on multicore systems; a blocking call only blocks its own thread.
- Implemented by Linux and Windows.

Many-to-Many

- Many user threads are multiplexed over a **pool** of kernel threads.
- Balances flexibility with scalability.



The diagram illustrates multiple user threads (top wavy lines) connected to several kernel threads (bottom circles) via a mapping layer, representing the many-to-many relationship.

Two-Level Many-to-Many (Hybrid)

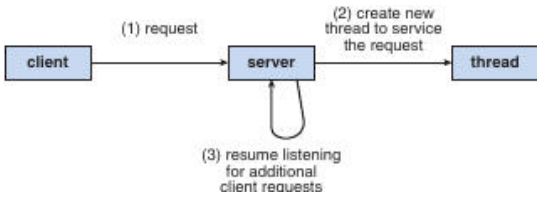
- User threads are bound to a subset of kernel threads; excess user threads are scheduled onto available kernel threads as needed.
- Used in older Solaris versions; newer systems adopt the one-to-one model for simplicity and better SMP support.

Thread Libraries

- Provide an API (e.g., pthread_create, pthread_join on POSIX) that abstracts kernel thread creation and management.
- The library handles stack allocation, attribute setting, and synchronization primitives (mutexes, condition variables).

📡 Server-Thread Interaction Example

> **Definition:** In a multithreaded server, each client request can be serviced by a dedicated thread, allowing the server to continue listening for new connections.



The flowchart shows a client sending a request to the server; the server spawns a new thread to handle the request while the main thread returns to listening for additional client connections.

🧵 Thread Libraries Overview

> **Definition:** A *thread library* provides the API and supporting code that applications use to create, manage, and synchronize threads. It may reside entirely in **user space** (no kernel support) or be **kernel-level**, where part of the library runs inside the operating system.

User-Level vs. Kernel-Level Libraries

Aspect	User-Level Library	Kernel-Level Library
Location of code & data	Entirely in user space; library functions are normal function calls.	Partially or wholly in kernel space; some calls invoke system calls.
Thread creation cost	Low (no system call).	Higher (requires a system call).
Scheduling	Performed by the library; the kernel sees only one process.	Handled by the OS scheduler; each thread is a kernel-level entity.
Blocking I/O	A blocking system call can stall **all** threads in the process.	Only the thread that issued the call is blocked.
Portability	May need separate implementations per OS.	Relies on OS-provided thread support.

📚 Major Thread Libraries (Three in Use)

Library	Typical Platform	Implementation Model
POSIX Pthreads	Unix-like (Linux, macOS, Solaris, etc.)	Can be user-level or kernel-level (implementation-defined).
Windows Threads	Windows NT family	Kernel-level (one-to-one with kernel threads).
Java Thread API	JVM on any host OS (Windows, Linux, macOS, Android)	Usually maps to the host's native thread library (Pthreads on Unix, Windows API on Windows).

Note: Java has **no global variables**; shared data must be passed via object references.

🔄 Asynchronous vs. Synchronous Threading

Strategy	Characteristics	Typical Use
Asynchronous	Parent creates a child thread and **continues** execution; threads run concurrently without the parent waiting.	Server request handling (Figure 4.2).
Synchronous (fork-join)	Parent creates children **and must wait** for all to finish before proceeding.	Parallel data-parallel work where results must be combined.

> **Example summation:** The function computes $ \displaystyle \sum_{i=0}^{N} i $.
> For $N = 5$, the result is $0+1+2+3+4+5 = 15$.

🛠️ Pthreads (POSIX) Example

```
/* Pthreads summation program (C) */
#include
#include
#include

int sum;   // shared global variable

void *runner(void *param) {
    int i, upper = atoi((char *)param);
    sum = 0;
    for (i = 1; i <= upper; i++)
        sum += i;
    pthread_exit(0);
}

int main(int argc, char *argv[]) {
    pthread_t tid;
    pthread_attr_t attr;

    if (argc != 2) {
        fprintf(stderr, "usage: %s \n", argv[0]);
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >= 0\n", atoi(argv[1]));
        return -1;
    }

    pthread_attr_init(&attr);                // default attributes
    pthread_create(&tid, &attr, runner, argv[1]);
    pthread_join(tid, NULL);                 // wait for child
    printf("sum = %d\n", sum);
    return 0;
}
```

Key points

- pthread_create starts runner in a new thread.
- Global sum is shared between parent and child.
- pthread_join implements the **fork-join** (synchronous) pattern.

Joining Multiple Threads

```
#define NUM_THREADS 10
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
    pthread_create(&workers[i], NULL, worker_func, NULL);

for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```

🪟 Windows Threads Example

```c
/* Windows thread summation program (C) */
#include
#include

DWORD Sum = 0;   // shared global variable

DWORD WINAPI Summation(LPVOID Param) {
    DWORD Upper = *(DWORD *)Param;
    for (DWORD i = 0; i <= Upper; i++)
        Sum += i;
    return 0;
}

int main(int argc, char *argv[]) {
    HANDLE ThreadHandle;
    DWORD ThreadId, Param;

    if (argc != 2) {
        fprintf(stderr, "An integer parameter is required\n");
        return -1;
    }
    Param = (DWORD)atoi(argv[1]);
    if ((int)Param < 0) {
        fprintf(stderr, "Parameter must be >= 0\n");
        return -1;
    }

    ThreadHandle = CreateThread(
        NULL,               // default security attributes
        0,                  // default stack size
        Summation,          // thread function
        &Param,             // argument to thread function
        0,                  // default creation flags
        &ThreadId);

    if (ThreadHandle != NULL) {
        WaitForSingleObject(ThreadHandle, INFINITE);  // synchronous wait
        CloseHandle(ThreadHandle);
        printf("sum = %lu\n", Sum);
    }
    return 0;
}
```

Key points

- CreateThread creates a kernel-level thread.
- WaitForSingleObject corresponds to pthread_join.
- WaitForMultipleObjects can wait on many handles simultaneously.

☕ Java Threads Example

```java
// Sum holder object
class Sum {
    private int sum;
    public int getSum() { return sum; }
    public void setSum(int s) { this.sum = s; }
}

// Runnable that computes the summation
class Summation implements Runnable {
    private final int upper;
    private final Sum sumObj;

    Summation(int upper, Sum sumObj) {
```

```
        this.upper = upper;
        this.sumObj = sumObj;
    }

    @Override
    public void run() {
        int s = 0;
        for (int i = 0; i <= upper; i++)
            s += i;
        sumObj.setSum(s);
    }
}

// Driver program
public class Driver {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.err.println("Usage: Summation ");
            return;
        }
        int upper = Integer.parseInt(args[0]);
        if (upper < 0) {
            System.err.println(upper + " must be >= 0.");
            return;
        }

        Sum sumObj = new Sum();
        Thread thrd = new Thread(new Summation(upper, sumObj));
        thrd.start();
        try {
            thrd.join();                    // wait for child thread
            System.out.println("The sum of " + upper + " is " + sumObj.getSum());
        } catch (InterruptedException ignored) { }
    }
}
```

Key points

- Java threads are created by new Thread(runnable) and started with start().
- Shared data is passed via object references (Sum).
- join() provides the same **fork-join** synchronization as pthread_join and WaitForSingleObject.

📦 Thread Pools

> **Definition:** A *thread pool* maintains a set of pre-created threads that wait for work. When a task arrives, a free thread is assigned; after completion the thread returns to the pool.

Benefits

1. **Reduced creation overhead** – reusing existing threads is faster than spawning new ones.
2. **Bounded concurrency** – limits the maximum number of active threads, preventing resource exhaustion.
3. **Separation of concerns** – task logic is decoupled from thread-management logic, enabling scheduling policies (delayed, periodic, priority-based).

Windows Thread-Pool API (example)

```
DWORD WINAPI PoolFunction(LPVOID Param) {
    /* work performed by a pooled thread */
    return 0;
}
```

```
/* Submit work to the pool */
QueueUserWorkItem(PoolFunction, NULL, 0);
```

- QueueUserWorkItem takes a pointer to the function, a parameter, and flags.
- For waiting on multiple pool tasks, WaitForMultipleObjects can be used.

Java java.util.concurrent (implicit)

- ExecutorService creates a pool; submit(Runnable) enqueues work.
- shutdown() and awaitTermination() replace explicit joins.

Dynamic Adjustment

- Modern pools (e.g., Apple's Grand Central Dispatch) grow or shrink the number of underlying OS threads based on current load and system capacity.

🧩 OpenMP

> **Definition:** *OpenMP* is a set of compiler directives, library routines, and environment variables for shared-memory parallelism in C/C++/Fortran.

Simple Parallel Region

```
#include
#include

int main(int argc, char *argv[]) {
    #pragma omp parallel
    {
        printf("I am a parallel thread %d\n", omp_get_thread_num());
    }
    return 0;
}
```

- The #pragma omp parallel directive tells the runtime to create a team of threads (usually one per CPU core).

Parallel Loop Example

```
#pragma omp parallel for
for (int i = 0; i < N; i++)
    c[i] = a[i] + b[i];
```

- The loop work is divided among the threads in the team.
- OpenMP also allows explicit control of the number of threads (omp_set_num_threads) and data-sharing attributes (private vs. shared).

🌀 Grand Central Dispatch (GCD) – macOS / iOS

> **Definition:** *GCD* combines language extensions (blocks), an API, and a runtime library that schedules work on a pool of POSIX threads.

Blocks Syntax (C/C++)

```
dispatch_queue_t queue = dispatch_get_global_queue(DISPATCH_QUEUE_PRIORITY_DEFAULT, 0);
```

```
dispatch_async(queue, ^{ printf("I am a block running on a GCD thread\n"); });
```

- **Blocks** (^{ … }) are self-contained units of work.
- **Serial queues** execute blocks FIFO, one at a time.
- **Concurrent queues** may run multiple blocks simultaneously.
- System-wide concurrent queues have three priority levels: **low**, **default**, **high**.

Internally, GCD manages a thread pool of POSIX threads, automatically scaling to match system demand.

📚 Other Implicit Threading Approaches

Approach	Language / Library	Key Feature
Threading Building Blocks (TBB)	C++	High-level algorithms and containers that abstract away explicit thread management.
Microsoft Parallel Patterns Library (PPL)	C++/C#	parallel_for, task_group, and async abstractions built on the Windows thread pool.
java.util.concurrent	Java	Executors, Future, CountDownLatch, etc., for implicit creation and coordination.

These frameworks aim to simplify concurrency while still exploiting multicore hardware.

⚙️ Fork & Exec in Multithreaded Programs

> **Definition:** fork() creates a new process; exec() replaces the current process image with a new program.

- **Two fork() variants:**
  1. **Full duplicate** – copies *all* threads into the child (rare; used when the child continues as a multithreaded program).
  2. **Thread-only duplicate** – copies only the calling thread (common in POSIX).
- **exec() semantics:** Regardless of how many threads existed before the call, exec() replaces the entire process image, discarding all threads.
- **Guideline:**
  - Use the *thread-only* fork() when the child will immediately exec() another program (no need to copy other threads).
  - Use the *full* fork() only when the child must retain the full thread set.

🚦 Signal Handling in Multithreaded Programs

> **Definition:** A *signal* is an asynchronous notification sent to a process (or thread) to inform it of an event such as SIGINT, SIGSEGV, or a timer expiration.

Delivery Options

Option	Description
Deliver to the thread that caused the signal	Typical for synchronous signals (e.g., division by zero).
Deliver to any thread that has not blocked the signal	Default for many asynchronous signals (e.g., SIGINT).
Deliver to a specific set of threads	Threads can mask (block) particular signals, limiting delivery.
Assign a single “signal-handling” thread	Using pthread_sigmask to block signals in all other threads and sigwait in the designated thread.

Synchronous vs. Asynchronous Signals

- **Synchronous signals** (generated by the faulting thread) are delivered directly to that thread.
- **Asynchronous signals** (generated by external events) may be delivered to any thread that does not block them, leading to nondeterministic handling unless explicitly managed.

Proper signal handling in multithreaded programs often involves:

1. **Blocking** the signal in all threads except a dedicated handler thread.
2. Using sigwait() or sigwaitinfo() in the handler thread to synchronously receive the signal.
3. Ensuring that shared data accessed from signal handlers is protected (e.g., via volatile sig_atomic_t or lock-free structures).

⚡ Signals in Multithreaded Environments

> **Definition:** A signal is an asynchronous notification sent to a process (or thread) to indicate an event such as termination (Ctrl-C) or a user-defined condition.

- The classic UNIX interface uses

  ```
  kill(pid_t pid, int signal);
  ```

  where **pid** identifies the target *process* and **signal** specifies the event.
- Modern UNIX variants let a thread *block* or *accept* specific signals.
  - If a signal is **blocked** in a thread, that thread will not receive it.
  - The kernel delivers an asynchronous signal to the **first** thread that does **not** have it blocked.
- POSIX adds a thread-specific API:

  ```
  pthread_kill(pthread_t tid, int signal);
  ```

  which directs the signal to a particular *thread* (tid).
- **Windows** does not have POSIX-style signals. Instead it provides **Asynchronous Procedure Calls (APCs)**:
  - An APC is queued to a specific thread and executes a user-supplied routine when the thread enters an alertable state.
  - APCs play the same role as UNIX signals but are inherently *thread-targeted* rather than process-wide.

❌ Thread Cancellation

> **Definition:** Thread cancellation terminates a thread before it has completed its normal execution.

Typical use-case: multiple worker threads search a database; once one finds the result, the others are cancelled.

Cancellation Scenarios

1. **Asynchronous cancellation** – the target thread is terminated immediately.
2. **Deferred cancellation** – the target thread periodically checks for a cancellation request and exits at a *cancellation point*.

Challenges

- Resources (memory, file descriptors) may be left unreleased if cancellation occurs while a thread holds them.
- Asynchronous cancellation can abort a thread while it is updating shared data, leading to inconsistency.
- The system usually reclaims the thread's kernel resources, but **application-level** resources must be cleaned up manually.

Pthreads Cancellation API

Attribute	Meaning
State	PTHREAD_CANCEL_ENABLE / PTHREAD_CANCEL_DISABLE
Type	PTHREAD_CANCEL_DEFERRED / PTHREAD_CANCEL_ASYNCHR

- The default is **deferred** cancellation.
- A thread can change its state with pthread_setcancelstate() and its type with pthread_setcanceltype().
- Cancellation is *requested* with pthread_cancel(tid). The request remains pending until the target reaches a cancellation point (e.g., pthread_testcancel(), read(), pthread_join()).

Example (deferred cancellation)

```
while (1) {
    /* perform work */
    pthread_testcancel();   /* cancellation point */
}
```

- When a cancellation request is pending, the runtime invokes any registered *cleanup handlers* so that the thread can release its own resources before termination.

> **Note:** On Linux, Pthreads cancellation is implemented using signals (see Section 4.6.2).

📦 Thread-Local Storage (TLS)

> **Definition:** TLS is data that is **unique to each thread** but persists across multiple function calls within that thread.

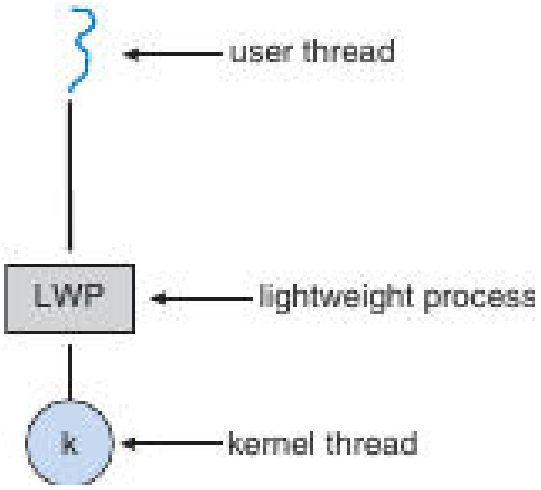
- Unlike ordinary local variables (which exist only for the duration of a single function call), TLS lives for the entire lifetime of the thread.
- Typical use: per-thread identifiers, transaction-specific buffers, or any state that must not be shared.

Platform	TLS Mechanism
POSIX	pthread_key_create(), pthread_getspecific(), pthread_setspecific()
Windows	TlsAlloc(), TlsGetValue(), TlsSetValue()
Java	ThreadLocal class

🔄 Scheduler Activations & Lightweight Processes (LWPs)

> **Definition:** Scheduler activations provide a communication channel between the kernel and a user-level thread library, enabling the library to manage *virtual processors* (LWPs).

- **LWPs** act as lightweight bridges: each LWP is bound to a kernel thread and appears to the user-level library as a *virtual processor* onto which user threads are scheduled.
- When a user thread blocks (e.g., on I/O), the kernel issues an **upcall** to the library, notifying it that a virtual processor will become idle.
- The library's upcall handler can then:
  1. Save the state of the blocked thread.
  2. Allocate a new LWP (if needed) or reuse an idle one.
  3. Schedule another ready user thread onto the newly available virtual processor.
- A second upcall occurs when the blocked operation completes, allowing the library to mark the original thread as runnable and schedule it.



The diagram shows the relationship between a user thread, its lightweight process (LWP), and the underlying kernel thread.

- Scheduler activations enable many-to-many or two-level many-to-many threading models to adapt dynamically to the number of available physical CPUs and I/O-bound workloads.

Windows Thread Implementation

> **Definition:** In the Windows family (95, NT, 2000, XP, 7, ...), each thread is represented by a set of kernel and user-mode structures.

Structure	Role
ETHREAD	Executive thread block (kernel-space)
KTHREAD	Kernel-mode thread control block (scheduling, kernel stack)
TEB	Thread Environment Block (user-space), holds thread ID, user stack, TLS, etc.

- **One-to-one** mapping: every user-level thread corresponds to a distinct kernel thread.
- The kernel maintains the full thread state (register set, stacks) and schedules them independently on multiple processors.

Linux Thread Implementation

> **Definition:** Linux treats both processes and threads as **tasks**, represented by struct task_struct.

- Threads are created with the clone() system call, which takes a set of flags to specify which resources are shared with the parent.

Flag	Shared Resource
CLONE_FS	File-system information (cwd, root)
CLONE_VM	Virtual memory space
CLONE_SIGHAND	Signal handlers
CLONE_FILES	Open file table

- Example flag combination (CLONE_FS | CLONE_VM | CLONE_SIGHAND | CLONE_FILES) creates a thread that shares the same address space, file descriptors, and signal handlers as its parent – the typical POSIX thread behavior.
- If **no** flags are set, clone() behaves like fork(), creating a separate process with its own copies of all resources.

- The kernel holds a unique task_struct for each task, containing pointers to the actual shared data structures (e.g., memory descriptors, file tables). This indirection enables fine-grained sharing as dictated by the clone flags.

🧩 Multithreaded Sudoku Validation

> *Definition:* A **worker thread** receives a portion of the Sudoku grid, checks the validity of its assigned rows, columns, or sub-grids, and reports the result back to the **parent thread**.

- **Parameter passing**
  - Allocate a structure with malloc(sizeof(parameters)) containing the coordinates (row, column, etc.).
  - Pass a pointer to this structure when creating the thread (pthread_create on POSIX, CreateThread on Windows).
- **Result communication**
  - Create a **global integer array** result[N] (where N equals the number of worker threads).
  - Index i of the array corresponds to worker thread *i*.
  - After checking, the worker stores 1 (valid) or 0 (invalid) in result[i].
  - The parent thread waits for all workers to finish, then scans the array; the Sudoku puzzle is valid only if every entry equals 1.

📊 Multithreaded Sorting Application

> *Definition:* A program that splits an unsorted array into two halves, sorts each half concurrently, and merges the results with a third thread.

- **Data layout**
  - **Global array** A[size] holds the original data.
  - **Second global array** B[size] receives the merged, sorted output.
- **Thread responsibilities**
  - **Sorting Thread 0** – sorts A[0 … size/2 - 1].
  - **Sorting Thread 1** – sorts A[size/2 … size-1].
  - **Merging Thread** – combines the two sorted sub-lists into B.
- **Parameter passing**
  - Each sorting thread receives the **starting index** and **length** of the segment it must sort (see the earlier “passing parameters to a thread” guidelines).
- **Final output**
  - After the merging thread finishes, the parent thread prints the sorted array from B.

🔄 Process Synchronization Overview

> *Definition:* **Cooperating processes** (or threads) share data and must coordinate their actions to avoid **data inconsistency**.

- **Typical problems** – race conditions, lost updates, and inconsistent views of shared resources.
- **Goal of synchronization** – enforce orderly access to critical resources while preserving progress and fairness.

⚙️ The Critical-Section Problem

> *Definition:* A **critical section** is a segment of code that accesses shared variables and must not be executed by more than one process (or thread) at the same time.

Requirement	Meaning

Mutual exclusion	No two processes may be in their critical sections simultaneously.
Progress	If no process is in a critical section, a requesting process will eventually enter.
Bounded waiting	There is an upper bound on how many times other processes may enter their critical sections before a requesting process is granted entry.

The typical process structure:

```
do {
    entry_section;
    critical_section;
    exit_section;
    remainder_section;
} while (true);
```

🤝 Peterson's Solution (Two-Process Algorithm)

> *Definition:* A classic software algorithm that satisfies mutual exclusion, progress, and bounded waiting for **exactly two** processes.

Key shared variables:

```
int turn;
bool flag[2];
```

- flag[i] → *true* when process *i* wishes to enter its critical section.
- turn → indicates whose turn it is to enter.

Algorithm (process i, where j = 1-i):

```
do {
    flag[i] = true;        // indicate interest
    turn = j;              // give priority to the other process
    while (flag[j] && turn == j) ;   // busy-wait
    /* critical section */
    flag[i] = false;       // exit protocol
    /* remainder section */
} while (true);
```

- Guarantees mutual exclusion because at least one of the two conditions (!flag[j] or turn != j) must be true before a process proceeds.
- Works on architectures where reads/writes to a single word are atomic; it may fail on modern CPUs with aggressive reordering.

🛠️ Synchronization Hardware Primitives

Primitive	Atomic behavior	Typical usage
Test-and-Set	Reads a Boolean, sets it to *true* in a single indivisible step.	Implements simple spin locks.
Compare-and-Swap (CAS)	Replaces a memory word with a new value only if it currently holds an expected value; returns the original word.	Basis for lock-free data structures and bounded-waiting algorithms.

Both primitives must execute **atomically** across all CPUs; the hardware guarantees that concurrent invocations are serialized in some order.

🧩 Test-and-Set Example

> *Definition:* test_and_set(&lock) returns the previous value of lock and sets lock to *true* atomically.

```
bool test_and_set(bool *target) {
    bool rv = *target;
    *target = true;
    return rv;
}
```

Spin-lock pattern:

```
while (test_and_set(&lock)) ;   // busy-wait until lock becomes free
/* critical section */
lock = false;                    // release
```

- Inefficient on multiprocessors because the busy loop consumes CPU cycles and may cause cache-line ping-pong.

🔄 Compare-and-Swap (CAS) Example

> *Definition:* compare_and_swap(&var, expected, new) atomically sets *var to new only if *var == expected; returns the original value.

```
int compare_and_swap(int *value, int expected, int new) {
    int temp = *value;
    if (*value == expected)
        *value = new;
    return temp;
}
```

CAS-based lock:

```
while (compare_and_swap(&lock, 0, 1) != 0) ;   // acquire
/* critical section */
lock = 0;                                      // release
```

- Eliminates the need for a separate “test-then-set” sequence, reducing the window for race conditions.

⏱️ Bounded-Waiting Algorithm Using Test-and-Set

> *Definition:* An extension of the basic test-and-set lock that guarantees each process will wait at most *n-1* turns (where *n* is the number of processes).

Shared data:

```
bool waiting[N];   // N = number of processes
bool lock = false;
```

Algorithm for process *i*:

```
do {
    waiting[i] = true;
    while (waiting[i]) {
        if (!lock && test_and_set(&lock) == false) {
            waiting[i] = false;          // acquire lock
        }
    }
    /* critical section */
    lock = false;                        // release
    waiting[i] = false;                  // allow next waiting process
    /* remainder section */
} while (true);
```

- The waiting array ensures that a process can be overtaken by at most *N-1* others before it gains the lock, satisfying the bounded-waiting requirement.

🔐 Mutex Locks (Software-Level)

> *Definition:* A **mutex** (mutual-exclusion lock) abstracts the hardware primitives into a simple acquire/release API.

Typical operations:

- acquire() – blocks the calling thread until the mutex becomes available, then marks it as held.
- release() – makes the mutex available for other threads.

Implementation sketch (using a Boolean lock and a test-and-set primitive):

```
bool lock = false;

void acquire(void) {
    while (test_and_set(&lock)) ;   // spin until lock is obtained
}

void release(void) {
    lock = false;                    // free the mutex
}
```

- In user-level libraries (e.g., POSIX pthread_mutex_t), the acquire/release functions may also block the thread (instead of spinning) to improve CPU utilization.
- A mutex satisfies **mutual exclusion**; additional fairness policies (e.g., FIFO ordering) are often added by the OS to meet **progress** and **bounded waiting**.

🔒 Mutexes & Spinlocks

> **Mutex** – a mutual-exclusion lock that guarantees only one process (or thread) can be in its critical section at a time.

- **acquire()** – attempts to obtain the lock; if the lock is unavailable the caller *spins* (busy-wait) until it becomes free.
- **release()** – atomically sets the lock to *available* so that another waiting process can acquire it.

```
/* simplified spin-lock */
bool lock = false;

void acquire(void) {
    while (test_and_set(&lock)) ;   // spin while lock is true
}
```

```
void release(void) {
    lock = false;                    // make lock available
}
```

- **Busy waiting** wastes CPU cycles because the waiting process repeatedly reads the lock variable.
- **Advantages** – no context switch while waiting, which is useful when the lock is held only a very short time (e.g., on a multiprocessor where one core can spin while another holds the lock).
- **Disadvantages** – on a single-CPU system the spinning process prevents any other process from making progress; the CPU is consumed without doing useful work.

Spinlocks are therefore preferred for short critical sections on multiprocessor machines.

🟢 Semaphores

> **Semaphore** – an integer variable accessed only through two atomic operations, **wait()** (P) and **signal()** (V).

Type	Value Range	Typical Use
Binary	0 or 1	Acts like a mutex for mutual exclusion
Counting	0 … ∞	Controls access to a pool of identical resources

Primitive Operations

```
/* wait (P) */
void wait(semaphore *S) {
    while (S->value <= 0) ;   // busy-wait
    S->value--;
}

/* signal (V) */
void signal(semaphore *S) {
    S->value++;
}
```

- Both operations must be **indivisible**: while one process is executing wait() or signal(), no other process may modify the same semaphore.
- The original names *P* (from Dutch “proberen”, to test) and *V* (from “verhogen”, to increment) reflect this history.

Binary vs. Counting

- A **binary semaphore** can replace a mutex when the system provides only semaphores.
- A **counting semaphore** is initialized to the number of available resources; each wait() consumes one resource, each signal() releases one.

⚙️ Semaphore Implementation (Blocking Queue)

> **Goal** – eliminate busy waiting by putting a process to sleep when the semaphore is unavailable.

```
typedef struct {
    int value;                   // current count
    queue *list;                 // queue of blocked PCBs
} semaphore;
```

- **wait(semaphore *S)**

1. Decrement S->value.
2. If the result is negative, place the calling process on S->list and **block** it (state → *waiting*).

- **signal(semaphore *S)**
  1. Increment S->value.
  2. If the new value ≤ 0, remove one process from S->list and **wake up** that process (state → *ready*).

Atomicity is achieved by disabling interrupts (single-CPU) or by using hardware primitives such as **compare-and-swap** on multiprocessors.

⚠️ Deadlocks & Starvation

> **Deadlock** – a set of processes where each is waiting for an event that only another member of the set can cause.

- Classic example: two processes each hold one semaphore and wait for the other (wait(S); wait(Q); … signal(S); signal(Q);).
- No process can proceed → system is deadlocked.

> **Starvation** – a process waits indefinitely because the scheduling or synchronization policy never grants it the needed resource.

- Can arise from **LIFO** waiting queues (newer processes repeatedly pre-empt older ones).
- Using a **FIFO** queue for semaphore wait lists helps guarantee bounded waiting.

Issue	Symptom	Typical Remedy
Deadlock	All involved processes are blocked	Enforce a global ordering of resource acquisition, or use a timeout/recovery protocol
Starvation	One or more processes never obtain the resource	FIFO queues, priority inheritance, or fair scheduling algorithms

---

🔼 Priority Inversion

> **Priority inversion** – a low-priority process holds a resource needed by a high-priority process, while a medium-priority process pre-emptively runs, delaying the low-priority holder.

Real-world case: *Mars Pathfinder* (1997) suffered frequent resets because a high-priority task waited for a resource owned by a low-priority task that was pre-empted by medium-priority tasks.

Solution – Priority Inheritance:

- When a higher-priority task blocks on a lock held by a lower-priority task, the holder **temporarily inherits** the higher priority.
- The inherited priority prevents intermediate-priority tasks from pre-empting the holder, allowing it to release the lock sooner.

📦 Classic Synchronization Problems

🏭 Bounded-Buffer (Producer-Consumer)

Shared data structures:

```
int n;                    // buffer size
semaphore mutex = 1;      // protects buffer access
semaphore empty = n;      // counts empty slots
semaphore full  = 0;      // counts filled slots
```

Producer

```
do {
    wait(empty);
    wait(mutex);
    /* add item to buffer */
    signal(mutex);
    signal(full);
} while (true);
```

Consumer

```
do {
    wait(full);
    wait(mutex);
    /* remove item from buffer */
    signal(mutex);
    signal(empty);
} while (true);
```

- mutex guarantees mutual exclusion; empty and full enforce the buffer capacity constraints.

📖 Readers–Writers

- **First readers-writers problem** – readers may proceed unless a writer is already active.
- **Second readers-writers problem** – once a writer is waiting, no new readers may start.

Common data structures:

```
semaphore rw_mutex = 1;   // protects the shared resource for writers
semaphore mutex   = 1;    // protects read_count
int read_count = 0;
```

Reader

```
do {
    wait(mutex);
    read_count++;
    if (read_count == 1) wait(rw_mutex);   // first reader locks writers out
    signal(mutex);

    /* reading section */

    wait(mutex);
    read_count--;
    if (read_count == 0) signal(rw_mutex); // last reader releases writers
    signal(mutex);
} while (true);
```

Writer

```
do {
    wait(rw_mutex);
    /* writing section */
    signal(rw_mutex);
} while (true);
```

- The first reader acquires rw_mutex; subsequent readers proceed without further blocking.
- When the last reader exits, rw_mutex is released, allowing a waiting writer to proceed.

🪐 Dining Philosophers

- **Resources:** five chopsticks, each represented by a binary semaphore chopstick[i] initialized to 1.

```
do {
    wait(chopstick[i]);              // pick up left chopstick
    wait(chopstick[(i+1) % 5]);      // pick up right chopstick
    /* eat */
    signal(chopstick[i]);            // put down left
    signal(chopstick[(i+1) % 5]);    // put down right
    /* think */
} while (true);
```

Deadlock prevention strategies

Strategy	Idea
Limit concurrent philosophers	Allow at most four philosophers to sit simultaneously.
Resource ordering	Require each philosopher to pick up the lower-numbered chopstick first.
Asymmetric solution	Even-numbered philosophers pick left then right; odd-numbered pick right then left.
Acquire both chopsticks inside a *single* critical section (using an extra mutex).	

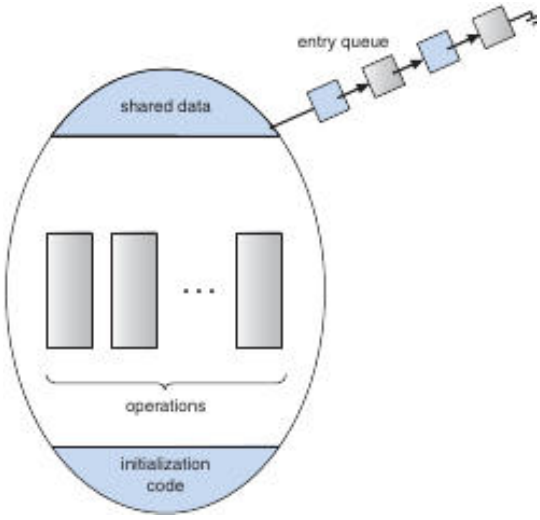
Even with deadlock avoidance, **starvation** must still be considered; fairness mechanisms (e.g., FIFO queues) are needed.

🖥️ Monitors & Condition Variables

> **Monitor** – a high-level synchronization construct that combines a lock with *condition variables*; only one process can execute inside the monitor at any time.

```
monitor Buffer {
    int count = 0;
    condition notFull, notEmpty;

    void put(item) {
        if (count == MAX) wait(notFull);
        /* add item */
        count++;
        signal(notEmpty);
    }

    item get() {
        if (count == 0) wait(notEmpty);
        /* remove item */
        count--;
        signal(notFull);
    }
}
```

- **Condition variable operations**
  - x.wait(); – the calling process is suspended and placed on x's waiting queue.
  - x.signal(); – wakes **exactly one** process waiting on x (if any).
- Unlike semaphores, a signal() has no effect when no process is waiting; the condition's state simply remains unchanged.



The figure shows a monitor's internal layout: shared data at the top, initialization code at the bottom, and three operation blocks in the middle. An entry queue on the right holds processes waiting to enter the monitor.

- The monitor's **implicit lock** ensures that only one process executes any of its functions at a time, removing the need for explicit wait(mutex)/signal(mutex) pairs.
- Condition variables enable processes to *block* inside the monitor while waiting for a particular state (e.g., buffer not full).

Monitors Overview

> **Definition:** A *monitor* is a high-level synchronization construct that encapsulates shared data, the procedures that operate on that data, and the condition variables used to coordinate access. Only one process (or thread) may be active inside the monitor at any time.

- **Signal semantics**
  1. **Signal-and-wait** – The signaling process **waits** until the waiting process leaves the monitor.
  2. **Signal-and-continue** – The signaling process **continues** immediately; the waiting process is resumed only after the signaling process exits the monitor.

  Both approaches have arguments in their favor. The former can guarantee that the condition still holds when the waiting process runs, while the latter reduces the time the signaling process is blocked.

  Compromise: Languages such as **Concurrent Pascal** adopt a hybrid rule: after executing signal, the signaling thread **immediately leaves** the monitor, allowing the waiting thread to be resumed.

Dining-Philosophers Solution Using Monitors

- **Problem:** Prevent deadlock while allowing each of the five philosophers to eat only when both adjacent chopsticks are free.
- **State enumeration**

```
enum { THINKING, HUNGRY, EATING } state[5];
condition self[5];
```

- **Monitor operations**

```
void pickup(int i) {
    state[i] = HUNGRY;
```

```
    test(i);
    if (state[i] != EATING) self[i].wait();
}

void putdown(int i) {
    state[i] = THINKING;
    test((i+4) % 5);   // left neighbor
    test((i+1) % 5);   // right neighbor
}

void test(int i) {
    if (state[i] == HUNGRY &&
        state[(i+4)%5] != EATING &&
        state[(i+1)%5] != EATING) {
        state[i] = EATING;
        self[i].signal();
    }
}
```

- **Correctness properties**
  - No two neighboring philosophers can be in **EATING** simultaneously (mutual exclusion).
  - The solution is **deadlock-free** because a philosopher never holds a single chopstick while waiting for the other.
  - **Starvation** is not addressed; guaranteeing progress for every hungry philosopher is left as an exercise.

⚙️ Implementing a Monitor Using Semaphores

- **Core semaphores**
  - mutex (initialized to 1) protects entry to the monitor.
  - next (initialized to 0) and integer next_count manage the hand-off between a signaling thread and the resumed thread.
- **Entry/exit protocol** (for any monitor operation F):
  1. wait(mutex); →enter monitor.
  2. Execute the body of F.
  3. If next_count > 0 then signal(next); else signal(mutex); →leave monitor.
- **Condition variable implementation** (for condition x):
  - Semaphore x_sem (0) and integer x_count (0).
  - **wait(x)**

```
x_count++;
if (next_count > 0) signal(next);
else signal(mutex);
wait(x_sem);
x_count--;
```

  - **signal(x)**

```
if (x_count > 0) {
    next_count++;
    signal(x_sem);
    wait(next);
    next_count--;
} else {
    // no waiting process
}
```

- This scheme works for both **Hoare** (immediate hand-off) and **Mesa** (signal-and-continue) monitor models; the choice is reflected in whether the signaling thread waits on next.

🔁 Process-Resumption Order in Monitors

- **FCFS (first-come, first-served)** is the simplest policy: the longest-waiting process is resumed first.
- **Priority-based resumption** uses x.wait(c) where c is an integer priority evaluated at wait time. The x.signal() operation resumes the waiting process with the **smallest** c.

ResourceAllocator Example

```
monitor ResourceAllocator {
    boolean busy;
    condition x;
    void acquire(int time) {
        if (busy) x.wait(time);
        busy = true;
    }
    void release() {
        busy = false;
        x.signal();
    }
}
```

- Processes request a resource for a maximum duration time. The monitor grants the resource to the request with the **shortest** time (lowest priority number).
- Correctness requires:
    1. All user processes must invoke acquire/release in the proper order.
    2. No process may bypass the monitor to access the shared resource directly.

☕ Java Monitors

> **Definition:** In Java, every object possesses an intrinsic monitor lock. Declaring a method synchronized makes the executing thread **acquire** that object's lock before entering the method and **release** it upon exit.

- **Lock acquisition** – If another thread already holds the lock, the calling thread blocks in the object's *entry set* (wait queue).
- **wait() / notify()** – Correspond to monitor wait and signal. A thread calling wait() releases the object's lock and is placed on the condition queue; notify() wakes one waiting thread, notifyAll() wakes all.
- **java.util.concurrent** – Provides higher-level constructs (semaphores, locks, condition variables, thread pools) that build on the same underlying monitor mechanisms.

🖥️ Synchronization Examples Across Operating Systems

🔷 Windows

- **Dispatcher objects** (mutexes, semaphores, events, timers) exist in either **signaled** or **nonsignaled** state.
- **Mutex** – Only one thread may own it; other threads block until it becomes signaled.
- **Critical-section object** – User-mode mutex that first spins, then falls back to a kernel mutex if contention persists, reducing kernel overhead for short critical sections.
- **Spinlocks** – Used inside the kernel; a thread holding a spinlock is never pre-empted.

🔷 Linux

- **Atomic integers** (atomic_t) provide lock-free operations (atomic_set, atomic_add, atomic_inc, atomic_read).
- **Mutexes** – mutex_lock() / mutex_unlock() protect kernel critical sections; a blocked thread sleeps and is awakened when the lock is released.
- **Spinlocks** – Preferred for very short sections on SMP systems; on single-processor systems the kernel disables preemption instead of spinning.
- **Preempt control** – preempt_disable() / preempt_enable() and a per-task preempt_count ensure that code holding a lock cannot be pre-empted.

🔷 Solaris

- **Adaptive mutexes** – Spin briefly if the lock holder is running on another CPU; otherwise the thread sleeps.
- **Turnstiles** – Queues attached to a lock that order waiting threads; each kernel thread owns its own turnstile, which can be reused for multiple locks.
- **Priority inheritance** – When a high-priority thread blocks on a lock held by a lower-priority thread, the holder temporarily inherits the higher priority, preventing priority inversion.
- **Reader–writer locks** – Allow concurrent reads while exclusive writes serialize access; more expensive than semaphores but beneficial for read-heavy workloads.

🔷 Pthreads (User-level)

- **Mutexes** – Created with pthread_mutex_init; acquired with pthread_mutex_lock, released with pthread_mutex_unlock.
- **Condition variables** – pthread_cond_wait and pthread_cond_signal implement the monitor wait/signal pattern.
- **Semaphores** (POSIX extension) – Unnamed (sem_init) and named (sem_open) variants; sem_wait / sem_post correspond to classic wait / signal.
- **Spinlocks** – Available via pthread_spin_init, pthread_spin_lock, pthread_spin_unlock; suitable for short critical sections on multiprocessor systems.

🔄 Alternative Approaches

💾 Transactional Memory

> **Definition:** *Transactional memory* allows a block of memory operations to execute atomically: either the entire block **commits** or **aborts** and rolls back, without explicit lock acquisition.

- **Programming model** – An atomic { … } block encloses the critical code. Example (conceptual):

```
void update() {
    atomic {
        /* modify shared data */
    }
}
```

- **Advantages**
  - Eliminates classic lock-based deadlocks.
  - Simplifies reasoning about concurrency; the runtime determines conflicts.
  - Can improve scalability under high contention.
- **Implementations**
  - **Software Transactional Memory (STM):** Instrumented by the compiler; manages read/write logs and validates at commit time.
  - **Hardware Transactional Memory (HTM):** Leverages cache-coherency protocols to detect conflicts with lower overhead, but requires processor support and modifications to the cache hierarchy.

🛠️ OpenMP Parallelism (brief)

- Provides compiler directives (e.g., #pragma omp parallel for) for shared-memory parallelism.
- Integrates with existing synchronization primitives (critical sections, atomic clauses, locks) to manage data races in multithreaded loops.

🟢 OpenMP Parallelism (Shared-Memory)

> **OpenMP** is a set of compiler directives, library routines, and environment variables that enable parallel programming in a shared-memory environment.

- The directive #pragma omp parallel defines a **parallel region**.
- By default the runtime creates a team of threads equal to the number of processing cores.
- Thread creation and management are handled by the OpenMP library, relieving the application developer of low-level bookkeeping.

Advantages

- Simplifies parallel code: only the region to be executed concurrently needs to be marked.
- Portable across many Unix-like and Windows compilers that support the OpenMP API.

🔐 OpenMP Critical Sections

> **Critical section** – a region of code that must be executed by only one thread at a time.

OpenMP provides the #pragma omp critical directive to protect shared data.
The construct behaves like a binary semaphore (or mutex) that guarantees mutual exclusion.

```
/* OpenMP critical-section example */
void update(int value) {
    #pragma omp critical
    {
        counter += value;   // only one thread modifies `counter` at a time
    }
}
```

- If a thread attempts to enter a critical section while another thread holds it, the caller **blocks** until the owner exits.
- Multiple named critical sections can be defined; each name has its own independent lock.
- **Pros:** Simple to use, no explicit lock handling.
- **Cons:** Still subject to deadlock if a thread acquires several critical sections in an inconsistent order.

🧩 Functional Programming Languages

> **Functional languages** emphasize immutable data and stateless computation.

Aspect	Imperative (e.g., C, C++, Java)	Functional (e.g., Erlang, Scala)
State	Mutable variables; state changes are explicit.	Variables are immutable; once bound they cannot change.
Concurrency concerns	Race conditions and deadlocks must be managed with locks, semaphores, etc.	Immutable data eliminates the need for mutual-exclusion primitives.
Typical use	General-purpose system and application programming.	Highly concurrent or distributed systems where safety is paramount.

- **Erlang** is renowned for lightweight processes and built-in message passing, making it suitable for large-scale concurrent applications.
- **Scala** blends object-oriented and functional paradigms; its syntax resembles Java while offering immutable collections and powerful concurrency abstractions.

⚔️ Deadlocks Overview

> **Deadlock** – a situation where a set of processes (or threads) are each waiting for an event that only another member of the set can cause, causing all of them to remain blocked forever.

Four Necessary Conditions (all must hold simultaneously)

#	Condition	Meaning
1	**Mutual exclusion**	At least one resource can be held by only one process at a time.

2	**Hold and wait**	A process holds one or more resources while waiting for additional ones.
3	**No preemption**	Resources cannot be forcibly taken away; they must be released voluntarily.
4	**Circular wait**	A circular chain of processes exists, each waiting for a resource held by the next.

If any condition is broken, deadlock cannot occur.

🗂️ System Model & Resource Types

- The system contains a **finite set of resource types**; each type may have multiple identical instances (e.g., 2 CPUs, 5 printers).
- A **resource class** groups instances that are interchangeable from the point of view of a process.
- Processes follow the **request → use → release** sequence for each resource they need.

Example Resource Types

Type	Instances	Notes
CPU cycles	2	Two instances → two CPUs.
Printers	5	May be treated as one class if any printer suffices; otherwise separate classes for location-specific printers.
Memory pages	many	Logical resources managed by the OS.

📊 Resource-Allocation Graph (RAG)

- A directed graph with **process vertices** (P) and **resource-type vertices** (R).
- **Assignment edge** R → P indicates an instance of R is allocated to P.
- **Request edge** P → R indicates P is waiting for an instance of R.

Interpretation Rules

- **Cycle with single-instance resources** ⇒ deadlock.
- **Cycle with multi-instance resources** ⇒ may indicate deadlock but is not sufficient; additional analysis required.

🧩 Deadlock Example Using Pthread Mutexes

The following C code demonstrates a classic deadlock caused by acquiring two mutexes in opposite order.

```
/* Pthread deadlock illustration */
#include
#include
#include

pthread_mutex_t first_mutex, second_mutex;

/* Thread 1 */
void *do_work_one(void *arg) {
    pthread_mutex_lock(&first_mutex);
    /* simulate work */
    pthread_mutex_lock(&second_mutex);
    /* critical section */
    pthread_mutex_unlock(&second_mutex);
    pthread_mutex_unlock(&first_mutex);
    return NULL;
```

```
}

/* Thread 2 */
void *do_work_two(void *arg) {
    pthread_mutex_lock(&second_mutex);
    /* simulate work */
    pthread_mutex_lock(&first_mutex);
    /* critical section */
    pthread_mutex_unlock(&first_mutex);
    pthread_mutex_unlock(&second_mutex);
    return NULL;
}
```

- Thread 1 locks first_mutex then second_mutex.
- Thread 2 locks second_mutex then first_mutex.
- If the two threads interleave such that each holds its first lock while waiting for the second, a circular wait forms → deadlock.

Avoidance tip: impose a **global lock acquisition order** (e.g., always lock first_mutex before second_mutex).

📉 Detecting & Handling Deadlocks

Detection Strategies

- **Resource-allocation-graph cycle detection**: periodic traversal to find cycles.
- **Wait-for graph** (process-level abstraction): edges represent "process i is waiting for process j".

If a cycle is found, the system can invoke recovery actions.

Recovery Techniques

1. **Process termination** – abort one or more involved processes to break the cycle.
2. **Resource preemption** – forcibly reclaim a resource from a process (requires careful state handling).
3. **Rollback** – revert processes to a safe checkpoint before the deadlock formed.

🛡️ Deadlock Prevention & Avoidance

Approach	How it Works	Example
Prevention	Enforce a rule that invalidates at least one of the four necessary conditions.	Disallow hold-and-wait by requiring processes to request all needed resources atomically.
Avoidance	Dynamically examine future requests using additional information (e.g., maximum resource need) and only grant a request if the system remains in a safe state.	**Banker's algorithm**: simulates allocation before committing.
Combined	Apply prevention for high-risk resources and avoidance for others, balancing overhead and safety.	Use a strict ordering for mutexes (prevention) while employing the Banker's algorithm for I/O devices.

🔄 Recovery After Deadlock Detection

1. **Select victim(s)** – choose one or more processes to terminate or roll back (often the one with least work done).
2. **Release resources** held by victims.
3. **Restart affected processes** (if possible) after resources become available.

Operating systems such as Linux and Windows typically **do not implement automatic deadlock recovery**; they rely on the application to avoid or resolve deadlocks, making careful design essential.

📚 Summary of Key Points (Reference)

- **OpenMP** simplifies shared-memory parallelism; #pragma omp critical provides a high-level mutual-exclusion construct.
- **Functional languages** avoid traditional synchronization problems by enforcing immutability.
- **Deadlocks** require all four classic conditions; breaking any one eliminates the possibility.
- **Resource-allocation graphs** give a visual method to reason about potential deadlocks; cycles with single-instance resources guarantee deadlock.
- **Mutex-order violations** (as shown in the pthread example) are a common source of deadlock; consistent ordering prevents the circular-wait condition.
- **Prevention, avoidance, and detection/recovery** are the three fundamental strategies for handling deadlocks; most real-world systems combine them based on resource characteristics.

🌀 Spinlocks vs Mutexes

> **Spinlock:** a lock that repeatedly tests a variable (busy-wait) until it becomes free.
> **Mutex:** a lock that puts waiting threads to sleep, releasing the CPU.

- **When to prefer a mutex:**
  1. Critical section may hold the lock for an *unbounded* time.
  2. System wants to avoid wasting CPU cycles on busy-waiting.
- **When a spinlock can be advantageous:**
  - The lock is held only for a *very short* interval (e.g., updating a single word).
  - Upper-bound time is known, so the cost of spinning is less than the context-switch overhead of a mutex.

Property	Spinlock	Mutex
CPU usage while waiting	High (busy-wait)	Low (blocked)
Suitable for short critical sections	✅	✅ (but slower)
Guarantees FIFO ordering of waiters	❌	✅ (implementation-dependent)

📈 Counter Synchronization Strategies

Two ways to protect the **hits** variable in a multithreaded web server:

Strategy	Code sketch	Synchronization primitive	Expected efficiency
Mutex lock	hit_lock.acquire(); hits++; hit_lock.release();	pthread_mutex (or OS mutex)	Guarantees mutual exclusion; incurs lock/unlock overhead.
Atomic integer	atomic_inc(&hits);	Hardware-supported atomic operation (fetch_add)	No explicit lock; typically faster for simple increments.

> **Observation:** For a single-word increment, the *atomic* version is usually more efficient because it avoids the kernel-mode transition and context switches required by a mutex.

🔧 Race Conditions in Process Allocation (Figure 5.27)

Problem:

```
int number_of_processes = 0;
int allocate_process() {
    if (number_of_processes == MAX_PROCESSES) return -1;
    ++number_of_processes;
    return new_pid;
}
void release_process() {
```

```
    --number_of_processes;
}
```

a. Identified race conditions

- Two threads may simultaneously test number_of_processes == MAX_PROCESSES and both proceed, exceeding the limit.
- Increment/decrement of number_of_processes is not atomic → lost updates.

b. Fix with a mutex

```
pthread_mutex_t proc_lock = PTHREAD_MUTEX_INITIALIZER;

int allocate_process() {
    pthread_mutex_lock(&proc_lock);
    if (number_of_processes == MAX_PROCESSES) {
        pthread_mutex_unlock(&proc_lock);
        return -1;
    }
    ++number_of_processes;
    pthread_mutex_unlock(&proc_lock);
    return new_pid;
}
void release_process() {
    pthread_mutex_lock(&proc_lock);
    --number_of_processes;
    pthread_mutex_unlock(&proc_lock);
}
```

c. Using an atomic integer

- Replacing int number_of_processes with atomic_int **eliminates the race** for the increment/decrement, but the *capacity check* (== MAX_PROCESSES) still requires a critical section to be safe.
- Therefore a full replacement is **not sufficient**; a lock (or compare-and-swap loop) is still needed for the bound check.

🌐 Controlling Concurrent Connections with Semaphores

> **Semaphore:** an integer counter with atomic wait() (P) and signal() (V) operations.

Server scenario: Allow at most **N** simultaneous socket connections.

```
sem connection_sem = N;   // initial count = N

/* On new connection request */
wait(connection_sem);    // blocks if N connections already active
accept_connection();     // proceed with socket handling

/* When a client disconnects */
close_connection();
signal(connection_sem);  // frees a slot for another client
```

- The semaphore **blocks** excess clients until a slot becomes free, guaranteeing the limit without busy-waiting.

🪟 Windows Vista Slim Reader-Writer (SRW) Locks

- **SRW lock** is a lightweight synchronization primitive introduced in Windows Vista.
- **Key properties:**
  - Supports *shared* (reader) and *exclusive* (writer) modes.
  - Does **not** enforce FIFO ordering; threads are scheduled by the kernel scheduler.

- No explicit “fairness” guarantee, but provides better performance than traditional CRITICAL_SECTION when many readers coexist.

🛠️ Implementing wait() / signal() with Test-and-Set (Multiprocessor)

Goal: minimal instruction sequence.

```
/* lock is a Boolean initialized to false */
void wait(semaphore *S) {
    while (true) {
        while (test_and_set(&S->lock)) ;   // acquire the spinlock
        if (S->value > 0) {
            --S->value;
            S->lock = false;               // release spinlock
            break;
        }
        S->lock = false;                   // release spinlock
        // optional pause or back-off before retry
    }
}

void signal(semaphore *S) {
    while (test_and_set(&S->lock)) ;       // acquire spinlock
    ++S->value;
    S->lock = false;                       // release
}
```

- The **test-and-set** primitive guarantees atomic acquisition of the internal spinlock; the outer loops implement the semaphore semantics without kernel calls.

👶 Parent-Child Thread Coordination (Fibonacci)

Original requirement: parent waits (pthread_join) before printing the Fibonacci numbers.

Modified design to allow early access:

- Use a **shared buffer** protected by a **mutex** and a **condition variable**.
- Child thread writes each newly computed Fibonacci value, then pthread_cond_signal.
- Parent thread loops, waiting on the condition variable, prints values as soon as they become available, and continues until the child signals completion (e.g., with a special sentinel).

```
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  cv  = PTHREAD_COND_INITIALIZER;
int fib_val;
bool done = false;
```

- This eliminates the need for the parent to block until the child terminates.

🔁 Monitors vs Semaphores Equivalence

- **Monitor:** encapsulates shared data, procedures, and condition variables; only one process may be active inside the monitor at a time.
- **Semaphore:** provides wait() and signal() primitives that can be used to build the same mutual-exclusion and waiting behavior.

Construction sketch:

- A binary semaphore (mutex = 1) implements the monitor’s implicit lock.
- Each monitor condition variable is represented by a **queue** of waiting processes plus a **counting semaphore** (cond_sem = 0).
- wait(C) → signal(mutex); wait(cond_sem);

- signal(C) → signal(cond_sem); (optionally re-acquire mutex depending on Hoare vs. Mesa semantics).

Thus any monitor can be expressed with semaphores, and conversely semaphores can be wrapped in a monitor interface.

📦 Bounded-Buffer Monitor Design (Embedded Portions)

Standard monitor version (small critical sections):

```
monitor Buffer {
    int count = 0;
    condition notFull, notEmpty;

    void put(item x) {
        if (count == MAX) wait(notFull);
        /* insert x */
        ++count;
        signal(notEmpty);
    }

    item get() {
        if (count == 0) wait(notEmpty);
        /* remove item */
        --count;
        signal(notFull);
    }
}
```

- Each wait/signal pair protects only the *tiny* buffer manipulation, making busy-waiting negligible.

a. Why unsuitable for large portions

- The monitor forces **exclusive entry** for the whole function body, so a long computation inside put or get would block all other threads, leading to poor throughput and unnecessary waiting.

b. Scheme for larger portions

- Split the monitor into **two layers**:
  1. **Fine-grained lock** (e.g., a mutex) protecting just the buffer indices.
  2. **Separate condition variables** for notFull/notEmpty that are signaled **outside** the critical section after the buffer update.
- This allows the producer/consumer to perform non-critical work (e.g., data preparation) without holding the monitor lock.

📚 Readers-Writers Fairness vs Throughput

- **Fairness-oriented solution** (e.g., giving priority to waiting writers) reduces writer starvation but can lower overall throughput because readers are frequently blocked.
- **Throughput-oriented solution** (reader-preferring) maximizes concurrent reads but may starve writers.

Starvation-free method

- Maintain a **queue counter** (waiting_writers).
- Readers check waiting_writers; if >0 they wait, otherwise they proceed.
- Writers acquire the lock only when read_count == 0.
- This policy gives writers eventual access while still allowing bursts of concurrent reads.

🔔 Signal Operation Differences (Monitor vs Semaphore)

Aspect	Monitor signal	Semaphore signal

Effect on waiting processes	May **transfer control** immediately (Hoare) or simply **wake** a waiter (Mesa).	Increments the semaphore count; a waiting process will be awakened by the scheduler, not by the signaling thread.
Ordering guarantee	Often **FIFO** within the condition variable's queue.	No intrinsic ordering; the kernel may wake any waiting thread.
Blocking behavior	In Hoare semantics, the signaling thread **waits** until the awakened thread exits the monitor.	The signaling thread never blocks; it continues execution.

🗃️ Simplified Monitor Implementation (Signal Only Inside Functions)

When signal() appears **exclusively** inside monitor procedures, the monitor's **entry protocol** can be simplified:

1. **Enter monitor** (wait(mutex)).
2. Execute procedure body; any signal(C) simply sets a flag need_wakeup = true.
3. **Exit monitor**:
   - If need_wakeup is true, perform signal(C) *after* releasing the mutex (Mesa style).
   - No extra hand-off (next) is required because no external code can signal the condition.

This reduces the monitor's internal state (no next semaphore) and eases implementation on platforms that only support basic mutexes and condition variables.

🖨️ Printer Allocation Monitor (Priority)

> **Goal:** Allocate three identical printers to processes, respecting each process's priority number (lower number = higher priority).

```
monitor PrinterAllocator {
    int free_printers = 3;
    condition printer_available;
    queue waiting;   // ordered by priority (min-heap)

    void request(int priority) {
        if (free_printers == 0) {
            /* insert into waiting queue sorted by priority */
            waiting.insert(this_process, priority);
            wait(printer_available);
        }
        --free_printers;
    }

    void release() {
        ++free_printers;
        if (!waiting.empty()) {
            /* wake highest-priority waiter */
            signal(printer_available);
        }
    }
}
```

- The monitor guarantees **mutual exclusion** on the printer count and respects the priority ordering via the waiting queue.

📂 File Access Monitor with Sum Constraint

Constraint: Sum of the *process numbers* of all currently accessing processes must stay **< n**.

```
monitor FileAccess {
    int current_sum = 0;
```

```
    condition ok_to_enter;
    const int LIMIT = n;

    void enter(int proc_id) {
        while (current_sum + proc_id >= LIMIT)
            wait(ok_to_enter);
        current_sum += proc_id;
    }

    void leave(int proc_id) {
        current_sum -= proc_id;
        signal(ok_to_enter);
    }
}
```

- Each process supplies its identifier when entering; the monitor blocks the process until the summed constraint is satisfied.

🔄 Signaling Semantics in Monitors

When a process executes signal(C) inside a monitor, two possible continuations exist:

1. **Signal-and-continue** (Mesa): the signaling process **continues** execution; the waiting process is merely placed on the ready queue and will run after the monitor is exited.
2. **Signal-and-wait** (Hoare): the signaling process **blocks** immediately, transferring control to the awakened process. The monitor is then re-entered by the awakened process, guaranteeing that the condition holds when it runs.

Effect on solutions:

- Hoare semantics simplify reasoning about the condition (it is true when the waiter runs).
- Mesa semantics require the waiter to **re-check** the condition after being awakened (commonly done with a while loop).

⏱️ Await Construct and Writer Problem

a. Monitor using await(B) for writers

```
monitor Writers {
    int active_readers = 0;
    bool writer_active = false;
    condition ok;

    void start_write() {
        await(!writer_active && active_readers == 0);
        writer_active = true;
    }

    void end_write() {
        writer_active = false;
        signal(ok);
    }
}
```

- await(expr) blocks until expr evaluates to true, then proceeds atomically.

b. Inefficiency reason

- General await may require **re-evaluation of an arbitrary Boolean expression**, potentially causing repeated scans of complex state and leading to *spurious wake-ups*.

c. Restrictions for efficient implementation

- Limit await to **simple predicates** involving a **single condition variable** and a **counter** (e.g., count == 0).

- This enables the runtime to use a **single semaphore** per condition, avoiding costly expression evaluation.

⏰ Alarm Monitor Using tick()

> **Goal:** Provide delay(t) that blocks the calling thread for *t* timer ticks.

```
monitor Alarm {
    int current_tick = 0;
    condition wakeup;

    void tick() {               // invoked periodically by hardware timer
        ++current_tick;
        signal(wakeup);
    }

    void delay(int t) {
        int target = current_tick + t;
        while (current_tick < target)
            wait(wakeup);
    }
}
```

- The monitor keeps a global tick count; delay waits on the wakeup condition until the required number of ticks have elapsed.

🚦 Traffic Deadlock (Figure 5.28)

a. Four necessary conditions

1. **Mutual exclusion:** each road segment can be occupied by only one vehicle.
2. **Hold and wait:** a vehicle may occupy one segment while waiting for the next.
3. **No preemption:** a vehicle cannot be forced to release a segment.
4. **Circular wait:** the four vehicles form a cycle, each waiting for the segment held by the next.

b. Simple avoidance rule

- **Enforce a global ordering** of road segments (e.g., north-south before east-west).
- Require every vehicle to request segments **in that order**, breaking the circular-wait condition.

🍽️ Dining Philosophers Deadlock Discussion

- Classic deadlock occurs when each philosopher picks up its **left** chopstick and then waits for the **right** (or vice-versa), forming a circular wait.
- The four conditions apply as in the traffic example.

Prevention strategies (refer to earlier monitor solutions):

- **Resource hierarchy:** each philosopher picks the lower-numbered chopstick first.
- **Asymmetric ordering:** even-numbered philosophers pick left then right; odd-numbered pick right then left.
- **Limit concurrent philosophers** (e.g., allow at most four of five to sit).

🆔 Thread-Safe PID Manager

Original design (single-threaded):

```
int number_of_processes = 0;
int allocate_process() { ++number_of_processes; return new_pid; }
void release_process() { --number_of_processes; }
```

Race condition: simultaneous allocate_process calls may return the same PID or corrupt the counter.

Thread-safe fix (Pthreads):

```
pthread_mutex_t pid_lock = PTHREAD_MUTEX_INITIALIZER;
int allocate_process() {
    pthread_mutex_lock(&pid_lock);
    if (number_of_processes == MAX_PROCESSES) {
        pthread_mutex_unlock(&pid_lock);
        return -1;
    }
    ++number_of_processes;
    int pid = compute_pid();
    pthread_mutex_unlock(&pid_lock);
    return pid;
}
void release_process() {
    pthread_mutex_lock(&pid_lock);
    --number_of_processes;
    pthread_mutex_unlock(&pid_lock);
}
```

- The mutex guarantees **atomicity** of allocation and release.

🧱 Resource Manager Race Condition & Condition Variable Solution

a. Data involved

- Shared variable available_resources (initially MAX_RESOURCES).

b. Race-prone locations

- The test if (available_resources < count) and the subsequent decrement are **non-atomic**; two threads may both see enough resources and both decrement, leading to a negative count.

c. Fix with a binary semaphore (mutex) and a condition variable to block until enough resources exist.

```
pthread_mutex_t  mtx = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t   enough = PTHREAD_COND_INITIALIZER;
int available = MAX_RESOURCES;

int decrease_count(int cnt) {
    pthread_mutex_lock(&mtx);
    while (available < cnt)               // wait until sufficient
        pthread_cond_wait(&enough, &mtx);
    available -= cnt;
    pthread_mutex_unlock(&mtx);
    return 0;
}

void increase_count(int cnt) {
    pthread_mutex_lock(&mtx);
    available += cnt;
    pthread_cond_broadcast(&enough);      // wake all waiting threads
    pthread_mutex_unlock(&mtx);
}
```

- Callers of decrease_count are **blocked** (no busy-wait) until enough resources become free.

🎲 Monte Carlo Parallel Estimation (Mutex Protection)

- Global counters: total_points, inside_circle.
- Each thread generates random points, tests inclusion, then updates the counters.

Required protection:

```
pthread_mutex_t cnt_lock = PTHREAD_MUTEX_INITIALIZER;
/* inside thread */
pthread_mutex_lock(&cnt_lock);
++total_points;
if (inside) ++inside_circle;
pthread_mutex_unlock(&cnt_lock);
```

- This eliminates the race on the shared counters; the final estimate $\pi \approx 4 \cdot \text{inside\_circle} / \text{total\_points}$ is then correct.

⛔ Barrier Synchronization API

> **Barrier:** a synchronization point where *N* threads must all arrive before any may proceed.

API (as described):

- int init(int n); → initialize barrier for *n* participants.
- int barrier_point(void); → called by each thread when it reaches the barrier.

Typical implementation (pseudocode):

```
int count = 0;
pthread_mutex_t bmtx = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  bcv  = PTHREAD_COND_INITIALIZER;

int barrier_point(void) {
    pthread_mutex_lock(&bmtx);
    ++count;
    if (count == N) {
        count = 0;              // reset for reuse
        pthread_cond_broadcast(&bcv);
    } else {
        while (count != 0)
            pthread_cond_wait(&bcv, &bmtx);
    }
    pthread_mutex_unlock(&bmtx);
    return 0;
}
```

- The last arriving thread **releases** all others, ensuring that no thread proceeds past the barrier until the group is complete.

💤 Sleeping TA Project Overview

Scenario: One TA, *n* student threads, three hallway chairs.

Synchronization primitives needed:

- **Semaphore ta_sleep** (initial value 0) – students signal the TA when they arrive while the TA is sleeping.
- **Semaphore chairs** (initial value 3) – controls access to hallway chairs.
- **Mutex help_lock** – protects the critical section where the TA assists a student.

High-level algorithm:

1. **Student thread:**
   - If chairs > 0, decrement chairs (sit).
   - If TA is sleeping, signal(ta_sleep).
   - Wait on help_lock to get help.

    - After help, signal(chairs) to free a chair.
2. **TA thread:**
    - Loop: wait(ta_sleep) (blocks when no students).
    - Acquire help_lock to assist the next waiting student.
    - Release help_lock after help; if no waiting students, go back to waiting on ta_sleep.
- This design guarantees **mutual exclusion** during help, **bounded waiting** for chairs, and wakes the TA only when needed.

🍴 Dining Philosophers with Monitors

Monitor implementation (Mesa style):

```
monitor Dining {
    enum { THINKING, HUNGRY, EATING } state[5];
    condition self[5];

    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING) wait(self[i]);
    }

    void putdown(int i) {
        state[i] = THINKING;
        test((i+4)%5);   // left neighbor
        test((i+1)%5);   // right neighbor
    }

    void test(int i) {
        if (state[i] == HUNGRY &&
            state[(i+4)%5] != EATING &&
            state[(i+1)%5] != EATING) {
            state[i] = EATING;
            signal(self[i]);   // wake if waiting
        }
    }
}
```

- Guarantees **mutual exclusion** of adjacent chopsticks, avoids deadlock because a philosopher never holds a single chopstick while waiting.

🧵 Pthreads Condition Variable Pattern

> **Condition variable:** used with a mutex to block a thread until a predicate becomes true.

Typical usage pattern (C/Pthreads):

```
pthread_mutex_lock(&mtx);
while (!predicate)              // re-check after wake-up
    pthread_cond_wait(&cv, &mtx);
... // critical section where predicate holds
pthread_mutex_unlock(&mtx);
```

- **Signaling:**

    ```
    pthread_mutex_lock(&mtx);
    predicate = true;
    pthread_cond_signal(&cv);   // or pthread_cond_broadcast
    pthread_mutex_unlock(&mtx);
    ```

The loop guards against **spurious wake-ups** and ensures the condition is still satisfied when the thread proceeds.

📦 Producer-Consumer with Semaphores & Mutex

Resources:

- sem empty – counts empty slots (initial = BUFFER_SIZE).
- sem full – counts filled slots (initial = 0).
- pthread_mutex_t buf_lock – protects buffer indices.

Producer thread:

```
wait(empty);
pthread_mutex_lock(&buf_lock);
insert_item(item);
pthread_mutex_unlock(&buf_lock);
signal(full);
```

Consumer thread:

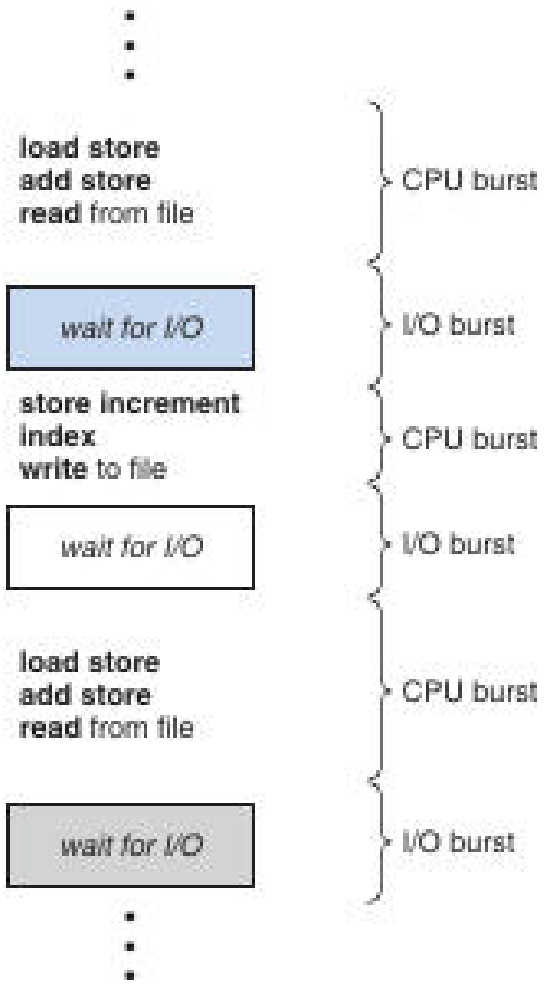
```
wait(full);
pthread_mutex_lock(&buf_lock);
remove_item(&item);
pthread_mutex_unlock(&buf_lock);
signal(empty);
```

- The **semaphores** enforce the bounded-buffer capacity, while the **mutex** prevents concurrent modifications of the circular-queue indices.

⚙️ CPU Scheduling Overview

> **Definition:** *CPU scheduling* is the operating-system mechanism that selects which of the ready processes will run on the CPU and for how long. It is the core of multiprogrammed system design, aiming to keep the processor busy while processes alternate between computation and I/O.

Multiprogramming keeps several processes resident in memory so that when one process blocks for I/O, another can use the CPU. The goal is to **maximize CPU utilization** and reduce idle time.



The diagram shows a sequence of CPU bursts (black text) and I/O bursts (blue/grey rectangles), demonstrating the alternating execution pattern of a typical process.

🧩 Basic Concepts

- **CPU-I/O burst cycle** – A process executes a *CPU burst*, then performs an *I/O operation* (the I/O burst), and repeats until termination.
- **CPU-bound vs. I/O-bound** –
  - *CPU-bound* processes have many long CPU bursts and few I/O operations.
  - *I/O-bound* processes have short CPU bursts and frequent I/O, causing the CPU to be idle often.
- Measured CPU-burst durations typically follow an **exponential or hyper-exponential** distribution: many short bursts, few long ones.

📋 CPU Scheduler Components

Component	Role
Short-term scheduler	Chooses a process from the *ready queue* for the next CPU allocation.
Dispatcher	Transfers control to the selected process: performs a context switch, switches to user mode, and jumps to the saved program counter.

Dispatch latency	Time taken by the dispatcher to stop one process and start another; should be as small as possible because it occurs on every scheduling event.

⏱️ Scheduling Criteria

Criterion	What it measures	Desired direction
CPU utilization	Percentage of time the CPU is doing useful work.	↑ (approach 100 % on a loaded system)
Throughput	Number of processes completed per unit time.	↑
Turnaround time	Time from process submission to completion (including waiting, execution, and I/O).	↓
Waiting time	Total time a process spends in the ready queue.	↓
Response time	Time from request submission to first output (important for interactive systems).	↓
Variance of response time	Predictability of response; lower variance is preferred for interactive workloads.	↓

🧩 Preemptive vs. Non-preemptive Scheduling

- **Preemptive scheduling** allows the operating system to interrupt a running process (e.g., on timer expiration or higher-priority arrival) and assign the CPU to another process.
  - Used by modern Windows (post-95), macOS, and most UNIX variants.
  - Can introduce **race conditions** because a process may be preempted while modifying shared data.
- **Non-preemptive (cooperative) scheduling** lets a process retain the CPU until it voluntarily yields (by terminating or requesting I/O).
  - Historically used in early Windows (3.x) and classic Mac OS.
  - Simpler but unsuitable for time-sharing because a misbehaving process can monopolize the CPU.

When preemption occurs

1. A running process moves to **waiting** (e.g., initiates I/O).
2. A running process is **preempted** by an interrupt and placed back in the ready queue.
3. A waiting process becomes **ready** (I/O completes).
4. A process **terminates**.

If the scheduler makes decisions only in cases 1 and 3, the scheme is **non-preemptive**; otherwise it is **preemptive**.

🏁 Scheduling Algorithms

🕑 First-Come, First-Served (FCFS)

> **Definition:** The *FCFS* algorithm assigns the CPU to the process that has been waiting the longest, using a simple FIFO queue.

- **Implementation:** Insert a PCB at the tail of the ready queue when a process becomes ready; dispatch the head of the queue when the CPU is free.
- **Pros:** Simple to implement; fair in the sense of arrival order.
- **Cons:** Can yield long **average waiting times** (convoy effect) and performs poorly for time-sharing systems.
- **Non-preemptive:** Once a process gets the CPU, it runs until it blocks or terminates.

Example:
Processes P1 = 24 ms, P2 = 3 ms, P3 = 3 ms arrive at time 0.
FCFS order → P1, P2, P3 → average waiting time = (0 + 24 + 27) / 3 = 17 ms.
Reordering to P2, P3, P1 reduces the average to 3 ms, illustrating the impact of burst length ordering.

⏱️ Shortest-Job-First (SJF)

> **Definition:** *SJF* selects the process with the **shortest next CPU burst** for execution.

- **Optimality:** Proven to minimize average waiting time for a given set of processes.
- **Non-preemptive SJF** runs the chosen process to completion of its current burst.
- **Preemptive SJF** (also called *Shortest-Remaining-Time-First*, SRTF) preempts the running process if a newly arrived process has a shorter remaining burst.

Burst-time prediction

Because the exact length of the next burst is unknown, the OS often predicts it using exponential averaging:

$ \tau_{n+1}= \alpha , t_n + (1-\alpha) , \tau_n $

- (t_n) = actual length of the nth burst
- (\tau_{n+1}) = predicted length of the next burst
- (\alpha) ($0 \le \alpha \le 1$) controls the weighting of recent vs. older history (common choice $\alpha = ½$).

Example: Processes P1 = 8 ms, P2 = 4 ms, P3 = 9 ms, P4 = 5 ms arriving at times 0, 1, 2, 3 respectively.
Preemptive SJF produces a Gantt chart where P1 runs first, then is preempted by P2, etc., yielding an average waiting time of 6.5 ms, better than the non-preemptive case.

⚡ Priority Scheduling

> **Definition:** *Priority scheduling* assigns the CPU to the ready process with the **highest priority** (numerically lowest in this text).

- **Relation to SJF:** When priority is defined as the inverse of the predicted next CPU burst, SJF becomes a special case of priority scheduling.
- **Tie-breaking:** Processes with equal priority are scheduled in FCFS order.
- **Priority ranges:** Often 0 – 15 or 0 – 4095; here **low numbers denote high priority**.

Example:

Process	Burst (ms)	Priority
P1	10	3
P2	1	0
P3	2	4
P4	1	1
P5	5	2

Scheduling by priority → P2, P4, P5, P1, P3 → average waiting time ≈ 8.2 ms.

- **Internal vs. external priorities** – Internal priorities are computed from process characteristics (e.g., CPU-burst length, I/O-burst ratio). External priorities are set by the OS or administrator (e.g., user class, payment level).
- **Aging** – To avoid indefinite postponement of low-priority jobs, a process's priority can be gradually increased the longer it waits, guaranteeing eventual execution.

🔁 Round-Robin (RR)

> **Definition:** *Round-Robin* is a preemptive time-sharing algorithm that gives each process a fixed **time quantum** (or time slice) in a cyclic order.

- **Ready queue** is treated as a circular FIFO.
- After a process consumes its quantum, the scheduler **preempts** it, places it at the tail, and dispatches the next process.
- **Quantum size** (typically 10–100 ms) strongly influences performance:
  - **Large quantum** → behavior approaches FCFS, fewer context switches, but longer response times.
  - **Small quantum** → many context switches, higher overhead, but better responsiveness.

Example: Quantum = 4 ms, processes: P1 = 24 ms, P2 = 3 ms, P3 = 3 ms (all arrive at 0).
RR schedule → P1(0-4), P2(4-7), P3(7-10), P1(10-14), … resulting in an average waiting time of ≈ 5.66 ms.

🛠️ Dispatcher Details

> **Definition:** The *dispatcher* is the OS module that gives control of the CPU to the process selected by the short-term scheduler.

Steps performed by the dispatcher:

1. **Context switch** – Save the state of the currently running process and load the state of the chosen process.
2. **Switch to user mode** – Change CPU privilege level.
3. **Jump to the saved program counter** – Resume execution at the point where the process previously stopped.

The **dispatch latency** is the total time taken for these steps; minimizing it is critical because the dispatcher is invoked on every scheduling event.

📉 Real-World Considerations

- **CPU-bound vs. I/O-bound mix:** A single long CPU-bound process can dominate the CPU, causing I/O devices to sit idle while many short I/O-bound processes wait. Algorithms that favor short jobs (e.g., SJF, RR with small quantum) improve overall utilization.
- **Starvation:** Priority and SJF can starve low-priority or long-burst processes; **aging** mitigates this by gradually raising waiting processes' priorities.
- **Response-time variance:** For interactive systems, predictable response time (low variance) may be more important than minimizing the average. Few scheduling algorithms explicitly target variance, leaving it an open research area.

⏱️ Time Quantum & Context-Switch Overhead

> **Time quantum** – the maximum amount of CPU time a process may run before the scheduler preempts it and performs a context switch.

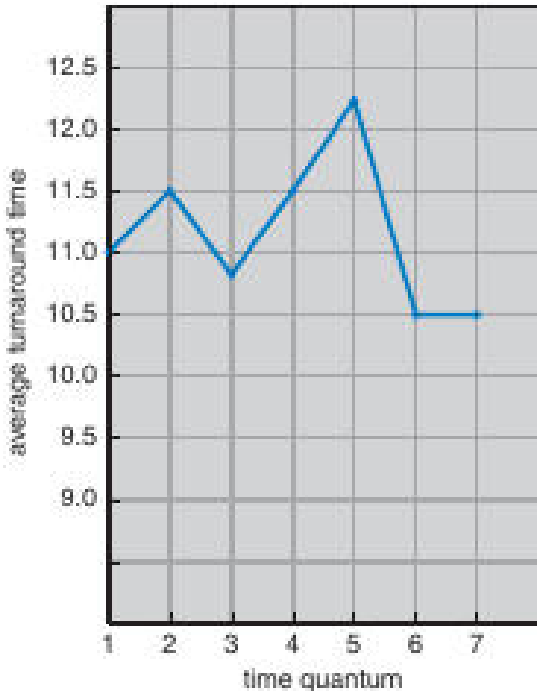
- A context switch costs roughly **10 %** of a quantum when the quantum is small; the larger the quantum, the smaller the fraction of CPU time spent on switching.
- **Average turnaround time** improves when most CPU bursts fit within a single quantum.

Example – three processes each require a CPU burst of 10 time units:

Quantum	Avg. turnaround (no context-switch cost)
1	29
10	20

When the overhead of a context switch (≈ 10 % of the quantum) is added, smaller quanta suffer larger increases in turnaround because more switches are required.

Guideline – aim for a quantum such that **≈ 80 %** of CPU bursts are shorter than the quantum. Modern systems typically use quanta of **10–100 ms**; a context switch takes about **10 µs**, a negligible fraction of the quantum.



The blue line shows how average turnaround time varies as the time quantum increases from 1 to 7. Turnaround drops quickly at first, then stabilizes, illustrating the diminishing returns of a very large quantum.

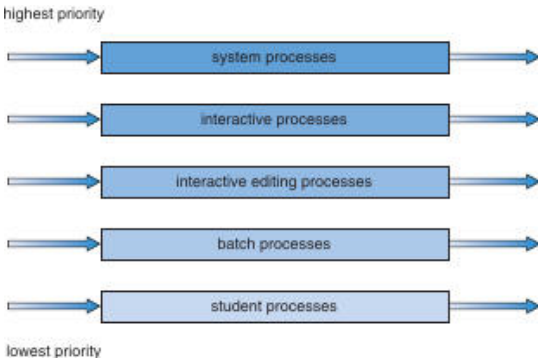
Multilevel Queue Scheduling

> **Multilevel queue** – partitions the ready queue into several *permanent* queues, each associated with a distinct class of processes (e.g., foreground vs. background).

- **Queue assignment** is based on a static property such as **process type**, **priority**, or **size**.
- Each queue can employ a different scheduling algorithm (e.g., RR for interactive, FCFS for batch).
- **Inter-queue scheduling** is usually **fixed-priority preemptive**: a higher-priority queue can preempt any lower-priority queue.

Typical hierarchy (high → low priority)

Priority	Queue content
1	System processes
2	Interactive editing processes
3	Interactive processes
4	Batch processes
5	Student processes



Bar chart showing the five priority classes from highest (system) to lowest (student).

Scheduling nuances

- A **foreground** queue may receive **80 %** of CPU time (RR among its processes), while the **background** queue gets the remaining **20 %** (FCFS).
- Lower-priority queues run only when all higher queues are empty; no process ever moves between queues, which yields low overhead but limited flexibility.

🔁 Multilevel Feedback Queue (MLFQ)

> **MLFQ** – a dynamic variant of multilevel queue where processes can **move between queues** based on their observed CPU-burst behavior.

- Short, I/O-bound bursts stay in **higher-priority queues**; long, CPU-bound bursts are **demoted** to lower queues.
- **Aging**: a process that waits too long in a low-priority queue is eventually **promoted** to avoid starvation.

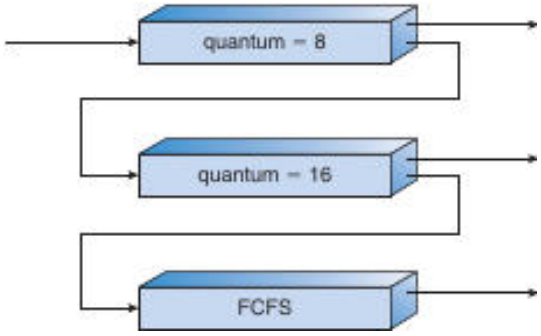
Illustrative three-queue example

Queue	Time quantum	Promotion/Demotion rule
0	8 ms	If a process finishes ≤ 8 ms → stays; otherwise → move to tail of queue 1
1	16 ms	Same rule; if not finished → move to queue 2
2	FCFS (no quantum)	Runs only when queues 0 and 1 are empty

A process always enters at **queue 0**. The scheduler always selects the **highest-priority non-empty queue**. This scheme gives priority to any burst ≤ 8 ms, allowing quick I/O-bound jobs to finish promptly while longer jobs gradually sink to queue 2.

Key parameters for an MLFQ scheduler

- Number of queues.
- Scheduling algorithm used in each queue.
- Criteria for **promotion** (e.g., after waiting *t* milliseconds).
- Criteria for **demotion** (e.g., after exhausting its quantum).
- Method for **initial queue selection** (often the highest-priority queue).



Flowchart contrasts three policies: quantum = 8 ms, quantum = 16 ms, and FCFS.

🧵 Thread Scheduling & Contention Scope

> **Thread scheduling** determines which runnable thread receives a CPU core at any moment.

Contention Scope

Scope	Meaning
Process-Contention Scope (PCS)	Threads compete *only* with other threads of the **same process**. Typically used in many-to-many models where a user-level library maps threads onto a pool of lightweight processes (LWPs).
System-Contention Scope (SCS)	Threads compete with **all** threads in the system; the kernel schedules them directly (one-to-one model).

- In **PCS**, the library may implement its own scheduler; the kernel sees only the underlying LWPs.
- In **SCS**, the kernel's scheduler decides, usually based on thread priority.

Pthread Scheduling API

```
/* Pthread scheduling example – shows contention-scope handling */
#include
#define NUM_THREADS 5

int main(int argc, char *argv[]) {
    int i, scope;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;

    pthread_attr_init(&attr);
    if (pthread_attr_getscope(&attr, &scope) != 0)
        fprintf(stderr, "Unable to get scheduling scope\n");
    else if (scope == PTHREAD_SCOPE_PROCESS)
        printf("PTHREAD_SCOPE_PROCESS\n");
    else if (scope == PTHREAD_SCOPE_SYSTEM)
        printf("PTHREAD_SCOPE_SYSTEM\n");
    else
        fprintf(stderr, "Illegal scope value.\n");

    /* Force system-scope (SCS) */
    pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);

    for (i = 0; i < NUM_THREADS; i++)
        pthread_create(&tid[i], &attr, runner, NULL);

    for (i = 0; i < NUM_THREADS; i++)
```

```
        pthread_join(tid[i], NULL);
    return 0;
}
```

- pthread_attr_getscope / pthread_attr_setscope query or set the contention scope.
- Many modern systems (e.g., Linux) support **only** PTHREAD_SCOPE_SYSTEM.

🖥️ Multiprocessor Scheduling

Scheduling Architectures

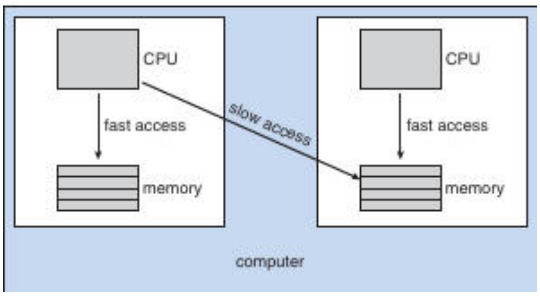
Architecture	Description
Asymmetric Multiprocessing (AMP)	One *master* processor runs the scheduler; other CPUs execute only user code. Simpler because only one processor updates scheduling data structures.
Symmetric Multiprocessing (SMP)	Every processor runs its own scheduler and may have a **private ready queue** or share a **global queue**. Requires synchronization to avoid double-scheduling.

Processor Affinity

> **Processor affinity** – the tendency to keep a thread on the same CPU core to preserve cache locality.

- **Soft affinity**: OS *tries* to keep a thread on its previous core but may migrate it when load-balancing.
- **Hard affinity**: Application explicitly restricts a thread to a set of CPUs (e.g., sched_setaffinity on Linux).

NUMA considerations – In a *non-uniform memory access* system, a CPU accesses its local memory faster than remote memory. Aligning a thread's affinity with memory placement improves performance.



Left side: fast local memory access; right side: slower remote memory access. Affinity helps keep a thread near its local memory.

Load Balancing

Two complementary strategies:

- **Push migration** – a busy processor periodically *pushes* tasks to less-loaded CPUs.
- **Pull migration** – an idle processor *pulls* tasks from a busy counterpart.

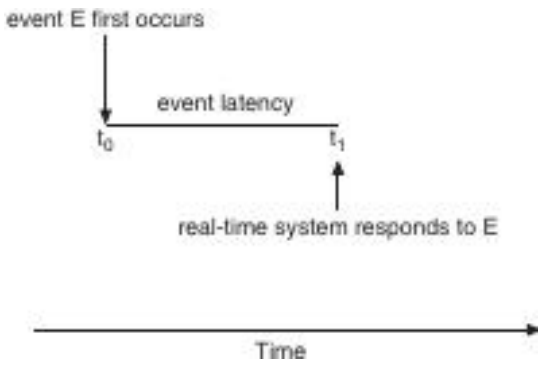
Both approaches appear in modern kernels (e.g., Linux, FreeBSD). Load balancing can conflict with affinity because moving a thread discards its cached data, so systems often employ thresholds to decide when migration is worthwhile.

🧩 Multicore & Multithreading

Memory Stalls

> **Memory stall** – a period during which the processor idles while waiting for data from memory (e.g., after a cache miss).

- Stalls can consume **up to 50%** of execution time on memory-bound workloads.



The diagram shows a processor spending a large fraction of its cycle waiting for data.

Hardware Multithreading

- **Coarse-grained**: a thread runs until it encounters a long-latency event (e.g., cache miss), then the core switches to another thread. Switching cost is relatively high.
- **Fine-grained**: the core interleaves instructions from multiple threads every cycle; the hardware incurs a small switching overhead.

Modern chips expose multiple **logical processors**:

- Dual-threaded cores → each core appears as two logical CPUs.
- Example: **UltraSPARC T3** – 16 cores ×8 hardware threads = **128 logical processors**.

The OS schedules software threads onto these logical processors using any of the algorithms discussed earlier (RR, priority, etc.). The hardware then selects which *hardware thread* to run (e.g., simple round-robin on each core, or urgency-based selection as on the Itanium).

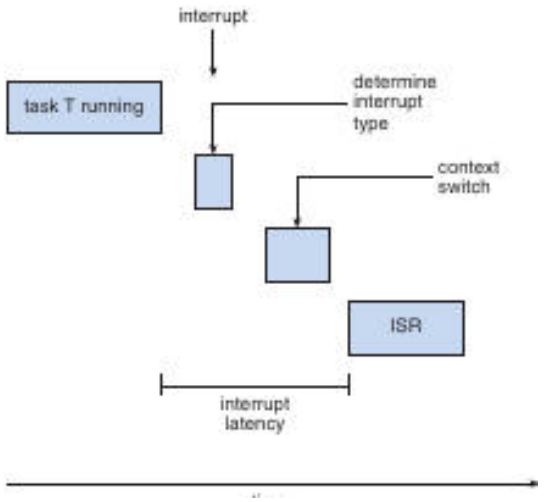
⏱️ Real-Time CPU Scheduling

Soft vs. Hard Real-Time

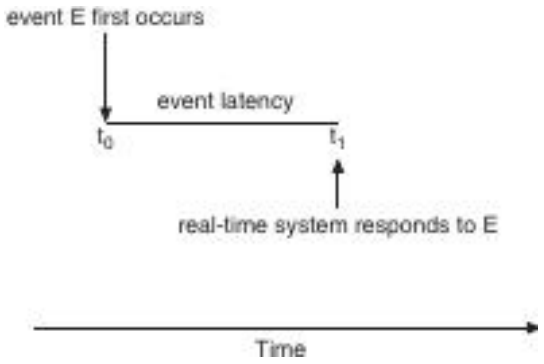
Category	Guarantees
Soft real-time	No strict deadline guarantee; tasks receive higher priority but may miss deadlines occasionally.
Hard real-time	**Deadline must be met**; missing it is considered a system failure.

Latency Types

1. **Interrupt latency** – time from an interrupt's arrival to the start of its **Interrupt Service Routine (ISR)**.
2. **Dispatch latency** – time from the moment a real-time task becomes ready to the moment it actually starts executing on the CPU.



The flowchart traces the steps from an interrupt occurring while a task runs, through context switching, to the ISR execution, highlighting interrupt latency.



Timeline shows an event occurring at (t_0) and the system's response at (t_1); the interval is the event latency.

Minimizing Latency

- **Preemptive kernels** allow an urgent real-time task to interrupt a lower-priority task instantly, reducing dispatch latency.
- Keep the period during which interrupts are disabled **as short as possible**; lengthy critical sections increase both interrupt and dispatch latency.
- Use **priority inheritance** (or similar mechanisms) to avoid priority inversion that can inflate latency.

⏱️ Dispatch Latency Diagram

> *Dispatch latency* is the total time from when a process becomes ready to when it actually starts executing on the CPU.

The latency consists of two phases:

1. **Preemption of the currently running kernel process** – the scheduler must interrupt the kernel task and save its state.
2. **Release of resources held by low-priority processes** – any locks, memory pages, or I/O buffers required by the newly ready real-time process must be freed.

> In Solaris, enabling preemption reduces dispatch latency from **> 100 ms** to **< 1 ms**, illustrating the importance of the preemption phase for hard real-time responsiveness.

⚙️ Priority-Based Scheduling

> *Priority-based scheduling* assigns each process a numerical priority; higher-priority processes are chosen before lower-priority ones.

- When **preemption** is supported, a running lower-priority process is immediately displaced if a higher-priority process becomes ready.
- Real-time operating systems typically reserve the highest priority levels for real-time tasks (e.g., Windows reserves levels 16–31).
- Preemptive, priority-based schedulers guarantee **soft real-time** behavior; **hard real-time** guarantees require additional mechanisms (e.g., admission control, deadline enforcement).

📅 Periodic Real-Time Processes

> A *periodic* real-time process repeats every fixed interval **p** and must finish its CPU burst **t** before its deadline **d** (where (0 \le t \le d \le p)).

Key parameters:

- **Processing time** (t) – CPU time needed each period.
- **Period** (p) – interval between successive releases.
- **Deadline** (d) – latest time the burst must complete (often (d = p)).

These attributes enable the scheduler to reason about feasibility and assign priorities based on rate or deadline.

🕒 Rate-Monotonic Scheduling (RMS)

> *RMS* is a static-priority algorithm that gives a higher priority to the task with the **shorter period**.

How RMS Works

1. Upon admission, each periodic task receives a priority inversely proportional to its period.
2. The scheduler always runs the highest-priority ready task; lower-priority tasks are preempted as soon as a higher-priority task arrives.

Utilization Test

For **N** periodic tasks, RMS guarantees that all deadlines are met if

$ U = \sum_{i=1}^{N}\frac{t_i}{p_i} \le N\left(2^{1/N}-1\right) $

- As (N \to \infty), the bound approaches **≈ 69 %**.
- With two tasks, the bound is **≈ 83 %**; with three tasks, **≈ 78 %**, etc.

Example

Task	(p) (ms)	(t) (ms)	(t/p)
P1	50	20	0.40
P2	100	35	0.35

Total utilization (U = 0.75) < 83 % → RMS can schedule them.
Assigning P1 the higher priority (shorter period) yields the schedule shown in Figure 6.17, where both deadlines are met.

When the utilization exceeds the bound (e.g., Figure 6.18 with (U ≈ 0.94)), RMS cannot guarantee deadline satisfaction; P1 may miss its deadline despite the CPU being less than fully loaded.

📆 Earliest-Deadline-First (EDF) Scheduling

> *EDF* is a dynamic-priority algorithm that always selects the ready task whose **absolute deadline** is soonest.

- Priorities are recomputed whenever a task becomes runnable.
- Unlike RMS, EDF does **not** require fixed periods; any task that announces its deadline can be scheduled.

Optimality

- EDF can achieve **100 % CPU utilization** in theory (all deadlines met if (U \le 1)).
- In practice, context-switch overhead and interrupt handling reduce achievable utilization.

Example (same tasks as RMS)

- Initially P1 (deadline 50) runs, then P2 (deadline 80) runs.
- When P2's deadline becomes earlier than P1's next deadline, EDF preempts P1, guaranteeing both tasks meet their deadlines (see Figure 6.19).

📊 Proportional-Share Scheduling

> *Proportional-share* allocates CPU time proportionally to **shares** requested by each client.

- Total shares **T** are divided among applications; an application receiving **S** shares gets (S/T) of the processor.
- An **admission-control** policy ensures that the sum of granted shares never exceeds **T**.

Example

- (T = 100) shares.
- A receives 50 shares → 50 % CPU, B receives 15 % → 15 % CPU, C receives 20 % → 20 % CPU.
- A new request for 30 shares (process D) would be denied because only 15 shares remain.

🛠️ POSIX Real-Time Scheduling (SCHED_FIFO & SCHED_RR)

> POSIX.1b defines two real-time scheduling classes for threads:

Class	Policy	Characteristics
SCHED_FIFO	First-In-First-Out	No time slicing; threads run to completion or block.
SCHED_RR	Round-Robin	Time-sliced among threads of equal priority (similar to traditional RR).
SCHED_OTHER	Implementation-defined (usually the default time-sharing class).	Behavior varies across OSes.

Key API functions:

- pthread_attr_getschedpolicy(pthread_attr_t *attr, int *policy) – retrieve current policy.
- pthread_attr_setschedpolicy(pthread_attr_t *attr, int policy) – set desired policy (e.g., SCHED_FIFO).

Error handling: both functions return non-zero on failure.

🐧 Linux Scheduling Evolution

O(1) Scheduler (pre-2.6)

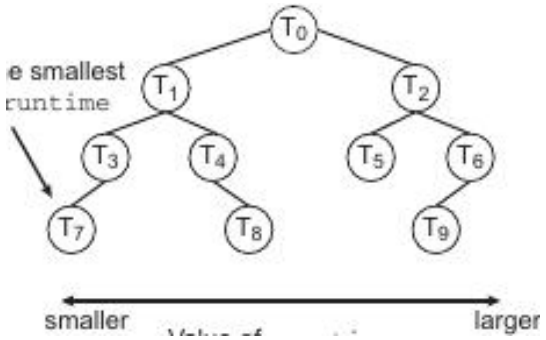
- Ran in **constant time** regardless of task count.
- Provided good SMP load balancing but exhibited poor interactive response.

Completely Fair Scheduler (CFS) – default since 2.6.23

- **Scheduling classes**: a default CFS class and a real-time class.
- **Nice values** (-20 \dots +19) map to relative priority; lower nice → higher share of CPU.
- **Virtual runtime (vruntime)** tracks how much "fair" CPU time each task has received; the task with the **smallest vruntime** is chosen next.

Red-Black Tree Structure

The runnable tasks are stored in a balanced red-black tree keyed by vruntime. The leftmost node holds the task with the smallest vruntime (i.e., highest priority).



The diagram shows nodes (T_0)–(T_9) ordered from smallest (left) to largest (right) vruntime. The leftmost node is the next task to run, enabling O(log N) selection, cached as rb_leftmost for O(1) access.

- **I/O-bound vs. CPU-bound**: I/O-bound tasks accumulate less vruntime and are favored, leading to natural preemption of CPU-bound tasks when I/O becomes ready.
- Real-time threads (SCHED_FIFO / SCHED_RR) run at higher priority than normal CFS tasks.

Windows Scheduling Overview

- Uses a **preemptive, priority-based** scheduler with **32 priority levels** (1–15 for normal, 16–31 for real-time; 0 reserved for the system).
- **Dispatcher** traverses priority queues from highest to lowest, selecting the first ready thread. If none are ready, the idle thread runs.

Priority Classes & Relative Priorities

Priority Class	Range	Typical Use
REALTIME	16-31	Critical system or multimedia tasks.
HIGH	8-13	Time-sensitive user processes.
NORMAL	4-7	Standard interactive applications.
IDLE	1-3	Background maintenance.

- Within a class, threads have a **relative priority** (e.g., THREAD_PRIORITY_NORMAL).
- Windows boosts the priority of the **foreground process** (typically ×3) to improve responsiveness.
- **Time quantum** expiration, I/O completion, or higher-priority arrival cause preemption.

User-Mode Scheduling (UMS)

- Introduced in Windows 7, allowing applications to manage their own threads without kernel intervention, reducing overhead for massive thread counts.
- Fibers (pre-Windows 7) lacked independent thread contexts; UMS provides full context for each user-mode thread.

Solaris Scheduling Highlights

- Employs **priority-based** thread scheduling similar to other UNIX-like systems.

- Real-time threads receive the highest static priorities; normal threads share a lower range.
- Supports **multiprocessor** environments with load-balancing and affinity mechanisms.

📈 Comparative Snapshot of Real-Time Scheduling Strategies

Strategy	Priority Assignment	Preemptive?	Utilization Bound	Typical Use
Rate-Monotonic (RMS)	Fixed, shorter period → higher priority	Yes	≤ (N(2^{1/N}-1)) (≈ 69 % → 83 % for few tasks)	Hard real-time systems with periodic workloads
Earliest-Deadline-First (EDF)	Dynamic, earliest absolute deadline → higher priority	Yes	≤ 1 (theoretically 100 %)	Soft real-time and mixed workloads
Proportional-Share	Shares → CPU proportion	Usually non-preemptive but can be combined with RR	≤ 1 (subject to overhead)	Fair resource allocation among competing applications
POSIX SCHED_FIFO / SCHED_RR	Static priority levels	Yes (FIFO: no slicing; RR: time-slicing)	No explicit bound	Real-time threads on POSIX-compatible OSes
Linux CFS	Nice-based virtual runtime	Yes (preemptive)	No hard bound; strives for fairness	General-purpose Linux systems
Windows Priority-Based	32-level numeric priority + class boost	Yes	No explicit bound	Desktop and server Windows environments

---

🛡️ Ensuring Hard Real-Time Guarantees

- **Admission control**: before admitting a task, the scheduler verifies that the cumulative utilization satisfies the algorithm's bound (e.g., RMS bound).
- **Deadline enforcement**: tasks that miss deadlines trigger corrective actions (e.g., task abort, mode switch).
- **Resource reservation**: CPU time, memory, and I/O bandwidth may be statically allocated to guarantee availability.

🗂️ Scheduling Classes Overview

> **Definition:** A *scheduling class* groups threads that share a common priority scheme and scheduling policy.

Solaris defines six classes:

Class	Abbreviation	Typical Use	Priority Behaviour
Time-Sharing	TS	General interactive and batch processes	Dynamic priorities; higher priority → shorter time quantum
Interactive	IA	GUI and window-manager tasks (e.g., KDE)	Treated as a special TS subset with higher base priority
Real-Time	RT	Hard-real-time threads requiring bounded response	Highest absolute priority; never pre-empted by lower classes
System	SYS	Kernel threads (paging daemon, interrupt handlers)	Fixed priorities; not altered by the scheduler
Fixed-Priority	FP	Legacy or specialized workloads that need static ordering	Priorities are static within the class
Fair-Share	FSS	Projects or groups of processes that receive a proportion of CPU	Priorities derived from assigned CPU shares

The **default** class for a newly created process is **Time-Sharing**.

⏱️ Time-Sharing & Interactive Scheduling

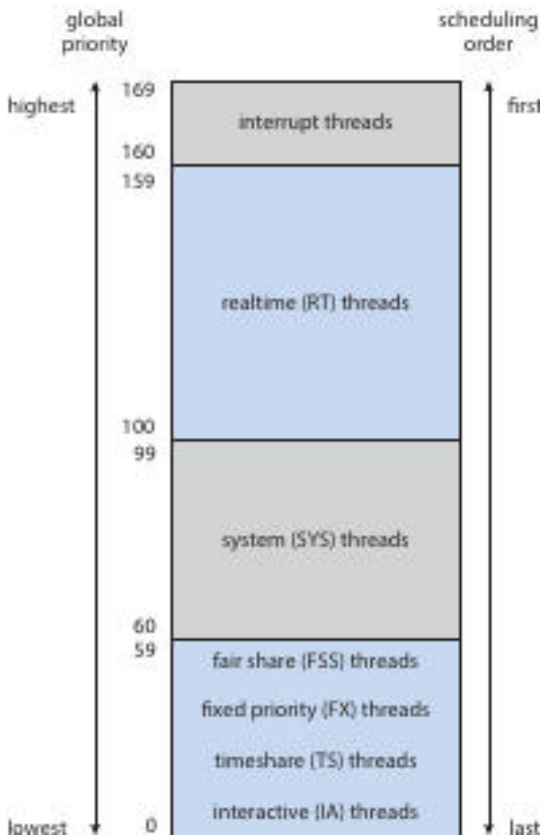
> **Definition:** The *time-sharing* class uses a **multilevel feedback queue (MLFQ)** that changes both a thread's priority and its time-slice length while it runs.

- **Priority ↔ Time Quantum:**
  - Higher numeric priority → **smaller** time quantum.
  - Lower numeric priority → **larger** time quantum.
- **Interactive processes** (e.g., window managers) receive higher base priorities, giving them better response and throughput.
- The **dispatch table** (Solaris) maps each priority to a specific quantum; lower-priority entries have larger quanta (up to 200 ms), while the highest priority (59) gets a 20 ms slice.

Solaris Dispatch Table (excerpt)

Priority	Time Quantum (ms)
0	200
5	160
10	160
15	120
20	120
25	80
30	80
35	40
40	40
45	40
50	40
55	40
59	20

- **Quantum Expired:** The thread's priority is *lowered* (moves to a lower-priority queue).
- **Return from Sleep / I/O:** The thread's priority is *boosted* (moved toward the higher-priority end) to improve response time for I/O-bound work.



The table shows priority numbers (0–59) on the left and the associated time-slice lengths on the right. Higher priorities have shorter slices, illustrating the inverse relationship discussed above.

🚀 Real-Time, System, Fixed-Priority, and Fair-Share Classes

- **Real-Time (RT):** Threads run before any other class. A real-time thread is guaranteed to execute for a **bounded period** once scheduled.
- **System (SYS):** Reserved for kernel-mode threads; priorities are set once at creation and never change.
- **Fixed-Priority (FP):** Behaves like the time-sharing class but **does not** adjust priorities dynamically; useful for legacy applications.
- **Fair-Share (FSS):** Instead of numeric priorities, each *project* receives a **CPU share** (e.g., 20 % of the processor). The scheduler converts these shares into **global priorities** that compete with other classes. Threads with equal global priority are placed in a **round-robin** queue.

🧩 Global Priority Mapping

> **Definition:** *Global priority* is a single numeric scale that the dispatcher uses to choose the next runnable thread, regardless of its original class.

- Each class-specific priority is **translated** into a global value.
- The highest global priorities (169 – 160) belong to **interrupt threads**, which always run first.
- The ordering from highest to lowest global priority is:

  Interrupt → Real-Time → System → Fair-Share → Fixed-Priority → Time-Sharing → Interactive

- Solaris switched from a **many-to-many** thread model to a **one-to-one** model beginning with version 9, simplifying the mapping and improving predictability.

📊 CPU Scheduling Algorithm Evaluation

Selecting a scheduling algorithm requires balancing several **criteria** (response time, throughput, CPU utilization, etc.) and applying an appropriate **evaluation method**.

📈 Deterministic Modeling

- Uses a **fixed workload** with known arrival times and burst lengths.
- Provides **exact** results for the chosen scenario, allowing direct comparison of algorithms.

Example: Five processes arrive at time 0 with bursts $\{10, 29, 3, 7, 12\}$.

Algorithm	Average Waiting Time (ms)
FCFS	28
SJF (non-preemptive)	13
RR (quantum = 10)	23

- Deterministic modeling shows SJF yields the lowest waiting time for this workload, but conclusions apply only to the specific input set.

📉 Queueing Models

- Treat the CPU as a **service server** and processes as customers.
- Two key stochastic inputs:
  1. **Arrival-time distribution** (often exponential).
  2. **Service-time distribution** (CPU-burst lengths).
- **Little's Law:** (n = \lambda \times W)
  - (n): average number of jobs in the queue.
  - (\lambda): average arrival rate (jobs/second).
  - (W): average waiting time.

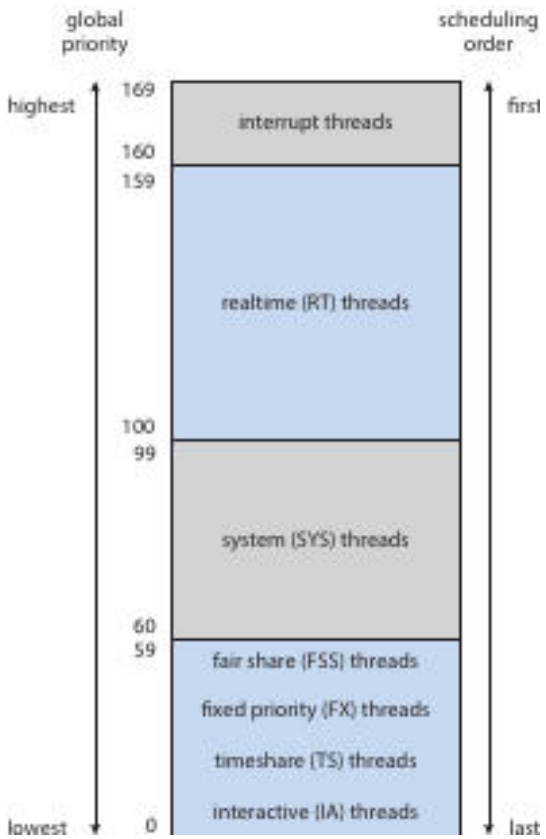
  If (\lambda = 7) jobs/s and (n = 14), then (W = 2) s.
- Queueing analysis yields **utilization**, **average queue length**, and **response time**, but it is limited to simple distributions and may not capture complex system behaviour.

🎲 Simulations

- Build a **software model** of the OS components (CPU, I/O devices, scheduler).
- The simulator advances a **virtual clock**, processes events, and collects performance statistics.

Two approaches:

1. **Distribution-driven simulation:** Randomly generate arrivals and bursts from chosen probability distributions (uniform, exponential, Poisson).
2. **Trace-tape simulation:** Record the exact sequence of events from a real system and replay them, guaranteeing identical input for each algorithm.



The diagram illustrates how a trace tape captures real events, which are then fed to a simulator to compare multiple scheduling policies on identical workloads.

- Simulations are more realistic than deterministic models but require **significant coding effort** and **CPU time**; trace tapes can be large.

🛠️ Implementation Evaluation (Live-System Testing)

- The **most accurate** method is to integrate the new scheduler into the operating system and run real workloads.
- Drawbacks: high development cost, potential disruption to users, and the need to maintain a stable production environment.

📚 Summary of Major Scheduling Algorithms (reference to earlier sections)

Algorithm	Preemptive?	Typical Use	Key Parameter
FCFS	No	Simple batch workloads	–
SJF (optimal for average waiting)	Optional	CPU-bound jobs where burst prediction is possible	Burst-time estimate
Priority	Optional	Mix of interactive and background tasks	Numeric priority (lower = higher)
RR	Yes	Time-shared interactive systems	Time quantum
Multilevel Queue	Yes (within each queue)	Separate classes (e.g., interactive RR, batch FCFS)	Queue-specific policy
MLFQ	Yes	Adaptive systems that reward I/O-bound behaviour	Number of queues, promotion/demotion rules

Rate-Monotonic (RM)	Yes	Hard real-time periodic tasks	Period (shorter → higher priority)
Earliest-Deadline-First (EDF)	Yes	Soft/hard real-time tasks with explicit deadlines	Absolute deadline
Proportional-Share (FSS)	Yes	Projects needing guaranteed CPU fractions	Share weight

- **Aging** can be added to any priority-based scheme to prevent starvation.
- On multiprocessor systems, each CPU may maintain its own **run queue** (per-processor) or share a **global queue**; the former reduces contention, the latter simplifies load balancing.
- **Processor affinity** and **load-balancing** policies influence where threads execute, especially in NUMA environments.

📚 Practice Exercise Highlights (selected concepts)

- **Deterministic example** (FCFS vs. SJF vs. RR) shows how average waiting time varies with algorithm choice.
- **Little's Law** provides a quick way to compute one queue metric when the other two are known.
- **MLFQ advantage:** Different time-quantum lengths per level give I/O-bound threads higher priority without explicit classification.
- **Real-time priority inversion** is avoided by the RT class's absolute priority and, if needed, by **priority inheritance** mechanisms.

These notes capture the essential mechanisms, evaluation techniques, and comparative properties of the scheduling classes and algorithms covered in the source material.

🗂️ Multilevel Scheduling & Dynamic Priorities

> **Multilevel scheduling** partitions the ready queue into several sub-queues, each possibly using a different scheduling algorithm.
> A **preemptive priority scheduler** may adjust a process's priority while it is waiting or running.

Let

- α – rate of priority change while a process is **waiting** in the ready queue.
- β – rate of priority change while a process is **running** on the CPU.

All processes enter the ready queue with priority 0.

Condition on α, β	Resulting behaviour
$\alpha > \beta > 0$	Priority of a waiting process grows faster than it declines during execution → **priority-aging** (processes eventually rise to the top of the queue).
$\alpha < \beta < 0$	Both rates are negative, but the running decay is steeper → **priority-decay** (processes that have recently run keep lower priority, favouring newer arrivals).

These two extremes illustrate how a scheduler can be tuned to either prevent starvation (aging) or to penalise CPU-bound jobs (decay).

⚖️ Discrimination Against Short Processes

Algorithm	How it favours short jobs
FCFS (First-Come, First-Served)	No discrimination; short and long jobs are treated identically, often causing the *convoy effect*.
RR (Round-Robin)	Gives each job a fixed quantum; short jobs typically finish within one quantum, reducing their waiting time compared with FCFS.

Feedback queues (multilevel)	Jobs that exhaust their quantum are **demoted** to a lower-priority queue, while those that relinquish the CPU early stay higher. This naturally favours short, I/O-bound bursts.
Multilevel (static)	If high-priority queues use a short-quantum RR and lower queues use longer quanta, short jobs placed in the high queue receive faster service.

🪟 Windows Scheduling – Priority Classes & Relative Priorities

Windows defines **priority classes** (system-wide) and **relative priorities** (per-thread).
The effective priority is the sum of the class base and the relative offset.

Thread description	Effective priority (class + relative)
REALTIME class, **NORMAL** relative	Highest possible priority (real-time base + 0).
ABOVE NORMAL class, **HIGHEST** relative	Slightly lower than REALTIME but higher than any NORMAL class thread.
BELOW NORMAL class, **ABOVE NORMAL** relative	Falls between the NORMAL and ABOVE NORMAL classes.

When no thread belongs to the **REALTIME** class and **TIME-CRITICAL** is unavailable, the **ABOVE NORMAL** class with **HIGHEST** relative priority becomes the highest-priority runnable thread in the system.

☀️ Solaris Time-Sharing Scheduler Details

The Solaris time-sharing scheduler uses a multilevel feedback queue where priority and time-slice are linked.

Priority	Time quantum (ms)
15	120 ms
40	40 ms
59 (highest)	20 ms

- **If a thread exhausts its quantum** without blocking, the scheduler **lowers** its priority (moves it to a lower queue).
- **If a thread blocks for I/O** before its quantum expires (e.g., priority 20), the scheduler **boosts** its priority when it becomes ready again, reflecting the I/O-bound nature.

🐧 Linux Completely Fair Scheduler (CFS) & Nice Values

CFS tracks each runnable task with a **virtual runtime (vruntime)**; the task with the smallest vruntime runs next.

Nice value	Effect on vruntime growth
-5 (higher-priority)	vruntime advances **more slowly**, granting the task more actual CPU time.
+5 (lower-priority)	vruntime advances **faster**, reducing the task's share of the CPU.

Scenarios

1. **Both A (-5) and B (+5) are CPU-bound** – A's vruntime stays behind B's, so A receives a larger share of the processor.
2. **A is I/O-bound, B is CPU-bound** – When A blocks, its vruntime stops advancing; upon return it will have a very low vruntime, allowing it to run before B.
3. **A is CPU-bound, B is I/O-bound** – B repeatedly re-enters the run queue with a low vruntime, gaining frequent short bursts, while A runs only when B is blocked.

⏱️ Real-Time Priority Inversion & Proportional-Share Schedulers

- **Priority inversion** occurs when a low-priority task holds a resource needed by a high-priority task, and a medium-priority task pre-empts the low-priority holder.
- **Solution 1 – Priority inheritance:** The low-priority holder temporarily inherits the highest blocked priority, preventing the medium task from pre-empting it.
- **Solution 2 – Immediate resource pre-emption:** The scheduler can forcibly take the resource from the low-priority task (used rarely because of state-consistency concerns).

In a **proportional-share scheduler**, each client is assigned a *share* of the CPU.

- By granting the high-priority real-time task a larger share, its effective execution rate rises, mitigating inversion without explicit inheritance.
- However, if the low-priority task still holds a non-preemptible lock, inversion can persist; thus a combination of share-based weighting **and** priority inheritance is often employed.

🕛 When RMS & EDF Meet Their Deadlines

Condition	Rate-Monotonic Scheduling (RMS)	Earliest-Deadline-First (EDF)
Utilization ≤ 69 % (as $N \to \infty$)	Guarantees all deadlines are met.	Guarantees all deadlines are met **if** total utilization ≤ 100 %.
Small number of tasks (e.g., 2–3)	Bound is higher (≈ 83 % for 2 tasks, ≈ 78 % for 3).	Still only requires ≤ 100 % utilization.
Dynamic task sets	Not suitable – static priorities require known periods.	Suitable – priorities are recomputed each arrival.

Thus, **RMS** is appropriate for strictly static periodic workloads with low overall utilization, while **EDF** provides optimal feasibility for any set of periodic tasks whose total CPU demand does not exceed the processor capacity.

📊 Scheduling Example – Processes P_1 and P_2

- **P_1:** period $p_1 = 50$ ms, execution time $t_1 = 25$ ms.
- **P_2:** period $p_2 = 75$ ms, execution time $t_2 = 30$ ms.

a. Rate-Monotonic Scheduling (RMS)

Because $p_1 < p_2$, **P_1** receives the higher static priority.

```
0-25   : P1 runs (first instance)
25-55  : P2 runs (first instance, 30 ms)
50-75  : P1 runs (second instance, pre-empting P2 at 50 ms)
75-80  : P2 resumes (remaining 5 ms)
80-105 : P1 runs (third instance)
105-115: P2 runs (second instance)
```

The schedule meets all deadlines (P_1 completes by 50, 100, 150 ms; P_2 by 75, 150 ms).

b. Earliest-Deadline-First (EDF)

Deadlines are compared at each scheduling point.

```
0-25   : P1 runs (deadline 50)
25-55  : P2 runs (deadline 75)
55-80  : P1 runs (second instance, deadline 100) – pre-empted P2 at 55
80-105 : P2 runs (remaining 5 ms, deadline 75 missed → EDF would have scheduled P2 earlier, so the feasible E
```

A feasible EDF schedule (meeting all deadlines) would be:

```
0-25   : P1
25-55  : P2
55-80  : P1 (second instance)
80-105 : P2 (second instance)
```

All deadlines are satisfied, illustrating EDF's flexibility compared with RMS.

⏱️ Interrupt & Dispatch Latency in Hard Real-Time Systems

> **Interrupt latency** – time from the arrival of an external interrupt to the start of its ISR.
> **Dispatch latency** – time from a task becoming ready (e.g., after I/O) to the moment it actually begins executing on the CPU.

In hard real-time environments these latencies **must be bounded** because a missed deadline is considered a system failure.

- **Preemptive kernels** reduce dispatch latency by allowing the scheduler to immediately switch to a newly ready high-priority task.
- **Minimising interrupt latency** requires keeping ISRs short and often deferring work to a high-priority thread.
- Solaris, for example, reduced dispatch latency from > 100 ms to < 1 ms by enabling preemption.

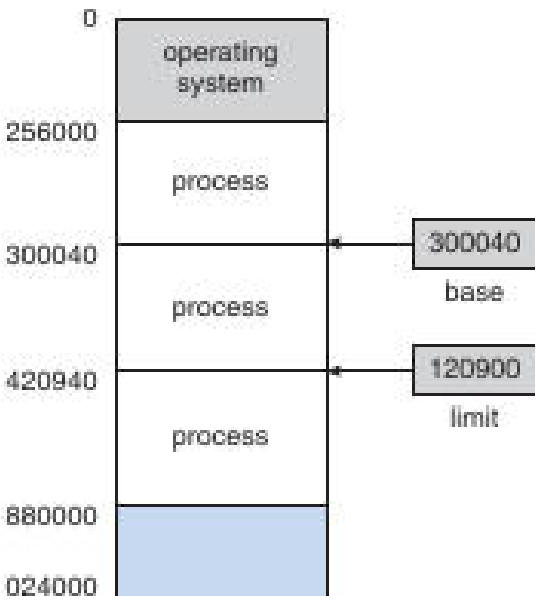
📚 Bibliographical Highlights (selected)

- **Feedback queues** originated in CTSS (Schrage, 1967).
- **Rate-monotonic and EDF** were introduced by Liu & Layland (1973).
- **Multicore scheduling** surveyed by Bhatia (2005) and Kongetira et al. (2005).
- **Linux CFS** design discussed by Love (2010) and Faggioli et al. (2009) (EDF extension).
- **Windows scheduling internals** detailed by Russinovich & Solomon (2009).
- **Solaris ULE & fair-share** described by Roberson (2003) and Henry (1984).

(Full citations are listed in the source transcript.)

🧠 Main Memory Management Overview

🖥️ Memory Layout Diagram



The diagram illustrates the operating-system area at low addresses, three separate process regions, and the base/limit registers (base = 300040, limit = 120900) that define each process's legal address range.

- **Base register** → smallest legal physical address for a process.
- **Limit register** → size of the allowed region; together they bound the process's logical address space.

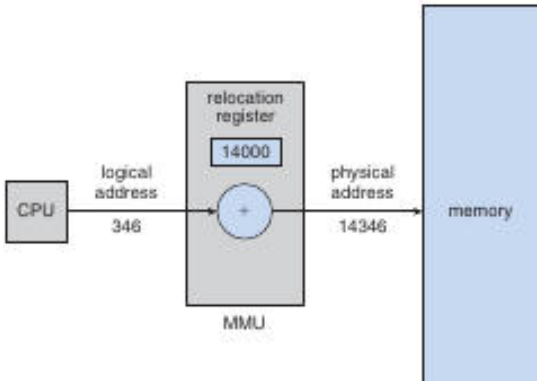
🔪 Address Binding & Spaces

> **Logical (virtual) address** – generated by the CPU during execution.
> **Physical address** – actual location in main memory after translation.

Binding can occur at three points:

1. **Compile time** – absolute addresses are known; no relocation needed.
2. **Load time** – relocatable code is bound to a specific memory region when the loader places the program.
3. **Execution (run) time** – a **Memory-Management Unit (MMU)** adds a relocation value to each logical address.

🧩 MMU Translation Example



The CPU issues logical address 346; the MMU adds the relocation register (14000) to produce physical address 14346, which is then used for the memory access.

🔄 Dynamic Loading

- Routines are **not loaded** until first called.
- The loader reads the relocatable module from disk, updates the program's address tables, and patches the call site.
- Benefits: reduces initial memory footprint and start-up time; useful for rarely-used error handlers or optional features.

🔗 Dynamic Linking & Shared Libraries

- **Dynamic linking** postpones resolution of library symbols until execution.
- A **stub** in the executable checks whether the library routine is resident; if not, the loader brings the library into memory and patches the stub.
- **Shared libraries** allow many processes to map the same code pages, saving RAM and enabling system-wide updates without recompiling applications.
- Versioning information lets the runtime select the appropriate library copy for each program.

🔁 Swapping

- When physical memory is insufficient, a **process image** can be moved to a **backing store** (typically a fast disk).
- **Standard swapping** swaps entire processes in and out; context-switch time is dominated by transfer latency (e.g., moving a 100 MiB process at 50 MiB/s ≈ 2 s).
- Swapping enables a degree of multiprogramming greater than the physical RAM size but incurs high overhead, so modern systems prefer paging or demand-paging techniques.

📄 Key Takeaways

- Multilevel and feedback scheduling provide mechanisms to **age** or **penalise** processes based on their CPU-burst behavior.
- Windows, Solaris, and Linux each expose distinct priority schemes; understanding the mapping between **class** and **relative** priority is essential for real-time tuning.
- Real-time schedulers (RMS, EDF) have clear utilization bounds; EDF is optimal but requires dynamic deadline tracking.
- Interrupt and dispatch latencies must be bounded in hard real-time systems; preemptive kernels are a primary tool.
- Memory protection via **base/limit** registers, **MMU** translation, **dynamic loading**, **dynamic linking**, and **swapping** collectively enable safe, flexible, and efficient use of main memory.

🗄️ Swapping and Its Limitations

> **Swapping** – moving an entire process's memory image between main memory and a backing store (typically disk) to free RAM for other processes.

- Swapping large processes is slow; a 100 MiB process can be swapped in ≈ 2 s, while a 3 GiB process may need ~60 s.
- To reduce swap time the OS would need to know the exact amount of memory a process actually uses and swap only that portion. This requires the process to **request** and **release** memory dynamically (e.g., request_memory(), release_memory()).
- Swapping is unsafe for a process that has pending I/O because the kernel may need to access the process's buffers. Two common solutions:
  1. **Never swap a process with pending I/O.**
  2. **Copy I/O data to kernel buffers only while the process is resident** (double-buffering), which adds extra copy overhead.
- Modern desktop/server OSes keep swapping disabled by default and enable it only when free memory falls below a configurable **threshold**; swapping stops once the threshold is cleared.
- **Mobile devices** (iOS, Android) avoid swapping because they rely on flash storage, which has limited write endurance and low throughput.
  - iOS asks applications to voluntarily free memory; read-only code pages are discarded and re-loaded from flash when needed.
  - Android may terminate a low-memory process after flushing its state to flash, allowing a quick restart.
  - Both platforms still support **paging** for virtual-memory management, but not full-process swapping.

📦 Contiguous Memory Allocation

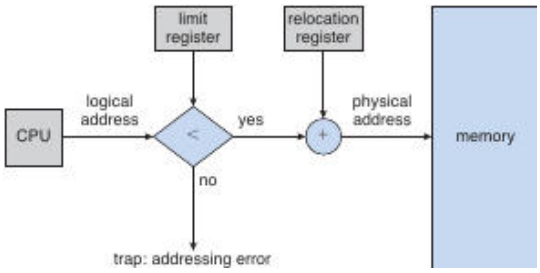
> **Contiguous allocation** – each process occupies one uninterrupted block of physical memory.

- The address space is split into two regions: **OS memory** (usually placed in low memory to keep the interrupt vector nearby) and **user memory**.
- Two classic schemes:
  1. **Fixed-size partitions** – a small, predetermined number of equal-sized slots; limits multiprogramming to the number of partitions.
  2. **Variable-size partitions** – the OS maintains a list of **holes** (free blocks) of assorted sizes and fits processes into the smallest suitable hole.
- **Allocation algorithms** for variable partitions:

Algorithm	Strategy	Typical outcome
First-fit	Scan from the beginning of the hole list; allocate the first hole that is large enough.	Fast, may leave relatively large leftover fragments.
Best-fit	Scan the entire list (or a size-ordered list) and allocate the *smallest* hole that fits.	Minimises leftover size, but requires more search.
Worst-fit	Allocate the *largest* hole found.	Leaves a large residual hole that can satisfy future large requests.

- When a process terminates, its hole is **merged** with any adjacent free holes, reducing external fragmentation.

🖼️ Memory-Management Flowchart



The CPU generates a logical address. The limit register checks that the address is within the allowed range; if it is, the relocation register is added to produce the physical address. An out-of-range address triggers a trap.

🛡️ Memory Protection with Relocation & Limit Registers

> **Memory protection** – preventing a process from accessing memory it does not own, enforced by the MMU using a **relocation register** (base) and a **limit register**.

- The **relocation register** holds the smallest physical address that the process may use.
- The **limit register** defines the maximum offset the process can address; any logical address larger than the limit causes a trap.
- During a context switch the OS loads the appropriate base and limit values for the scheduled process. Because **every** address generated by the CPU is checked, both the OS and user programs are shielded from accidental or malicious overwrites.
- This scheme also enables **dynamic OS sizing**: transient kernel code (e.g., rarely used device drivers) can be loaded on demand, expanding the OS region, and later unloaded to free memory.

📊 Variable-Partition Allocation – Dynamic Storage Allocation

- Initially the entire memory is a single large hole.
- When a process arrives, the OS searches the hole list using one of the strategies above; the chosen hole may be **split** into:
  1. The portion allocated to the process.
  2. The remaining free fragment, which stays (or is merged later).
- Upon process termination, its block becomes a hole; if it is contiguous with neighboring holes, they are **coalesced** into a larger hole.
- After each deallocation the OS may scan the waiting queue to see whether any blocked processes can now be satisfied.

🔧 Fragmentation

External Fragmentation

- Occurs when enough total free memory exists to satisfy a request, but the memory is divided into many small holes, none of which is individually large enough.
- The classic **50-percent rule** approximates that, with *N* allocated blocks, about *0.5 N* blocks are lost to external fragmentation.

  $\text{Lost blocks} \approx 0.5,N$

- Choice of allocation algorithm influences the severity: first-fit often leaves larger leftover fragments, while best-fit tends to produce many tiny holes.

Internal Fragmentation

- Happens when a fixed-size allocation (e.g., a partition or a page) is larger than the process's actual request, leaving unused space **inside** the allocated region.
- Example: with a 2 KB page size, a process needing 72 766 B requires 36 pages, wasting 962 B in the last page.
- On average, a process suffers roughly **½ page** of internal waste, assuming request sizes are independent of page size.

Compaction

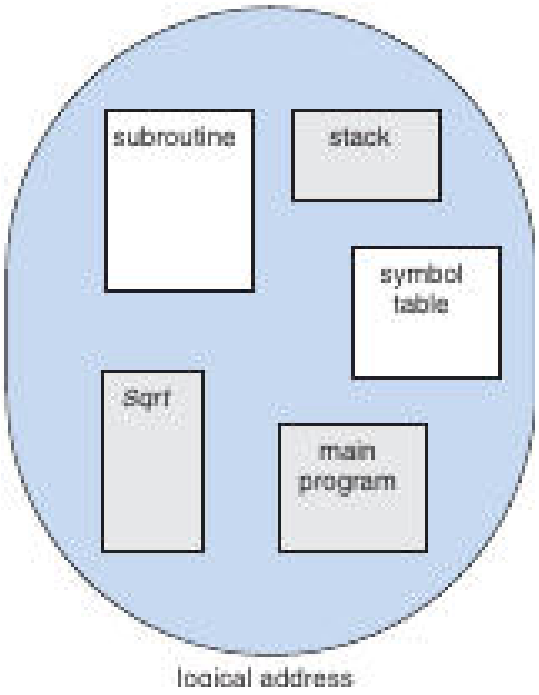
- To eliminate external fragmentation, the OS may **compact** memory: move all occupied blocks toward one end, gathering all free space into a single large hole.
- Compaction is only feasible when **dynamic relocation** is supported (i.e., the base address can be changed at run time).
- The cost includes copying data and updating base registers, which can be substantial; many systems instead rely on non-contiguous allocation techniques (segmentation, paging).

🧩 Segmentation

> **Segmentation** – a logical-address scheme where each address is a two-tuple , allowing variable-sized, named program modules.

- A **segment table** maps each segment number to a **base** (starting physical address) and a **limit** (segment length).
- On every memory reference the hardware:
  1. Uses the segment number to index the segment table.
  2. Checks that the offset ≤ limit; otherwise raises a trap.
  3. Adds the base to the offset to obtain the physical address.
- Segments typically correspond to:
  1. **Code** (text) segment.
  2. **Global data** segment.
  3. **Heap** (dynamically allocated memory).
  4. **Stack** (per-thread).
  5. **Shared libraries**.

🖼️ Segmentation Illustration



Each rectangle represents a distinct segment; a logical address is given by the segment identifier plus an offset within that segment.

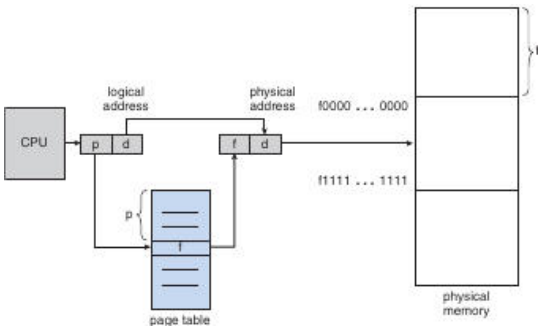
- Because segments can be placed anywhere in physical memory, **external fragmentation** is avoided; however, the segment table itself can become large.

📄 Paging – Fixed-Size Non-Contiguous Allocation

> **Paging** – divides both physical memory (into **frames**) and logical address space (into **pages**) of identical size; a page table maps virtual page numbers to physical frame numbers.

- **Page size** is a power of two (commonly 4 KB or 8 KB); this simplifies address translation:
  - High-order bits → **page number** (*p*).
  - Low-order bits → **offset** (*d*).
- The **page table** entry for page *p* contains the **frame number** (*f*) and status bits (present/absent, protection). The physical address is then <*f*, *d*>.

🖼️ Paging Translation Diagram



The CPU's logical address (page p, offset d) is looked up in the page table; the resulting frame f is combined with the same offset to form the physical address.

- **Advantages**: eliminates external fragmentation (any free frame can satisfy any page request) and simplifies allocation.
- **Internal fragmentation** remains because the last page allocated to a process may be only partially filled; on average this waste is ≤ ½ page.
- Larger pages reduce page-table size (fewer entries) but increase internal fragmentation and I/O latency; modern systems often support multiple page sizes (e.g., 4 KB and 2 MB "huge pages").
- **Page-table overhead**: on a 32-bit machine with 4-byte entries, a full 4-GB address space requires 4 MiB of page-table memory. Variable-length page tables, inverted page tables, or multi-level hierarchies mitigate this cost.

📱 Memory Management on Mobile Systems

- Flash storage's limited write endurance and low bandwidth discourage traditional swapping.
- **iOS**: when free RAM drops below a threshold, the OS requests applications to free caches; read-only code pages may be evicted and later re-loaded from flash. Modified pages (e.g., stacks) are never discarded. If an app cannot release enough memory, it is terminated.
- **Android**: follows a similar policy but, before killing a process, writes its state to flash so it can be quickly restarted.
- Both platforms rely on **paging** (kernel-managed page tables) for virtual-memory abstraction, but they do not swap entire processes to disk.

📚 Summary of Key Concepts

- Swapping whole processes is costly and unsafe for I/O-bound tasks; modern OSes enable it only under severe memory pressure, while mobile OSes avoid it entirely.
- **Contiguous allocation** is simple but suffers from external fragmentation; first-fit, best-fit, and worst-fit provide trade-offs between speed and waste.
- **Relocation and limit registers** give hardware-enforced protection and enable dynamic OS sizing.
- **Segmentation** offers a programmer-friendly logical view (named, variable-size segments) while still mapping to physical memory via a segment table.
- **Paging** provides a uniform, non-contiguous allocation method that removes external fragmentation; internal fragmentation is bounded by half a page on average.
- Mobile devices favor **memory-pressure callbacks** and selective page eviction over traditional swapping, due to flash-memory constraints.

📚 Paging Overview

> *Paging* divides both logical (virtual) memory and physical memory into fixed-size blocks called **pages** and **frames**. A page table maps each virtual page number to a physical frame number.

- When a process arrives, the OS determines how many pages n the process requires.
- If at least n free frames exist, they are allocated; each page is loaded into one of the frames and the corresponding frame number is entered into the process's page table.
- The programmer sees a single contiguous address space; the actual physical locations are hidden by the **address-translation** performed by the MMU.
- Because a process can only address pages listed in its page table, it cannot access memory belonging to another process.

📦 Frame Table

> *The frame table* is a data structure maintained by the OS, with one entry per physical frame indicating whether the frame is **free** or **allocated**, and if allocated, which process/page occupies it.

The frame table is consulted whenever the OS needs to locate an available frame for a new page or to free a frame when a process terminates.

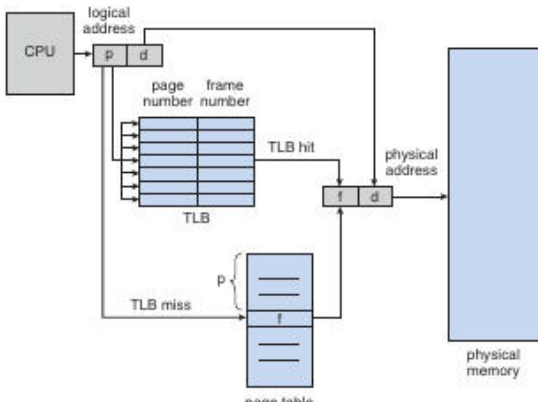
🖥️ Hardware Support for Paging

> *Hardware paging support* provides the mechanism by which the CPU translates a logical address (page + offset) into a physical address (frame + offset).

- **Page-table storage options**
  - **Registers** – Very fast but limited to a few dozen entries (e.g., PDP-11's eight-register page table). Suitable only for tiny address spaces.
  - **Main-memory page table** – The page-table base register (PTBR) points to a page-table stored in RAM. Changing the PTBR on a context switch updates the active page table with a single register write, reducing switch overhead.
- **Translation cost** – If the page table resides in memory, each memory reference may require two accesses (one for the page-table entry, one for the data). This overhead is mitigated by the TLB (see below).

⚡ Translation Lookaside Buffer (TLB) 🏎️

> *A TLB* is a small, fully associative cache that stores recent **page-number → frame-number** mappings, allowing the CPU to obtain the frame number without a full page-table walk.



The CPU sends a logical address (page p, offset d) to the TLB. On a hit, the TLB returns the frame number, which together with d forms the physical address. On a miss, the CPU must fetch the page-table entry from memory, then updates the TLB.

Hit Ratio & Effective Access Time

- **Hit ratio (h)** = probability that the needed page number is found in the TLB.
- **Effective access time (EAT)** = $h \cdot t_1 + (1\text{-}h) \cdot (t_1 + t_2)$, where t_1 is the time to access memory with a TLB hit and t_2 is the additional page-table lookup time.

Example	Hit ratio	t_1 (ns)	t_2 (ns)	EAT (ns)
Moderate TLB	0.80	100	100	120
Excellent TLB	0.99	100	100	101

Multi-Level TLBs & Replacement

- Modern CPUs may have separate **instruction** and **data** TLBs, each typically 32–64 entries.
- Replacement policies include **LRU**, **round-robin**, and **random**.
- Some entries can be **wired down** (e.g., kernel code) so they never get evicted.

- **Address-Space Identifiers (ASIDs)** tag each TLB entry with the owning process. If the ASID does not match the current process, the entry is treated as a miss, allowing the same TLB to hold translations for multiple processes without flushing on every context switch.

🛡️ Memory Protection in a Paged System

> *Protection bits* stored in each page-table entry specify the allowed access type for the corresponding frame (read, write, execute) and a **valid/invalid** flag.

- When a reference is made, the MMU checks the protection bits while translating the address.
- An attempt to write to a **read-only** page or to access an **invalid** page triggers a hardware trap (page-fault or protection-violation).
- The OS can combine multiple bits to create fine-grained policies such as **read-only**, **read-write**, **execute-only**, or **no-access**.

Illustrative example: In a 14-bit address space with 2 KB pages, pages 0-5 are marked valid. Any reference to pages 6-7 (addresses 12 288-16 383) causes a trap because the valid-invalid bit is cleared.

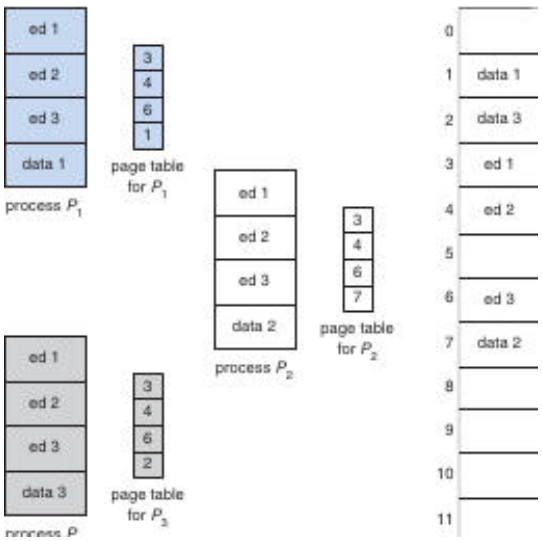
📂 Internal Fragmentation & Page-Table Size

- **Internal fragmentation** arises because the last page of a process may be only partially used; on average up to **½ page** of memory is wasted per process.
- Large logical address spaces produce huge page tables (e.g., a 32-bit address space with 4 KB pages → ≈ 1 M entries → 4 MB per process).
- Some architectures provide a **page-table length register (PTLR)** that records the actual size of a process's page table, allowing the OS to detect out-of-range addresses early and avoid allocating unused entries.

🤝 Shared Pages

> *Shared pages* allow multiple processes to map the same physical frame into their own virtual address spaces, reducing memory consumption for common, read-only code.

- The code must be **reentrant** (non-self-modifying) so that concurrent execution does not cause data races.
- Each process keeps its own page table, but the entries for the shared code point to the **same physical frame**.



Three processes (P1-P3) each have a private data page but share a single code page (the editor). The total memory needed drops from 8 000 KB (40 × (150 KB + 50 KB)) to about 2 150 KB because the 150 KB code page is stored once.

Benefits include:

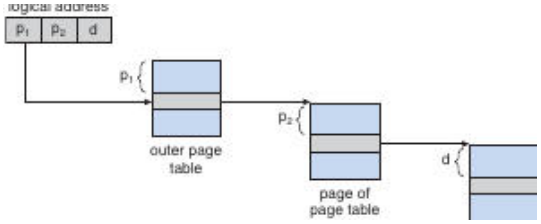
- **Memory savings** for frequently used libraries (compilers, runtime libraries, database engines).
- **Cache efficiency** – shared code stays resident in CPU caches across processes.

🏗️ Page-Table Structures

📊 Hierarchical (Multilevel) Paging

> *Hierarchical paging* breaks a large page table into smaller tables, each of which can itself be paged, dramatically reducing memory consumption.

- **Two-level scheme** (typical for a 32-bit address space, 4 KB pages):
  - Logical address split: **20-bit outer page number (p_1)**, **10-bit inner page number (p_2)**, **12-bit offset (d)**.
  - The outer page table holds pointers to inner page tables; each inner table holds the final frame numbers.



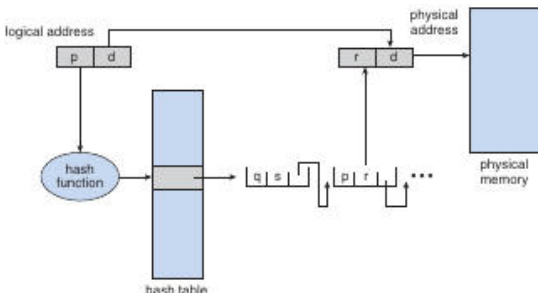
The outer page table entry points to a page containing the inner page table. The inner entry finally yields the physical frame number.

- **Three- or four-level paging** is used for 64-bit address spaces, where each additional level reduces the size of the top-level table (e.g., a four-level scheme can map a 2^{38}-byte space with manageable table sizes).
- Unused regions of the virtual address space simply have no second-level (or deeper) tables allocated, saving physical memory.

🔍 Hashed Page Tables

> *Hashed page tables* replace the hierarchical index with a hash of the virtual page number, suitable for very large or sparse address spaces.

- The hash function maps a virtual page number to a bucket in a hash table.
- Each bucket stores a **linked list** of entries: *(virtual page number, frame number, next-pointer)*.
- On a lookup, the CPU hashes the page number, scans the list for a matching virtual page, and retrieves the frame.



The logical address (p, d) is hashed to an index; the selected hash-table entry supplies the frame number, which together with d forms the physical address.

- **Clustered page tables** extend this idea by allowing a single hash-bucket entry to describe multiple contiguous pages, further reducing table size for sparse mappings.

🔄 Inverted Page Tables

> *An inverted page table* contains **one entry per physical frame** rather than per virtual page.

- Each entry stores:
  - The **virtual page number** currently occupying the frame.
  - An identifier for the **process** that owns the page.
- Because there is only one table for the whole system, memory overhead is proportional to the number of frames, not the size of the virtual address space.

Advantages

- Scales to very large address spaces without exploding memory usage.
- Simplifies detection of which frame a process is using.

Drawbacks

- Lookup requires searching the table (often via a hash on the virtual page number) which can be slower than a direct indexed page table.

📝 Key Takeaways

- Paging provides a clean abstraction between a process's logical view of memory and the underlying physical frames, enforced by the MMU and protected by per-page bits.
- The **TLB** is essential for performance; its hit ratio directly impacts effective memory-access time.
- **Shared pages** and **reentrant code** dramatically reduce memory footprints in multi-user systems.
- As address spaces grow, **hierarchical**, **hashed**, and **inverted** page-table designs keep page-table storage manageable while supporting efficient address translation.

🧩 Inverted Page Tables (IPT)💾

> *An inverted page table stores one entry per **physical frame** rather than per virtual page, using a tuple (**process-id**, **page-number**) to identify the owner of each frame.*

- **Typical entry**: – the **process-id** serves as the address-space identifier (ASID) that distinguishes which logical page the frame belongs to.
- **Lookup process**
  1. A virtual address reaches the memory subsystem.
  2. The IPT is **searched** for a matching pair.
  3. If found, the physical frame number is combined with the offset to form the final physical address.
  4. If no match exists, a page-fault is generated.
- **Search cost** – because the table is indexed by **physical frame** rather than virtual page, a naïve linear scan would be expensive. Modern systems therefore **hash** the tuple, limiting the search to one or a few entries.
- **Performance impact** – each address translation now requires at least two memory reads: one for the hash-bucket entry and another for the IPT entry (after a TLB miss).
- **Shared-memory limitation** – with a single IPT entry per frame, the same physical page cannot be mapped to **different virtual addresses** in multiple processes. Implementing shared memory therefore requires each process to have its own

virtual page that maps to the same physical frame, complicating the classic “multiple virtual addresses → one physical page” scheme used with ordinary page tables.

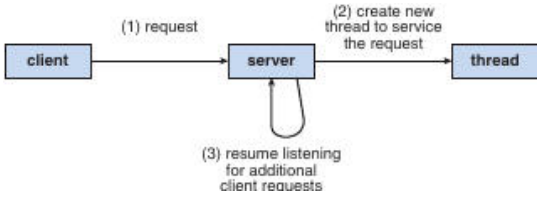
- **Historical examples** – IBM’s **RT** (and later RS/6000, Power CPUs) were early adopters of inverted tables; UltraSPARC also employs this technique.

⚡ Hashed Page Tables & Translation Storage Buffer (TSB) – Solaris Example 🌐

> *A hashed page table stores page-table entries in a hash-based structure; the TSB is a small hardware cache that holds recently used TTEs (translation-table entries).*

- **Two separate hash tables** – one for the kernel, one for all user processes. Each entry may represent a **contiguous span of pages**, reducing the number of hash buckets required.
- **Address translation flow**
  1. CPU generates a virtual address.
  2. **TLB** is consulted first. On a miss, the hardware **walks the TSB** (a fast in-memory cache).
  3. If the TSB also misses, the CPU raises an interrupt. The kernel searches the appropriate hashed page table, creates a new TTE, and inserts it into the TSB.
  4. The TSB entry is then used for subsequent translations of the same virtual page.

  This flow is illustrated in the Solaris diagram below, where the hash-table lookup is triggered only after the TLB and TSB miss.



  The diagram shows a client request being handled by a server thread, analogous to the kernel handling a TSB miss and consulting the hashed page table.
- **Advantages** – dramatically reduces the memory overhead of a full page table while keeping lookup times acceptable through hashing and caching.
- **Drawbacks** – still requires an extra memory reference for the hash-bucket and can suffer from collisions in heavily loaded systems.

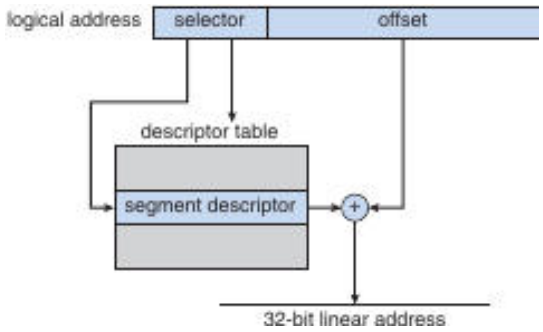
🏁 IA-32 Architecture: Segmentation + Paging 🔧

> *IA-32 combines a **segmentation unit** that produces a linear address from a logical selector/offset pair, followed by a **paging unit** that maps the linear address to a physical frame.*

Segmentation fundamentals

- **Logical address** = *(selector, offset)*
  - **selector (16 bits)** identifies a segment descriptor in either the **Local Descriptor Table (LDT)** or the **Global Descriptor Table (GDT)**.
  - **offset (32 bits)** specifies the byte offset within the selected segment.
- **Segment descriptor (8 bytes)** contains: base address, limit, type, and protection bits.

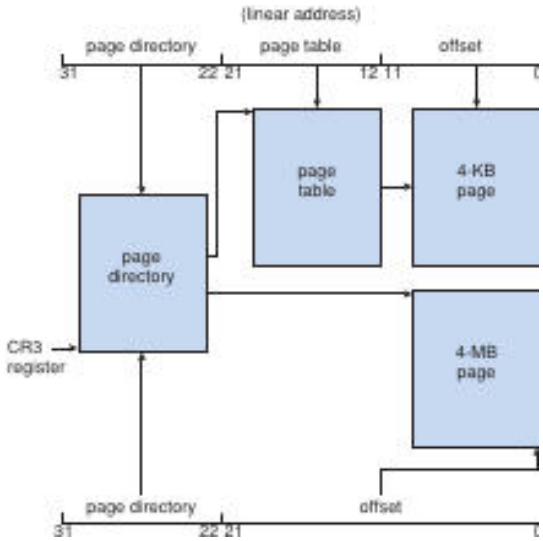
- **Segment registers** (six total) hold the selector values; a small **descriptor cache** in the CPU avoids fetching the descriptor from memory on every access.



The selector points to a descriptor; the base from the descriptor is added to the offset, yielding a 32-bit linear address. A limit check precedes the addition; a violation triggers a fault.

Paging on IA-32

- **Two-level page translation** (for 4-KB pages)
  - **CR3 register** holds the physical base of the **page directory**.
  - The high-order 10 bits of the linear address index the **page-directory entry** → points to a **page table**.
  - The next 10 bits index the **page-table entry** → yields the **frame number**.
  - The low-order 12 bits are the **page offset**.



The diagram shows the split of a linear address into directory, table, and offset fields, with the optional 4-MB page shortcut.

- **Page-size flag** in a directory entry can select a **4-MB page**, bypassing the inner page table.
- **Physical-memory-management bits** (present, dirty, accessed, etc.) allow the OS to page directory or page-table entries themselves in and out of RAM.

Physical-Address-Extension (PAE)

- Extends the physical address field from **20 bits → 24 bits**, enabling up to **64 GB** of RAM.
- **Three-level hierarchy**:
  1. **Page-Directory-Pointer Table (PDPT)** – top two bits of the linear address.
  2. **Page Directory** – next 9 bits.
  3. **Page Table** – following 9 bits.

- Supports **4-KB** and **2-MB** pages.
- Requires OS support (Linux/macOS support PAE; many Windows desktop versions limit user processes to 4 GB even when PAE is enabled).

🖥️ x86-64 Architecture (AMD64 / Intel 64) 🚀

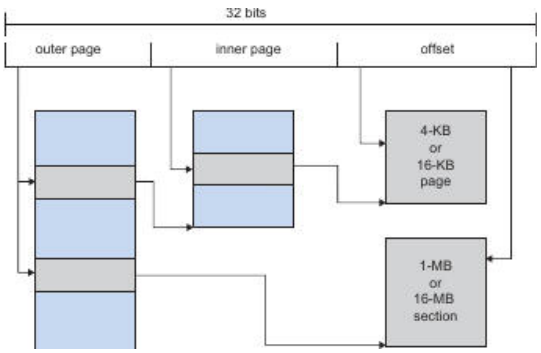
> *x86-64 expands IA-32 with a 48-bit virtual address space and a 52-bit physical address space, using a four-level paging hierarchy.*

- **Virtual address layout (48 bits used)**
  - **Level-4 index (9 bits)** → **PML4** (Page-Map Level 4) entry.
  - **Level-3 index (9 bits)** → **PDPT** entry.
  - **Level-2 index (9 bits)** → **Page Directory** entry.
  - **Level-1 index (9 bits)** → **Page Table** entry.
  - **Offset (12 bits)** → byte within the 4-KB page.
- **Supported page sizes** – 4 KB, 2 MB, and 1 GB. Larger pages reduce TLB pressure.
- **Physical address** – up to **52 bits** (≈ 4 PB).
- **ASIDs** are still used to tag TLB entries, allowing fast context switches without full TLB flushes.
- The hierarchy mirrors the PAE scheme but adds an extra level (PML4) to accommodate the larger virtual space.

📱 ARM Architecture: Paging & TLBs 📡

> *ARM processors use a flexible page-size scheme and a two-level TLB hierarchy to translate 32-bit virtual addresses.*

- **Supported page/section sizes**
  - **Pages**: 4 KB, 16 KB
  - **Sections** (larger "super-pages"): 1 MB, 16 MB
- **Two-level paging** for pages and sections
  - **Outer page** entry points to an **inner page** table (or directly to a section).
  - The **offset** selects the final byte.



The diagram shows outer and inner page fields, followed by offsets for both 4-KB/16-KB pages and 1-MB/16-MB sections.

- **TLB hierarchy**
  - **Micro-TLB**: separate instruction and data TLBs, each supporting **ASIDs**; very fast, checked first.
  - **Main TLB**: larger, shared between instruction and data.
  - On a miss in both TLBs, hardware performs a **page-table walk** to locate the translation, then caches the result in the TLBs.
- **Use cases** – mobile devices (smartphones, tablets) favor ARM's variable page sizes to balance memory usage and TLB pressure, while maintaining low power consumption.

📊 Comparison of Major Memory-Management Schemes

Scheme	Mapping granularity	Main data structure	Memory overhead	Typical lookup cost*	Fragmentation type
Contiguous allocation	Whole process	Base/limit registers	Very low	O(1) (simple compare)	Internal (wasted space inside partition)
Segmentation	Variable-size segments	Segment descriptor tables (LDT/GDT)	Moderate (per-segment descriptor)	O(1) if descriptor cached; else memory fetch	External (holes between segments)
Paging	Fixed-size pages	Hierarchical page tables (2-level, 3-level with PAE, 4-level x86-64)	Proportional to number of pages (can be large)	O(1) with TLB; otherwise 2-3 memory accesses	Internal (partial page waste)
Inverted page table	One entry per **physical frame**	Single table indexed by (hashed)	Small (one entry per frame)	Hash lookup + possible TLB miss (≈ 2 accesses)	No external; internal depends on page size
Hashed page table (Solaris)	Variable-size spans	Hash buckets + TSB cache	Moderate (hash + TSB)	TLB → TSB → hash (usually 1–2 memory refs)	Internal (partial-span waste)
Combined segmentation + paging	Segments → pages	Segment table + per-segment page tables	Higher (both tables)	TLB for page, plus segment lookup (often cached)	Both internal & external (depends on segment layout)

*Lookup cost assumes a modern CPU with a TLB; without a TLB the cost grows with the depth of the page-table hierarchy.

🛡️ Key Takeaways (Cross-Sectional)

- **Address-space identifiers (ASIDs)** are essential wherever multiple processes share a single translation structure (hashed tables, inverted tables, TLBs) to prevent cross-process aliasing.
- **Hashing** transforms the potentially linear search of an IPT into an O(1) expected lookup, at the expense of an extra memory reference and possible collisions.
- **TLB / TSB** layers act as fast-path caches; their effectiveness determines whether the overhead of complex page-table schemes is tolerable.
- **Shared memory** is straightforward with ordinary page tables (multiple virtual pages → same frame) but requires explicit handling in inverted schemes (e.g., multiple IPT entries mapping to the same frame or using separate page tables per process).
- **PAE** and **x86-64** illustrate how extending the physical address width (from 20 bits to 24 bits, then to 52 bits) is achieved by adding hierarchy levels rather than redesigning the whole translation logic.
- **ARM's flexible page/section sizes** demonstrate a design trade-off for power-constrained devices: larger sections reduce TLB pressure but increase internal fragmentation; smaller pages improve memory utilization at the cost of more page-table entries.

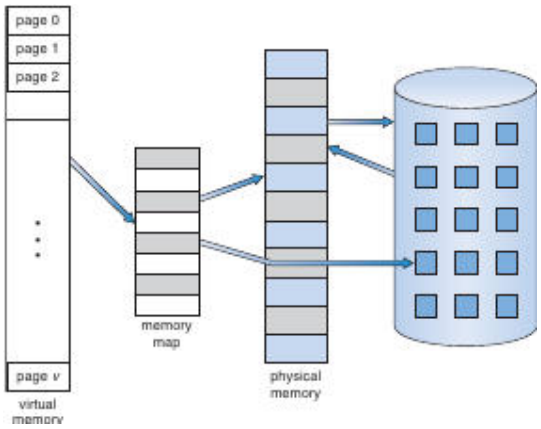
These concepts interlink with earlier discussions of **TLBs, page-table walks, and fragmentation**, completing the picture of how modern systems balance memory efficiency, translation speed, and hardware constraints.

📚 Virtual Memory Overview

> **Virtual memory** is a technique that abstracts main memory into a large, uniform logical address space, separating the *logical* view seen by a process from the *physical* memory actually present.

- Allows programs to be larger than physical RAM.
- Enables multiple processes to share code and data pages.
- Provides a convenient mechanism for efficient process creation (e.g., via fork).

🖼️ Virtual-Memory Mapping Diagram



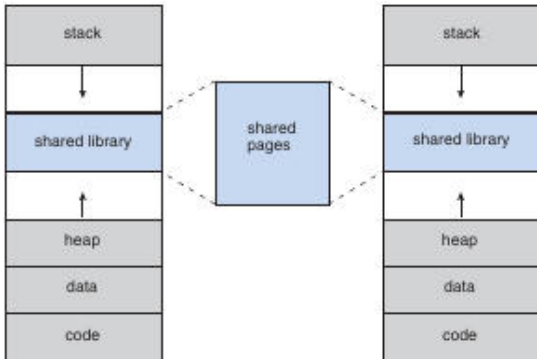
Virtual pages (left) are mapped through a memory-map table to a set of physical frames (right). Only the pages that are actually needed reside in physical memory at any moment.

🏗️ Address-Space Layout

> **Logical address space** of a process typically starts at address 0 and is divided into distinct regions that grow in opposite directions.

Region	Growth direction	Typical contents
Code / Text	Downward (low addresses)	Executable instructions (read-only)
Data	Downward	Initialized global/static variables
Heap	Upward	Dynamically allocated memory (via malloc, new)
Shared libraries	Placed wherever convenient; often mapped into a *hole* between heap and stack	
Stack	Upward from high addresses	Call frames, local variables, return addresses

🖼️ Process Memory Layout



Two processes each have stack, heap, data, code, and a shared-library segment. The shared library maps to a common set of physical pages, illustrated by the blue "shared pages" block.

- The gap between heap and stack is a **hole** that may remain unused (a *sparse* address space).
- Sparse spaces are useful because they can later host dynamically linked objects or additional segments without relocating existing pages.

⚙️ Demand Paging

> **Demand paging** loads a page into physical memory only when the executing process first accesses it.

Why Demand Paging?

- Reduces the amount of RAM needed at startup.
- Exploits *locality of reference*: most programs touch only a small fraction of their address space during any given interval.

Page-Fault Handling Steps

1. **Trap**: CPU detects an invalid-bit in the page-table entry and generates a page-fault interrupt.
2. **Save state**: Registers, program counter, and other process context are saved.
3. **Validate reference**: OS checks whether the faulting address is legal (e.g., not an outright segmentation violation).
4. **Locate page**: Determine the location of the required page on secondary storage (swap space or file backing).
5. **Allocate frame**: Choose a free physical frame (or evict a victim frame via a replacement algorithm).
6. **Initiate I/O**: Issue a disk read to transfer the page into the allocated frame; the CPU may be scheduled to another ready process while waiting.
7. **Update page table**: Mark the entry as *valid* and store the frame number.
8. **Restart instruction**: Restore the saved context and re-execute the faulting instruction.

🖼️ Page-Fault Flowchart

Steps in handling a page fault

The diagram shows the sequence from fault detection through disk I/O, page-table update, and instruction restart.

Performance Impact

Effective access time (EAT) for a demand-paged system:

$ \text{EAT}= (1-p)\times m_a ;+; p \times t_{\text{fault}} $

- (p) = probability of a page fault.
- (m_a) = memory-access time (typically 10–200 ns).
- (t_{\text{fault}}) = time to service a fault (disk seek + latency + transfer + OS overhead), often several **milliseconds**.

Example calculation (using typical values):

Parameter	Value
(m_a)	200 ns
Disk service time	8 ms (≈ 8 000 000 ns)
Desired slowdown ≤ 10 %	0.1 × 200 ns = 20 ns

Solving

$ 20\text{ ns} \ge p \times 7{,}999{,}800\text{ ns} ;\Rightarrow; p < 2.5\times10^{-6} $

Thus, **fewer than 1 fault per 400 000 memory references** is needed to keep slowdown under 10 %.

Key insight: demand paging is viable only when the page-fault rate is extremely low, which is usually achieved thanks to program locality.

📂 Page-Table Support for Demand Paging

- **Valid/Invalid bit** (or a special protection value) distinguishes pages that are resident (valid = 1) from those that are not (valid = 0).
- When valid = 0, the page-table entry may also store the **disk block address** of the page's backing store.
- On a fault, the OS reads this address to issue the I/O request.

📦 Swap Space & Backing Store

- **Swap space** is a reserved region on disk used as the backing store for pages that have been evicted from RAM.
- Some systems treat **file-backed pages** (e.g., executable code, read-only data) as their own backing store, reading directly from the file system instead of swap.
- **Anonymous pages** (heap, stack, and other pages without a file) must be backed by swap.

Page type	Typical backing store
File-backed (code, read-only data)	Original file on disk
Anonymous (heap, stack)	Swap space
Shared library mappings (read-only)	Shared file or memory-mapped region

Mobile operating systems (iOS, Android) often **omit swap**, relying on aggressive reclamation of file-backed pages and on-the-fly compression rather than a dedicated swap partition.

🔁 Copy-On-Write (COW)

> **Copy-on-write** is a technique that lets a parent and child process share the same physical pages after a fork(). Pages are marked **read-only**; the first write by either process triggers a page fault, at which point the OS allocates a new private copy for the writer.

- **Advantages**
  - Immediate process creation without copying all pages.
  - Only pages that are actually modified become duplicated, saving memory and I/O.
- **Typical workflow**
  1. fork() creates child; both address spaces point to the same set of frames.
  2. All shared frames are marked **COW** (read-only).
  3. When either process writes to a COW page, a fault occurs.
  4. OS allocates a free frame, copies the contents, updates the page-table entry for the faulting process, and clears the COW flag.
- **Illustration** (conceptual):

  Before write: Parent P and Child C → same physical frame **F** (marked COW).

  After child writes: C gets a new frame **F′**, P continues to use **F**.

COW is employed by modern OSes such as **Linux**, **Windows XP**, and **Solaris**.

📂 Shared Libraries & Page Sharing

- System libraries are **mapped read-only** into each process that links them.
- Because the mapping is identical, all processes reference the **same physical pages**.
- This reduces overall memory consumption and speeds up process startup (no need to load duplicate copies).

Benefit table

Benefit	Explanation
Memory savings	One copy of library code serves many processes.
Cache efficiency	Frequently used library code stays in CPU caches across context switches.
Fast fork()	The child inherits the same library mappings; COW handles any later modifications (rare for read-only code).

📊 Sparse Address Spaces

> A **sparse address space** contains large unmapped "holes" that do not correspond to any physical pages.

- Useful for **dynamic linking**: a shared object can be mapped into a hole without moving existing segments.
- Holes also accommodate future growth of the heap or stack without requiring relocation of other regions.

🛠️ Hardware Support for Paging & Demand Paging

- **Translation Lookaside Buffer (TLB)**: caches recent virtual-to-physical translations; a TLB miss triggers a page-table walk, which may uncover an invalid entry → page fault.
- **Valid/Invalid bits** in page-table entries are examined by the MMU during address translation.
- **Protection bits** (read/write/execute) help detect illegal accesses before a fault is raised.

> As discussed earlier, a **TLB hit** yields effective access time ≈ memory-access time, while a miss that leads to a page fault incurs the full fault service overhead.

📈 Summary of Demand-Paging Performance Factors

Factor	Influence on EAT
Page-fault rate (p)	Linear increase; must be kept extremely low.
Disk service time	Dominates fault cost; faster disks (SSD) can reduce t_{fault}.
Replacement algorithm	Affects how often faults occur; good algorithms lower (p).
Prefetching / clustering	Can reduce faults for sequential accesses.
TLB hit ratio	Higher hit ratio reduces the chance of encountering an invalid entry early.

📚 Selected Historical References

- Early segmentation concepts: J. B. Dennis (1965).
- First paged systems: GE 645 implementation (Organick 1972).
- Inverted page-table research: Chang & Mergen (1988).
- 64-bit address-space page-table designs: Talluri et al. (1995).
- Large-page hardware support studies: Fang et al. (2001).
- PAE support in Windows: Microsoft MSDN documentation.
- IA-32 and x86-64 architecture manuals: Intel® 64 and IA-32 Architectures Software Developer's Manual.
- ARM Cortex-A9 architecture overview: ARM.com.

📂 Copy-On-Write and vfork

> **Copy-on-Write (COW)** – a memory-management optimization where parent and child processes initially share the same physical pages; a page is duplicated only when one of the processes attempts to modify it.

- After a fork(), pages **A**, **B**, and **C** are marked read-only and mapped into both processes.
- When **process 1** writes to page **C**, the hardware raises a protection fault; the OS copies page C to a new frame, updates the page table of the writer, and clears the read-only flag for that process.
- Pages that remain unchanged stay shared, saving RAM and avoiding unnecessary I/O.

Zero-fill-on-demand pages are created by the OS as all-zero frames that are allocated only when a process first accesses them; the frame is cleared before it becomes visible to the process.

vfork()

- vfork() creates a child that **shares the entire address space** of the parent without using COW.
- The parent is **suspended** until the child either calls exec() or exits.
- Because the child can modify the parent's pages, **the programmer must ensure the child does not alter the address space** (e.g., only calls exec() immediately).
- vfork() is therefore an efficient way to launch a new program from a shell, avoiding the page-copy overhead of a regular fork().

📊 Page-Replacement Fundamentals

When a page fault occurs and **no free frame** exists, the OS must **evict** a resident page to make room for the needed one.

Standard replacement steps

1. Locate the required page on secondary storage.
2. Choose a *victim* frame (via a replacement algorithm).
3. If the victim's **modify/dirty bit** is set, write the frame back to disk; otherwise, simply discard it.
4. Read the desired page into the now-free frame and update all relevant page-table entries.
5. Restart the faulting process.

> **Modify (dirty) bit** – set by hardware when a page is written; indicates that the in-memory copy differs from the copy on disk.

Using the dirty bit cuts I/O roughly in half for read-only or unmodified pages.

📈 Page-Fault Rate & Reference Strings

A **reference string** is the ordered list of page numbers referenced by a process.
Only the **page number** matters (the offset is ignored) and consecutive references to the same page cannot cause additional faults.

Example reduction

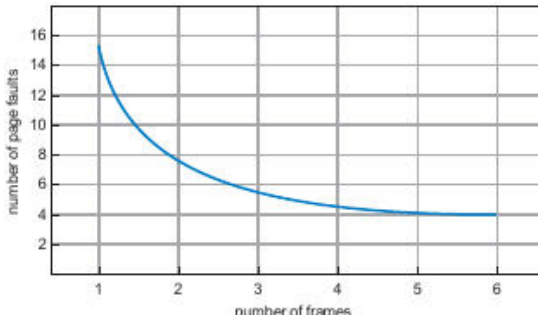
Original addresses (page size = 100 bytes):

0100, 0432, 0101, 0612, … → Page numbers: 1, 4, 1, 6, …

The number of frames influences the fault count:

- More frames ⇒ fewer faults (the fault-rate curve is monotonic decreasing for most algorithms).
- With very few frames, a fault may occur on *every* reference.

The graph below shows how the number of **edge faults** drops as the number of frames grows:



The blue line illustrates that increasing the frame count from 1 to 6 steadily reduces the number of faults.

📉 Page-Replacement Algorithms

🗃️ FIFO (First-In, First-Out)

- Pages are kept in a **queue** ordered by arrival time.
- On a fault, the page at the **head** of the queue is evicted; the newly loaded page is appended to the **tail**.

Illustrative reference string (3 frames)

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 …

Step	Frames (left→right)	Fault?
7	7 – –	✓
0	7 0 –	✓
1	7 0 1	✓
2	2 0 1	✓ (evict 7)
0	2 0 1	–
3	2 3 1	✓ (evict 0)
…	…	…

Total faults: **15**.

Belady's anomaly – for FIFO, increasing the number of frames can *increase* the fault count (e.g., 3 frames → 15 faults, 4 frames → 17 faults). This counter-intuitive behavior is specific to certain non-stack algorithms.

🏆 Optimal (OPT / MIN)

- Replace the page whose **next use is farthest in the future** (or never used again).
- Guarantees the **lowest possible fault rate** for a fixed number of frames.

Using the same reference string, OPT yields **9 faults**—the theoretical minimum.
Because it requires knowledge of future references, OPT is used only for **benchmarking**.

⏳ LRU (Least-Recently-Used)

- Approximate optimality by evicting the page that has **not been used for the longest time** (look-back).
- **Implementations**
  1. **Counters** – each page-table entry stores a timestamp; the OS updates the timestamp on every reference.
  2. **Stack** – a doubly-linked list where the most-recently-referenced page is moved to the front; the tail holds the LRU page.

LRU never exhibits Belady's anomaly and typically outperforms FIFO (12 faults on the example string).

🌀 Approximate LRU

Additional-Reference-Bits Algorithm

- Each page has an **8-bit shift register** (or configurable length).
- At regular intervals (e.g., every 100 ms), the OS shifts the register right, inserting the current **reference bit** (set by hardware on any access) into the high-order position.
- The register's integer value reflects recent usage; the page with the **smallest value** is chosen as the victim.

Second-Chance (Clock) Algorithm

- Pages are arranged in a **circular queue** with a pointer ("clock hand").
- When a victim is needed, the algorithm scans pages:
  - If the **reference bit = 0**, the page is evicted.
  - If the **reference bit = 1**, the bit is cleared and the pointer moves on, giving the page a "second chance."
- If all bits are set, the algorithm cycles once, clearing bits, and then selects the first cleared page.
- In the worst case the scan touches every frame twice, but it requires only the single reference bit per page.

Enhanced Second-Chance (Class Algorithm)

- Combines **reference** and **modify (dirty)** bits to define four classes:

Class	Ref-bit	Dirty-bit	Replacement priority
(0,0)	0	0	**Best** – not recently used, clean
(0,1)	0	1	Next – clean-to-dirty (needs write-back)
(1,0)	1	0	Later – recently used, clean
(1,1)	1	1	**Worst** – recently used, dirty

The clock hand scans for the **lowest-nonempty class**; this reduces unnecessary I/O because clean pages are preferred over dirty ones.

📊 Counting-Based Algorithms

Algorithm	Metric	Typical behavior
LFU (Least Frequently Used)	Access count per page	Evicts the page with the smallest count; suffers from "cache-pollution" if a page is heavily used early then becomes idle.
MFU (Most Frequently Used)	Access count per page	Evicts the page with the largest count, assuming it has already been used enough; rarely used in practice.
Decay-based LFU	Periodically right-shift counters (exponential decay)	Allows counts to "age," preventing stale high-frequency pages from monopolizing memory.

📄 Quick Comparison of Common Page-Replacement Policies

Policy	Knowledge required	Typical fault rate (example)	Anomaly?	Hardware support
FIFO	None	15 faults (3 frames)	**Yes** (Belady)	Simple queue
OPT	Future references	9 faults (minimum)	No	None (theoretical)
LRU	Past usage timestamps	12 faults	No	Counters or stack (software)
Approx. LRU (Clock)	Reference bit	≈ 13 faults	No	Single reference bit per page
LFU / MFU	Access counts	Varies; may be high	No	Counters (more storage)

📚 Key Takeaways (integrated with earlier sections)

- **Copy-on-Write** and **vfork** are complementary techniques that reduce the cost of process creation; they rely on the same **page-table and dirty-bit mechanisms** discussed in the demand-paging section.
- **Page replacement** completes the logical-to-physical mapping loop introduced when describing the MMU and TLB: after a page is loaded, it must eventually be evicted to keep the frame pool bounded.
- The **modify/dirty bit** ties together page-fault handling, copy-on-write, and replacement decisions, enabling the OS to avoid unnecessary writes.
- Understanding **reference strings** and **frame allocation** is essential for evaluating any replacement algorithm; the graph above visualizes the generic trade-off between frame count and fault frequency.
- Approximate LRU schemes (second-chance, clock, additional-reference-bits) are widely used in real systems because they require only the minimal hardware support (a single reference bit) already present for **page-access tracking**.

📚 Page-Buffering Algorithms

> *Page-buffering* augments a conventional page-replacement algorithm by keeping a **pool of free frames** that can be used to read a faulted page **before** the victim frame is written back to disk.

- When a page fault occurs:
  1. The needed page is read into a **free frame** from the buffer pool.
  2. The process can be resumed immediately; the victim page is written back **later**, when the buffer pool has idle capacity.
- Variants keep a **list of modified pages** (dirty pages). While the paging device is idle, a dirty page is selected, written to disk, and its modify bit is cleared. This increases the chance that a victim selected later will already be clean, avoiding an extra write-out.

Advantages

- Reduces the latency of the faulting process (no immediate I/O for the victim).
- Allows the OS to schedule victim writes during periods of low paging activity.

Implementation notes

- The VAX/VMS system uses this technique together with FIFO; a page that is still in active use is quickly retrieved from the buffer pool, eliminating unnecessary I/O.
- Some UNIX variants combine the buffer pool with the **second-chance** algorithm to mitigate the penalty of an incorrectly chosen victim.

⚙️ Applications and Page Replacement

> Certain applications (e.g., database servers, data-warehouses) perform their own **I/O buffering** and may achieve better performance when they bypass the OS's generic paging mechanisms.

- **Database engines** often keep their own cache of frequently accessed pages, reducing the number of page faults that reach the OS.
- In **sequential-read workloads** (typical of data-warehouses), the OS's LRU policy can evict pages that will be needed soon, while an **MFU** (most-frequently-used) policy would keep those pages longer and improve throughput.
- Some systems expose a **raw I/O** interface: a large, contiguous disk partition used as a "raw" array without filesystem metadata. This eliminates overhead from file-system services (locking, allocation, directory lookup) and can be advantageous for special-purpose applications that manage their own layout.

> *Note*: For most general-purpose programs, using the regular filesystem yields better overall performance because the OS can apply sophisticated caching and prefetching strategies.

📦 Allocation of Frames

> Frame allocation determines how the **fixed pool of physical frames** is divided among active processes.

Minimum Number of Frames

> A process must receive **enough frames** to hold all pages that can be simultaneously referenced by a single instruction.

- For simple architectures where an instruction references **one address**, the minimum is **2 frames** (one for the instruction, one for the operand).
- With **single-level indirect addressing**, at least **3 frames** are required (instruction, pointer page, operand page).
- Complex instructions (e.g., PDP-11 move, IBM 370 MVC) can need **6–8 frames** because the instruction itself, its operands, and possible indirect references may each straddle page boundaries.
- To avoid pathological cases, many systems impose a **limit on indirection depth** (e.g., 16 levels). The limit caps the worst-case number of frames needed for a single instruction.

Allocation Algorithms

Algorithm	Principle	Example (62 free frames)
Equal allocation	Each process receives ⌊m/n⌋ frames (m = total frames, n = processes).	Two processes → 31 frames each; leftover frames form a buffer pool.
Proportional allocation	Frames are distributed in proportion to each process's **virtual size** s_i. $a_i = \left\lceil \frac{s_i}{\sum_j s_j}; m \right\rceil$	Process A = 10 KB, Process B = 127 KB, m = 62 → A gets 4 frames, B gets 58 frames (rounded).

- Both schemes may reserve a small **buffer pool** of free frames to guarantee that a fault can be serviced without immediate eviction.
- High-priority processes can be granted extra frames beyond their proportional share, either by **priority-weighted proportional allocation** or by borrowing from lower-priority processes.

Global vs. Local Replacement

- **Global replacement**: the victim can be any frame in the system, even those belonging to other processes.
  - Allows a high-priority process to "steal" frames, improving its own performance.
  - Drawback: a process's fault rate becomes dependent on the behavior of *all* other processes, leading to unpredictable performance.
- **Local replacement**: each process may only replace frames that it already owns.
  - Guarantees that a process's fault rate reflects only its own paging pattern.
  - May reduce overall throughput because idle frames owned by a low-priority process cannot be used by a high-priority one.

> In practice, many OSes combine the two: a **global pool** supplies frames when a process's local allocation is exhausted, but otherwise each process works with its own local set.

Non-Uniform Memory Access (NUMA)

> **NUMA** systems have memory modules with **different access latencies** depending on the CPU that issues the request.

- Goal: allocate pages **as close as possible** to the CPU that will use them, minimizing remote-memory latency.
- Typical strategy:
  1. The scheduler records the **last CPU** on which a process (or thread) ran.
  2. The memory manager attempts to place new pages in the node that contains that CPU's local memory.
- **Solaris** implements this with **lgroups** (latency groups). An lgroup groups together CPUs and memory with low inter-node latency; the kernel schedules all threads of a process within a single lgroup when possible.
- When a process's threads span multiple nodes, the OS may allocate pages across several lgroups, preferring the group that hosts the majority of the threads.

🚨 Thrashing

> **Thrashing** occurs when a process does not have enough frames to keep its active pages resident, causing it to fault repeatedly on the same set of pages.

Cause of Thrashing

1. **Degree of multiprogramming** rises → more processes compete for a limited frame pool.
2. A **global replacement** algorithm selects victims without regard to process ownership; a newly started process can steal frames from an already-running process.
3. The victim process now faults on pages it just lost, stealing frames from yet another process, and so on.
4. The **ready queue empties** because processes spend most of their time waiting for the paging device.
5. **CPU utilization drops**, prompting the scheduler to increase the degree of multiprogramming even further, which exacerbates the problem.

The characteristic curve (Figure 8.18) shows CPU utilization rising with multiprogramming until a peak, after which it collapses due to thrashing.

Working-Set Model

> The **working-set** of a process at time t is the set of pages referenced during the most recent Δ references (the **working-set window**).

- A process's **working-set size** WSS_i approximates the number of frames it needs to avoid thrashing.
- If the sum of all processes' working-set sizes $D = \Sigma\ WSS_i$ exceeds the total number of frames, at least one process will thrash.

Operating-system strategy

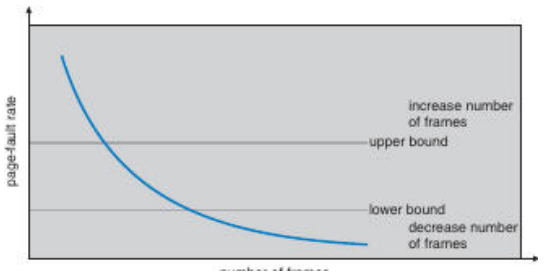
- Periodically (e.g., every 5 000 references) the OS samples each page's **reference bit**.
- Pages with the bit set are considered part of the working set; those without are candidates for eviction.
- The OS may **suspend** a process whose working set cannot be satisfied, swap its pages out, and reallocate the frames to other processes.

Page-Fault Frequency (PFF) Control

> Instead of tracking exact working-set windows, the **PFF** approach monitors the **page-fault rate** of each process.

- Define an **upper bound** and a **lower bound** on acceptable fault rates.
- If a process's fault rate exceeds the upper bound, the OS **adds a frame** to that process.
- If the fault rate falls below the lower bound, the OS **removes a frame** (if possible).

The graph below illustrates the relationship between **number of frames** and **page-fault rate**; the shaded region between the upper and lower bounds indicates the desirable operating range.



The curve shows that as more frames are allocated, the page-fault rate drops sharply. Horizontal grey lines mark the upper and lower bounds; the OS aims to keep the system within this corridor to avoid thrashing.

Practical Remarks

- Providing **sufficient physical memory** is the most effective way to prevent thrashing.

- Modern systems combine **local replacement** (to protect high-priority processes) with **working-set monitoring** to keep multiprogramming levels high without sacrificing throughput.

📂 Memory-Mapped Files

> **Memory-mapping** a file associates a region of the process's virtual address space with a file stored on disk.

- The first access to a mapped page generates a **demand-page fault**, after which the page is loaded into a physical frame just like any other page.
- Subsequent accesses to the same page behave as ordinary memory reads/writes, eliminating the need for explicit read()/write() system calls.

Benefits

- Reduces system-call overhead for large sequential reads or writes.
- Allows the OS to apply its **page-replacement** and **caching** policies uniformly to file data and program data.
- Enables **shared memory** between processes simply by mapping the same file region into each process's address space.

Typical workflow

1. Process invokes mmap() (or the OS-specific equivalent) specifying the file and desired protection flags.
2. The OS creates a **page-table entry** that points to the file's disk blocks.
3. On first reference, a page fault brings the block into a frame; the page-table entry is updated to reference the frame.
4. Writes to a mapped page set the **dirty bit**; the page will be written back to the file on eviction or when msync()/munmap() is called.

Memory-mapped I/O is especially advantageous for **large data sets** (e.g., multimedia processing, scientific simulations) where the cost of repeated read() calls would dominate performance.

📚 Working Set Dynamics & Locality

> **Working set** – the set of pages a process actively uses during a given interval of execution.

- When a process moves to a **new working set**, the page-fault rate initially spikes, then falls as the new pages are loaded.
- The interval between the start of one fault-rate peak and the next marks the transition from one working set to another.
- Frequent switches between large working sets can cause **thrashing**, where the system spends most of its time handling page faults.

📁 Memory-Mapped Files

> **Memory-mapped file** – a file whose contents are mapped into a process's virtual address space, allowing the program to access file data through ordinary memory reads and writes.

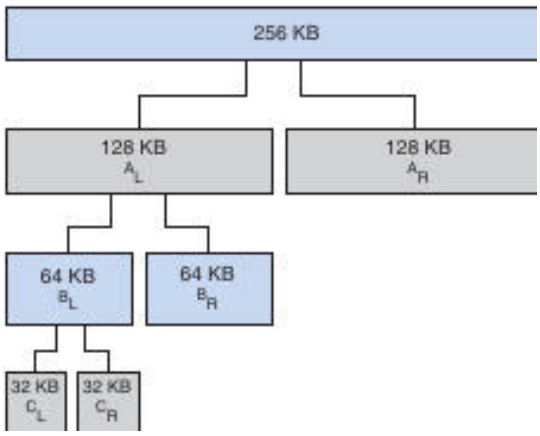
- Reads and writes become simple memory operations; no explicit read()/write() system calls are needed.
- **Writes are not immediately synchronous** – the OS may defer updating the physical file until the page is flushed (e.g., by the page-replacement daemon).
- When a mapped page is evicted, any modifications are written back to the file and the page is removed from the process's address space.

Key properties

Property	Effect
Transparency	File I/O is performed via normal memory accesses.
Lazy updating	Modified pages are written back only when the OS decides (periodic flushing or on unmap).
Sharing	Multiple processes can map the same file, seeing a single physical copy of each page.

Copy-on-Write (COW)	Read-only mappings can be shared; a write triggers a private copy for the writer.

Illustration of shared mapping



The diagram (Figure 8.22) shows two processes whose virtual address spaces map the same file. Both virtual pages point to the same physical page, so a write by one process is immediately visible to the other.

🤝 Shared Memory via Memory-Mapped Files

> **Shared memory** – a region of memory that multiple processes can map, enabling direct communication without extra system calls.

- Implemented by creating a **memory-mapped file** that all participating processes open and map into their address spaces.
- The file can be a regular on-disk file or a special **POSIX shared-memory object** (shm_open).
- Because the underlying pages are the same physical pages, writes by any process are instantly visible to all others.

Typical workflow

1. **Create** a shared-memory object (or ordinary file).
2. **Map** it into the address space of each process (mmap).
3. **Read/write** the shared region as ordinary memory.
4. **Unmap** (munmap) and **close** the object when done.

🪟 Windows API Memory-Mapped Files

> **CreateFileMapping** – Windows API call that creates a kernel object representing a file mapping; the object can be named for sharing.

> **MapViewOfFile** – establishes a view of the mapping in the calling process's virtual address space and returns a pointer to the mapped region.

Producer-consumer example (simplified)

1. **Producer**
   - CreateFile opens the backing file.
   - CreateFileMapping creates a named mapping ("SharedObject").
   - MapViewOfFile maps the entire file; the returned pointer is used to write a message.
2. **Consumer**
   - OpenFileMapping opens the existing named mapping.

- MapViewOfFile obtains a pointer to the same physical pages.
- Reads the message from the shared region.

Both processes later call UnmapViewOfFile and close their handles.

Note: The mapping is **demand-paged**; pages are brought into physical memory only when accessed, not at mapping creation time.

🖥️ Memory-Mapped I/O

> **Memory-mapped I/O** – a technique where device registers are mapped into the processor's address space, allowing the CPU to read/write them with ordinary load/store instructions.

- Ideal for **fast-response devices** (e.g., video controllers, serial ports).
- The CPU accesses a specific address range; the hardware translates the access to a device register read or write.
- **Polling** (busy-waiting) and **interrupt-driven** I/O are both possible; the difference lies in whether the CPU repeatedly checks a control bit or is notified by an interrupt.

Advantages

- Simpler code – no special I/O instructions.
- Can leverage the same caching and protection mechanisms used for regular memory (though device pages are typically marked non-cacheable).

🧩 Kernel Memory Allocation

🧱 Buddy System

> **Buddy system** – a power-of-2 allocator that splits a fixed-size memory pool into pairs of "buddies" to satisfy allocation requests.

- Memory is divided recursively into halves (buddies) until a block of size ≥ request is found.
- When a block is freed, its buddy is checked; if both are free they are **coalesced** back into a larger block.

Advantages

- Fast allocation and deallocation (simple bit-maps).
- Easy coalescing of adjacent free blocks.

Drawbacks

- Internal fragmentation: allocation size is rounded up to the next power of 2, potentially wasting up to ~50 % of a block.

Example – allocating 21 KB from a 256 KB pool results in successive splits to 32 KB blocks; the 21 KB request occupies one 32 KB buddy, leaving the remainder available for future splits.

🧱 Slab Allocation

> **Slab allocator** – a cache-based allocator that maintains per-object-type slabs, each consisting of one or more contiguous pages holding multiple objects of the same size.

- **Cache**: one per kernel data structure (e.g., semaphores, task structs).
- **Slab**: a set of pages belonging to a cache.
- **Object**: an individual instance stored inside a slab.

Slab states

State	Description
Full	All objects in the slab are in use.
Partial	Some objects are free, some allocated.

Empty	All objects are free; the slab can be reclaimed.

Benefits

- No fragmentation – objects are allocated exactly to their size.
- Fast allocation: a free object is taken from a partial slab; if none exist, a new slab is created.
- Immediate reuse: when an object is freed, it returns to its cache, ready for the next request.

Variations

- **SLAB** (original Solaris implementation).
- **SLUB** (Linux default since 2.6.24) – stores metadata in the page structures and removes per-CPU queues, improving scalability on many-core systems.
- **SLOB** – simple allocator for memory-constrained embedded systems, using three size classes (small, medium, large) with a first-fit policy.

📈 Other Considerations

🔧 Prepaging

> **Prepaging** – proactively loading a set of pages into memory before they are actually referenced, aiming to reduce the initial burst of page faults.

- Often used when a process is about to resume after being swapped out or when a small file is opened.
- The trade-off: if a large fraction of the prefetched pages are never used, prepaging adds unnecessary I/O.
- Effectiveness can be expressed as s * (1-α) where s is the number of prefetched pages and α is the fraction actually used.

📏 Page Size Selection

> **Page size** – the granularity of memory paging, always a power of 2 (commonly 4 KB to 4 MB).

Factors influencing the choice

Factor	Impact of Larger Pages	Impact of Smaller Pages
Page-table size	Fewer entries → smaller tables, less memory overhead.	More entries → larger tables, higher overhead per process.
Internal fragmentation	Higher – the final partially used page can waste many bytes.	Lower – less unused space per process.
I/O transfer time	Fewer I/O operations (each transfers more data).	More I/O operations, but each transfer is quicker; overall effect limited because seek/latency dominate.
TLB coverage	Fewer TLB entries needed to cover the same address space.	More TLB entries required; may increase miss rate.

Typical design balances **memory-utilization efficiency** (favoring small pages) against **overhead** (favoring larger pages). Modern architectures often support multiple page sizes (e.g., 4 KB “base” pages and 2 MB “huge” pages) to allow the OS to choose per-allocation.

📚 Operating Systems Overview

Brief Overview

This note covers **Operating Systems** and was created from the PDF (273 pages). It covers **distributed file systems**, pipes and IPC, process and thread scheduling, memory management, and synchronization primitives.

Key Points

- Distributed file system request handling and DFS daemon operations.
- Interprocess communication: ordinary and named pipes, message passing, sockets.
- Process and thread scheduling strategies: FCFS, RR, SJF, RMS, EDF, and multilevel feedback.
- Memory management fundamentals: paging, segmentation, virtual memory, swap and COW.

📡 Distributed File System (DFS) Operations

> **Definition:** A DFS request message contains the file operation to be performed on the server's disk (e.g., read, write, rename, delete) and any data needed for the call.
> The DFS daemon on the server executes the operation on behalf of the client and returns a response message with the operation's status and any resulting data.

- **Typical workflow**
  1. Client sends a request message describing the file-related system call.
  2. DFS daemon performs the corresponding disk operation.
  3. Server replies with a status code and, if applicable, the data requested (e.g., file contents).

🔀 Pipes Overview

> **Definition:** A pipe is an inter-process communication (IPC) mechanism that provides a unidirectional or bidirectional conduit for data flow between processes.

Design considerations

Question	Possible Answers
Directionality	Unidirectional vs. bidirectional
Duplex mode	Half-duplex (one-way at a time) or full-duplex
Process relationship	Requires parent–child relationship?
Network scope	Same-machine only or network-transparent?

📂 Ordinary Pipes

UNIX Implementation

> **Definition:** An ordinary (anonymous) pipe is a special file type created by pipe(int fd[]); it yields two file descriptors: fd[0] (read end) and fd[1] (write end).

- **Typical usage pattern**
  1. Parent creates the pipe.
  2. Parent forks a child.
  3. Each process closes the end it does not use.
  4. Parent writes; child reads (or vice-versa).
  5. Closing the write end allows the reader to detect end-of-file (read() returns 0).

```
/* UNIX anonymous pipe example */
#include
#include
#include
#define BUFFER_SIZE 25
#define READ_END 0
#define WRITE_END 1
```

```
int main(void) {
    char write_msg[BUFFER_SIZE] = "Greetings";
    char read_msg[BUFFER_SIZE];
    int fd[2];
    pid_t pid;

    if (pipe(fd) == -1) {
        perror("Pipe creation failed");
        return 1;
    }

    pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }

    if (pid > 0) {               /* Parent process */
        close(fd[READ_END]);                    // close unused read end
        write(fd[WRITE_END], write_msg, strlen(write_msg));
        close(fd[WRITE_END]);                   // signal EOF
    } else {                     /* Child process */
        close(fd[WRITE_END]);                   // close unused write end
        read(fd[READ_END], read_msg, BUFFER_SIZE);
        printf("Child read: %s\n", read_msg);
        close(fd[READ_END]);
    }
    return 0;
}
```

Windows Implementation (Anonymous Pipe)

> **Definition:** In Windows, an anonymous pipe is created with CreatePipe, producing separate read and write handles. The pipe is half-duplex and typically used between a parent and a child process.

```
/* Windows anonymous pipe example */
#include
#include
#define BUFFER_SIZE 25

int main(void) {
    HANDLE ReadHandle, WriteHandle;
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    char message[BUFFER_SIZE] = "Greetings";
    DWORD written;

    SECURITY_ATTRIBUTES sa = { sizeof(SECURITY_ATTRIBUTES), NULL, TRUE };
    ZeroMemory(&pi, sizeof(pi));
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);

    if (!CreatePipe(&ReadHandle, &WriteHandle, &sa, 0)) {
        fprintf(stderr, "CreatePipe failed\n");
        return 1;
    }

    si.hStdOutput = WriteHandle;
    si.hStdInput  = ReadHandle;
    si.dwFlags    = STARTF_USESTDHANDLES;

    // Inherit handles for the child
    if (!CreateProcess(NULL, "child.exe", NULL, NULL, TRUE,
                       0, NULL, NULL, &si, &pi)) {
```

```
        fprintf(stderr, "CreateProcess failed\n");
        return 1;
    }

    CloseHandle(ReadHandle);                    // parent does not read
    WriteFile(WriteHandle, message, BUFFER_SIZE, &written, NULL);
    CloseHandle(WriteHandle);                   // signal EOF
    WaitForSingleObject(pi.hProcess, INFINITE);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
    return 0;
}
```

- **Key points**
  - The child inherits the pipe handles via STARTUPINFO.
  - The parent closes the read handle; the child closes the write handle.
  - Half-duplex: only one direction of data flow at a time.

📡 Named Pipes (FIFOs)

UNIX FIFO

> **Definition:** A FIFO (named pipe) appears as a file in the filesystem, created with mkfifo(). It persists beyond the lifetimes of the processes that use it.

- **Characteristics**
  - Can be accessed by unrelated processes.
  - Supports bidirectional communication, but only half-duplex per FIFO; two FIFOs are needed for full duplex.
  - Communication limited to the same machine (network communication requires sockets).

Windows Named Pipe

> **Definition:** Windows named pipes are created with CreateNamedPipe() and support full-duplex, byte- or message-oriented transfer. Processes may reside on the same or different machines.

```
/* Windows named pipe (server) example */
#include
#include
#define BUFFER_SIZE 25

int main(void) {
    HANDLE ReadHandle, WriteHandle;
    SECURITY_ATTRIBUTES sa = { sizeof(SECURITY_ATTRIBUTES), NULL, TRUE };
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    DWORD written;
    char message[BUFFER_SIZE] = "Error writing to pipe.";

    // Create the pipe
    if (!CreatePipe(&ReadHandle, &WriteHandle, &sa, 0)) {
        fprintf(stderr, "CreatePipe failed\n");
        return 1;
    }

    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    si.hStdInput  = ReadHandle;
    si.hStdOutput = WriteHandle;
    si.dwFlags    = STARTF_USESTDHANDLES;

    // Launch child that inherits the handles
```

```
    if (!CreateProcess(NULL, "child.exe", NULL, NULL, TRUE,
                       0, NULL, NULL, &si, &pi)) {
        fprintf(stderr, "CreateProcess failed\n");
        return 1;
    }

    CloseHandle(ReadHandle);                    // parent writes only
    WriteFile(WriteHandle, message, BUFFER_SIZE, &written, NULL);
    CloseHandle(WriteHandle);                   // signal EOF
    WaitForSingleObject(pi.hProcess, INFINITE);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
    return 0;
}
```

- **Key differences vs. UNIX FIFO**
  - Full-duplex communication is native.
  - Can operate across networked machines.
  - Supports both byte-oriented and message-oriented data.

🔧 Pipes in Practice (Command-Line)

- **UNIX:** ls | more
  - ls (producer) writes directory listing to the pipe; more (consumer) reads from the pipe and paginates the output.
- **Windows:** dir | more
  - Same concept with dir as the producer and more as the consumer.

The pipe character | connects the standard output of the left command to the standard input of the right command.

🛠️ Process States & Scheduling

> **Definition:** A process is an executing program instance; its state reflects its current activity.

- **Possible states**
  - **new** – just created
  - **ready** – waiting for CPU allocation
  - **running** – currently executing
  - **waiting** – blocked for I/O or other event
  - **terminated** – execution finished
- **Scheduling queues**

Queue type	Contains
Ready queue	Processes ready to run on the CPU
I/O request queue	Processes awaiting I/O completion
Waiting queues	Processes blocked for specific events

- **Scheduler categories**
  - **Long-term (job) scheduler:** selects which processes enter the ready queue (controls degree of multiprogramming).
  - **Short-term (CPU) scheduler:** chooses the next process from the ready queue for execution.

📡 Interprocess Communication (IPC) Mechanisms

Mechanism	Typical use	Responsibility

Shared memory	Large data exchange, low latency	Application programmers allocate and manage the shared region; OS provides mapping facilities
Message passing	Small control messages, synchronization	OS may provide the messaging API (e.g., sockets, pipes)
Sockets	Networked client-server communication	OS handles endpoint creation, routing, and data transport
Remote Procedure Calls (RPCs)	Distributed procedure invocation	OS or middleware implements request/response handling
Pipes	Parent-child or related processes (ordinary) or unrelated local processes (named)	OS creates and manages the pipe objects

---

🧩 Selected Programming Problems (Key Concepts)

Collatz Conjecture with fork()

- Parent creates a child that computes the Collatz sequence for a given integer (passed on the command line).
- Child prints the sequence; parent waits (wait()) for child termination before exiting.
- Demonstrates separate address spaces and synchronization via wait().

PID Manager Design

- Maintain a bitmap for PIDs in the range [MIN_PID, MAX_PID] (300-5000).
- Operations:
  - int allocate_pid(void); → returns an available PID or -1.
  - void release_pid(int pid); → marks the PID as free.
  - void init_pid_manager(void); → allocates and zeroes the bitmap.

File-Copy Program Using Pipes

- Parent opens the source file, writes its contents into an anonymous pipe, then forks a child.
- Child reads from the pipe and writes to the destination file.
- Illustrates producer-consumer pattern with a single pipe.

Echo Server (Java)

- Accepts a client connection with ServerSocket.accept().
- Reads bytes into a buffer; writes the same bytes back to the client.
- Loop terminates when InputStream.read() returns -1 (client closed).
- Demonstrates socket-based IPC and handling of binary data.

🖥️ Simple Shell Design

> **Definition:** A *shell* is a command-line interpreter that reads user input, creates processes to execute commands, and optionally manages background execution.

Main Loop & Process Creation

1. Print prompt osh> and flush stdout.
2. Read a line of input from the user.
3. **Parse** the line into separate tokens (command + arguments) and store them in an args[] array.
4. Call fork() to create a child process.
5. In the **child**: invoke execvp(args[0], args) to replace the child's image with the requested program.
6. In the **parent**:
   - If the command **does not** end with &, call wait() to block until the child terminates.
   - If the command **does** end with &, skip wait() so parent and child run **concurrently**.

```
/* Simplified shell outline (C) */
#include
#include
#define MAX_LINE 80

int main(void) {
    char *args[MAX_LINE/2 + 1];
    int should_run = 1;

    while (should_run) {
        printf("osh> "); fflush(stdout);
        /* (1) read input, (2) tokenize into args[] */
        /* (3) fork child */
        /* (4) child: execvp(args[0], args) */
        /* (5) parent: wait() unless '&' present */
    }
    return 0;
}
```

Handling the Ampersand (&)

- The presence of & tells the shell **not** to wait for the child.
- The parent immediately returns to the prompt, allowing the user to start another command while the first runs in the background.

📜 Shell History Feature

> **Definition:** The *history* buffer records the most recent commands entered, enabling recall and re-execution.

- **Capacity:** Stores the **10 most recent** commands, but numbering continues beyond 10 (e.g., 26–35 for the latest ten).
- **Listing:** Typing history prints the numbered list, most recent first.

Input	Action
!!	Re-execute the **most recent** command.
!N	Re-execute the command with number **N** in the history list.
history	Display the stored commands with their numbers.

- **Echoing:** When a command is invoked via !! or !N, the shell first prints the command line before execution.
- **Error handling:**
  - If no command exists for !!, output: No commands in history.
  - If the requested number does not exist, output: No such history.

The history buffer must be updated **after** a command finishes (including those run in background).

🐧 Linux Kernel Module: Listing Tasks

> **Definition:** A kernel module can traverse the kernel's task structures to report information about all running processes.

Part I – Linear Iteration

- Include to access struct task_struct.
- Use the macro for_each_process(task) to walk the global task list.

```
struct task_struct *task;
for_each_process(task) {
    /* task->comm = name, task->pid = PID, task->state = state */
    printk(KERN_INFO "Task: %s  PID: %d  State: %ld\n",
```

```
            task->comm, task->pid, task->state);
}
```

- The printk output appears in the kernel log (dmesg).
- Verify correctness by comparing the output with ps -el; minor differences are expected because kernel threads (e.g., idle threads) may not appear in ps.

Part II – Depth-First Search (DFS) of the Process Tree

- Each task_struct contains list_head children and list_head sibling.
- Use list_for_each and list_entry to traverse children recursively.

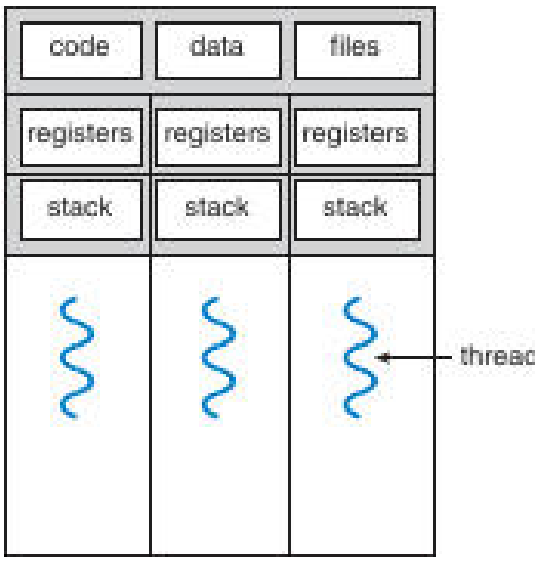
```
void dfs_task(struct task_struct *task) {
    struct list_head *list;
    struct task_struct *child;

    printk(KERN_INFO "Task: %s  PID: %d  State: %ld\n",
           task->comm, task->pid, task->state);

    list_for_each(list, &task->children) {
        child = list_entry(list, struct task_struct, sibling);
        dfs_task(child);   /* recursive depth-first walk */
    }
}
```

- Invoke dfs_task(&init_task) from the module's entry function.
- The DFS order typically differs from the flat list produced by for_each_process, showing the hierarchical parent-child relationships.

🧵 Thread Fundamentals

> **Definition:** A *thread* is the smallest unit of CPU utilization, consisting of its own register set and stack, while sharing the process's code, data, and open resources.

Memory Layout of a Multithreaded Program



*The diagram shows three threads (T1, T2, T3). The **code**, **data**, and **files** regions are shared among all threads, whereas each thread has its own **registers** and **stack**.*

Benefits of Multithreading

Benefit	Explanation
Responsiveness	UI remains interactive while a long-running operation executes in a separate thread.
Resource Sharing	Threads automatically share memory and open file descriptors, avoiding explicit IPC.
Economy	Creating a thread is cheaper than creating a full process; context switches are faster.
Scalability	On multicore CPUs, threads can run in parallel, improving throughput.

Multicore Programming

- **Concurrency vs. Parallelism** – *Concurrency* is the interleaved progress of multiple tasks; *parallelism* requires multiple cores executing tasks simultaneously.
- **Amdahl's Law** quantifies speedup:

  $\text{Speedup} \le \frac{1}{(1 - S) + \frac{S}{N}}$

  where S is the serial fraction and N the number of cores.
  - Example: 25 % serial, 75 % parallel on 4 cores → speedup ≈ 2.28×.
  - As $N \to \infty$, speedup approaches $1/(1-S)$.

Types of Parallelism

Type	Focus
Data parallelism	Same operation applied to different subsets of a data set (e.g., splitting an array between threads).
Task parallelism	Different operations executed concurrently on the same or different data (e.g., one thread sorts, another computes statistics).

Thread Models

Many-to-One

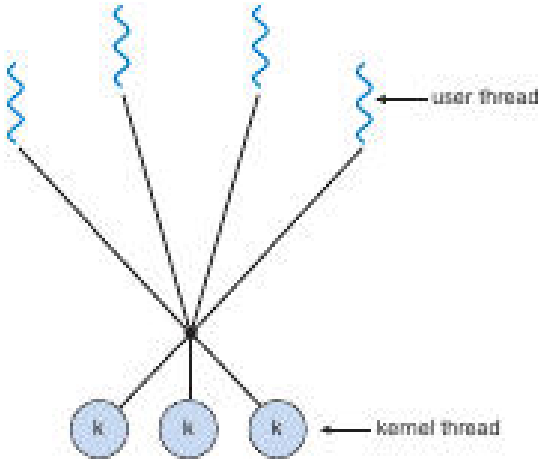
- Multiple **user-level** threads are mapped onto **one** kernel thread.
- Scheduling is done in user space; a blocking system call stalls **all** threads.
- Historically used by "green threads" (early Java, Solaris).

One-to-One

- Each user thread has a dedicated kernel thread.
- Allows true parallelism on multicore systems; a blocking call only blocks its own thread.
- Implemented by Linux and Windows.

Many-to-Many

- Many user threads are multiplexed over a **pool** of kernel threads.
- Balances flexibility with scalability.



The diagram illustrates multiple user threads (top wavy lines) connected to several kernel threads (bottom circles) via a mapping layer, representing the many-to-many relationship.

Two-Level Many-to-Many (Hybrid)

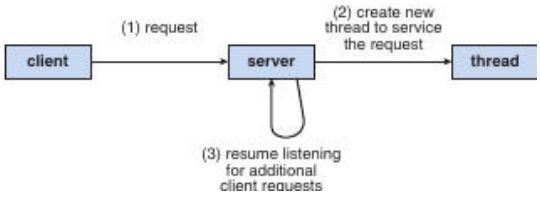
- User threads are bound to a subset of kernel threads; excess user threads are scheduled onto available kernel threads as needed.
- Used in older Solaris versions; newer systems adopt the one-to-one model for simplicity and better SMP support.

Thread Libraries

- Provide an API (e.g., pthread_create, pthread_join on POSIX) that abstracts kernel thread creation and management.
- The library handles stack allocation, attribute setting, and synchronization primitives (mutexes, condition variables).

📡 Server-Thread Interaction Example

> **Definition:** In a multithreaded server, each client request can be serviced by a dedicated thread, allowing the server to continue listening for new connections.



The flowchart shows a client sending a request to the server; the server spawns a new thread to handle the request while the main thread returns to listening for additional client connections.

🧵 Thread Libraries Overview

> **Definition:** A *thread library* provides the API and supporting code that applications use to create, manage, and synchronize threads. It may reside entirely in **user space** (no kernel support) or be **kernel-level**, where part of the library runs inside the operating system.

User-Level vs. Kernel-Level Libraries

Aspect	User-Level Library	Kernel-Level Library

Location of code & data	Entirely in user space; library functions are normal function calls.	Partially or wholly in kernel space; some calls invoke system calls.
Thread creation cost	Low (no system call).	Higher (requires a system call).
Scheduling	Performed by the library; the kernel sees only one process.	Handled by the OS scheduler; each thread is a kernel-level entity.
Blocking I/O	A blocking system call can stall **all** threads in the process.	Only the thread that issued the call is blocked.
Portability	May need separate implementations per OS.	Relies on OS-provided thread support.

📚 Major Thread Libraries (Three in Use)

Library	Typical Platform	Implementation Model
POSIX Pthreads	Unix-like (Linux, macOS, Solaris, etc.)	Can be user-level or kernel-level (implementation-defined).
Windows Threads	Windows NT family	Kernel-level (one-to-one with kernel threads).
Java Thread API	JVM on any host OS (Windows, Linux, macOS, Android)	Usually maps to the host's native thread library (Pthreads on Unix, Windows API on Windows).

Note: Java has **no global variables**; shared data must be passed via object references.

🔄 Asynchronous vs. Synchronous Threading

Strategy	Characteristics	Typical Use
Asynchronous	Parent creates a child thread and **continues** execution; threads run concurrently without the parent waiting.	Server request handling (Figure 4.2).
Synchronous (fork-join)	Parent creates children **and must wait** for all to finish before proceeding.	Parallel data-parallel work where results must be combined.

> **Example summation:** The function computes $ \displaystyle \sum_{i=0}^{N} i $.
> For $N = 5$, the result is $0+1+2+3+4+5 = 15$.

🛠️ Pthreads (POSIX) Example

```
/* Pthreads summation program (C) */
#include
#include
#include

int sum;   // shared global variable

void *runner(void *param) {
    int i, upper = atoi((char *)param);
    sum = 0;
    for (i = 1; i <= upper; i++)
        sum += i;
    pthread_exit(0);
}
```

```
int main(int argc, char *argv[]) {
    pthread_t tid;
    pthread_attr_t attr;

    if (argc != 2) {
        fprintf(stderr, "usage: %s \n", argv[0]);
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >= 0\n", atoi(argv[1]));
        return -1;
    }

    pthread_attr_init(&attr);                // default attributes
    pthread_create(&tid, &attr, runner, argv[1]);
    pthread_join(tid, NULL);                 // wait for child
    printf("sum = %d\n", sum);
    return 0;
}
```

Key points

- pthread_create starts runner in a new thread.
- Global sum is shared between parent and child.
- pthread_join implements the **fork-join** (synchronous) pattern.

Joining Multiple Threads

```
#define NUM_THREADS 10
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
    pthread_create(&workers[i], NULL, worker_func, NULL);

for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```

Windows Threads Example

```
/* Windows thread summation program (C) */
#include
#include

DWORD Sum = 0;   // shared global variable

DWORD WINAPI Summation(LPVOID Param) {
    DWORD Upper = *(DWORD *)Param;
    for (DWORD i = 0; i <= Upper; i++)
        Sum += i;
    return 0;
}

int main(int argc, char *argv[]) {
    HANDLE ThreadHandle;
    DWORD ThreadId, Param;

    if (argc != 2) {
        fprintf(stderr, "An integer parameter is required\n");
        return -1;
    }
```

```
    Param = (DWORD)atoi(argv[1]);
    if ((int)Param < 0) {
        fprintf(stderr, "Parameter must be >= 0\n");
        return -1;
    }

    ThreadHandle = CreateThread(
        NULL,               // default security attributes
        0,                  // default stack size
        Summation,          // thread function
        &Param,             // argument to thread function
        0,                  // default creation flags
        &ThreadId);

    if (ThreadHandle != NULL) {
        WaitForSingleObject(ThreadHandle, INFINITE);  // synchronous wait
        CloseHandle(ThreadHandle);
        printf("sum = %lu\n", Sum);
    }
    return 0;
}
```

Key points

- CreateThread creates a kernel-level thread.
- WaitForSingleObject corresponds to pthread_join.
- WaitForMultipleObjects can wait on many handles simultaneously.

☕ Java Threads Example

```
// Sum holder object
class Sum {
    private int sum;
    public int getSum() { return sum; }
    public void setSum(int s) { this.sum = s; }
}

// Runnable that computes the summation
class Summation implements Runnable {
    private final int upper;
    private final Sum sumObj;

    Summation(int upper, Sum sumObj) {
        this.upper = upper;
        this.sumObj = sumObj;
    }

    @Override
    public void run() {
        int s = 0;
        for (int i = 0; i <= upper; i++)
            s += i;
        sumObj.setSum(s);
    }
}

// Driver program
public class Driver {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.err.println("Usage: Summation ");
            return;
        }
        int upper = Integer.parseInt(args[0]);
```

```
        if (upper < 0) {
            System.err.println(upper + " must be >= 0.");
            return;
        }

        Sum sumObj = new Sum();
        Thread thrd = new Thread(new Summation(upper, sumObj));
        thrd.start();
        try {
            thrd.join();                    // wait for child thread
            System.out.println("The sum of " + upper + " is " + sumObj.getSum());
        } catch (InterruptedException ignored) { }
    }
}
```

Key points

- Java threads are created by new Thread(runnable) and started with start().
- Shared data is passed via object references (Sum).
- join() provides the same **fork-join** synchronization as pthread_join and WaitForSingleObject.

📦 Thread Pools

> **Definition:** A *thread pool* maintains a set of pre-created threads that wait for work. When a task arrives, a free thread is assigned; after completion the thread returns to the pool.

Benefits

1. **Reduced creation overhead** – reusing existing threads is faster than spawning new ones.
2. **Bounded concurrency** – limits the maximum number of active threads, preventing resource exhaustion.
3. **Separation of concerns** – task logic is decoupled from thread-management logic, enabling scheduling policies (delayed, periodic, priority-based).

Windows Thread-Pool API (example)

```
DWORD WINAPI PoolFunction(LPVOID Param) {
    /* work performed by a pooled thread */
    return 0;
}

/* Submit work to the pool */
QueueUserWorkItem(PoolFunction, NULL, 0);
```

- QueueUserWorkItem takes a pointer to the function, a parameter, and flags.
- For waiting on multiple pool tasks, WaitForMultipleObjects can be used.

Java java.util.concurrent (implicit)

- ExecutorService creates a pool; submit(Runnable) enqueues work.
- shutdown() and awaitTermination() replace explicit joins.

Dynamic Adjustment

- Modern pools (e.g., Apple's Grand Central Dispatch) grow or shrink the number of underlying OS threads based on current load and system capacity.

🧩 OpenMP

> **Definition:** *OpenMP* is a set of compiler directives, library routines, and environment variables for shared-memory parallelism in C/C++/Fortran.

Simple Parallel Region

```
#include
#include

int main(int argc, char *argv[]) {
    #pragma omp parallel
    {
        printf("I am a parallel thread %d\n", omp_get_thread_num());
    }
    return 0;
}
```

- The #pragma omp parallel directive tells the runtime to create a team of threads (usually one per CPU core).

Parallel Loop Example

```
#pragma omp parallel for
for (int i = 0; i < N; i++)
    c[i] = a[i] + b[i];
```

- The loop work is divided among the threads in the team.
- OpenMP also allows explicit control of the number of threads (omp_set_num_threads) and data-sharing attributes (private vs. shared).

🌀 Grand Central Dispatch (GCD) – macOS / iOS

> **Definition:** *GCD* combines language extensions (blocks), an API, and a runtime library that schedules work on a pool of POSIX threads.

Blocks Syntax (C/C++)

```
dispatch_queue_t queue = dispatch_get_global_queue(DISPATCH_QUEUE_PRIORITY_DEFAULT, 0);
dispatch_async(queue, ^{ printf("I am a block running on a GCD thread\n"); });
```

- **Blocks** (^{ … }) are self-contained units of work.
- **Serial queues** execute blocks FIFO, one at a time.
- **Concurrent queues** may run multiple blocks simultaneously.
- System-wide concurrent queues have three priority levels: **low**, **default**, **high**.

Internally, GCD manages a thread pool of POSIX threads, automatically scaling to match system demand.

📚 Other Implicit Threading Approaches

Approach	Language / Library	Key Feature
Threading Building Blocks (TBB)	C++	High-level algorithms and containers that abstract away explicit thread management.
Microsoft Parallel Patterns Library (PPL)	C++/C#	parallel_for, task_group, and async abstractions built on the Windows thread

		pool.
java.util.concurrent	Java	Executors, Future, CountDownLatch, etc., for implicit creation and coordination.

These frameworks aim to simplify concurrency while still exploiting multicore hardware.

⚙️ Fork & Exec in Multithreaded Programs

> **Definition:** fork() creates a new process; exec() replaces the current process image with a new program.

- **Two fork() variants:**
  1. **Full duplicate** – copies *all* threads into the child (rare; used when the child continues as a multithreaded program).
  2. **Thread-only duplicate** – copies only the calling thread (common in POSIX).
- **exec() semantics:** Regardless of how many threads existed before the call, exec() replaces the entire process image, discarding all threads.
- **Guideline:**
  - Use the *thread-only* fork() when the child will immediately exec() another program (no need to copy other threads).
  - Use the *full* fork() only when the child must retain the full thread set.

🚦 Signal Handling in Multithreaded Programs

> **Definition:** A *signal* is an asynchronous notification sent to a process (or thread) to inform it of an event such as SIGINT, SIGSEGV, or a timer expiration.

Delivery Options

Option	Description
Deliver to the thread that caused the signal	Typical for synchronous signals (e.g., division by zero).
Deliver to any thread that has not blocked the signal	Default for many asynchronous signals (e.g., SIGINT).
Deliver to a specific set of threads	Threads can mask (block) particular signals, limiting delivery.
Assign a single "signal-handling" thread	Using pthread_sigmask to block signals in all other threads and sigwait in the designated thread.

Synchronous vs. Asynchronous Signals

- **Synchronous signals** (generated by the faulting thread) are delivered directly to that thread.
- **Asynchronous signals** (generated by external events) may be delivered to any thread that does not block them, leading to nondeterministic handling unless explicitly managed.

Proper signal handling in multithreaded programs often involves:

1. **Blocking** the signal in all threads except a dedicated handler thread.
2. Using sigwait() or sigwaitinfo() in the handler thread to synchronously receive the signal.
3. Ensuring that shared data accessed from signal handlers is protected (e.g., via volatile sig_atomic_t or lock-free structures).

⚡ Signals in Multithreaded Environments

> **Definition:** A signal is an asynchronous notification sent to a process (or thread) to indicate an event such as termination (Ctrl-C) or a user-defined condition.

- The classic UNIX interface uses

```
kill(pid_t pid, int signal);
```

where **pid** identifies the target *process* and **signal** specifies the event.

- Modern UNIX variants let a thread *block* or *accept* specific signals.
  - If a signal is **blocked** in a thread, that thread will not receive it.
  - The kernel delivers an asynchronous signal to the **first** thread that does **not** have it blocked.
- POSIX adds a thread-specific API:

```
pthread_kill(pthread_t tid, int signal);
```

which directs the signal to a particular *thread* (tid).

- **Windows** does not have POSIX-style signals. Instead it provides **Asynchronous Procedure Calls (APCs)**:
  - An APC is queued to a specific thread and executes a user-supplied routine when the thread enters an alertable state.
  - APCs play the same role as UNIX signals but are inherently *thread-targeted* rather than process-wide.

❌ Thread Cancellation

> **Definition:** Thread cancellation terminates a thread before it has completed its normal execution.

Typical use-case: multiple worker threads search a database; once one finds the result, the others are cancelled.

Cancellation Scenarios

1. **Asynchronous cancellation** – the target thread is terminated immediately.
2. **Deferred cancellation** – the target thread periodically checks for a cancellation request and exits at a *cancellation point*.

Challenges

- Resources (memory, file descriptors) may be left unreleased if cancellation occurs while a thread holds them.
- Asynchronous cancellation can abort a thread while it is updating shared data, leading to inconsistency.
- The system usually reclaims the thread's kernel resources, but **application-level** resources must be cleaned up manually.

Pthreads Cancellation API

Attribute	Meaning
State	PTHREAD_CANCEL_ENABLE / PTHREAD_CANCEL_DISABLE
Type	PTHREAD_CANCEL_DEFERRED / PTHREAD_CANCEL_ASYNCHR

- The default is **deferred** cancellation.
- A thread can change its state with pthread_setcancelstate() and its type with pthread_setcanceltype().
- Cancellation is *requested* with pthread_cancel(tid). The request remains pending until the target reaches a cancellation point (e.g., pthread_testcancel(), read(), pthread_join()).

Example (deferred cancellation)

```
while (1) {
    /* perform work */
    pthread_testcancel();   /* cancellation point */
}
```

- When a cancellation request is pending, the runtime invokes any registered *cleanup handlers* so that the thread can release its own resources before termination.

> **Note:** On Linux, Pthreads cancellation is implemented using signals (see Section 4.6.2).

📦 Thread-Local Storage (TLS)

> **Definition:** TLS is data that is **unique to each thread** but persists across multiple function calls within that thread.

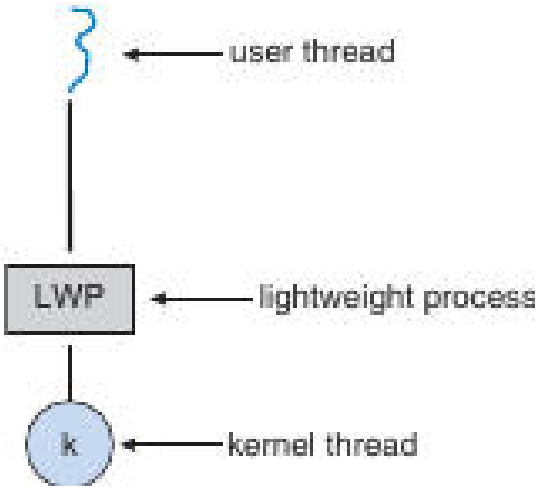
- Unlike ordinary local variables (which exist only for the duration of a single function call), TLS lives for the entire lifetime of the thread.
- Typical use: per-thread identifiers, transaction-specific buffers, or any state that must not be shared.

Platform	TLS Mechanism
POSIX	pthread_key_create(), pthread_getspecific(), pthread_setspecific()
Windows	TlsAlloc(), TlsGetValue(), TlsSetValue()
Java	ThreadLocal class

🔄 Scheduler Activations & Lightweight Processes (LWPs)

> **Definition:** Scheduler activations provide a communication channel between the kernel and a user-level thread library, enabling the library to manage *virtual processors* (LWPs).

- **LWPs** act as lightweight bridges: each LWP is bound to a kernel thread and appears to the user-level library as a *virtual processor* onto which user threads are scheduled.
- When a user thread blocks (e.g., on I/O), the kernel issues an **upcall** to the library, notifying it that a virtual processor will become idle.
- The library's upcall handler can then:
  1. Save the state of the blocked thread.
  2. Allocate a new LWP (if needed) or reuse an idle one.
  3. Schedule another ready user thread onto the newly available virtual processor.
- A second upcall occurs when the blocked operation completes, allowing the library to mark the original thread as runnable and schedule it.



The diagram shows the relationship between a user thread, its lightweight process (LWP), and the underlying kernel thread.

- Scheduler activations enable many-to-many or two-level many-to-many threading models to adapt dynamically to the number of available physical CPUs and I/O-bound workloads.

🪟 Windows Thread Implementation

> **Definition:** In the Windows family (95, NT, 2000, XP, 7, ...), each thread is represented by a set of kernel and user-mode structures.

Structure	Role
ETHREAD	Executive thread block (kernel-space)
KTHREAD	Kernel-mode thread control block (scheduling, kernel stack)
TEB	Thread Environment Block (user-space), holds thread ID, user stack, TLS, etc.

- **One-to-one** mapping: every user-level thread corresponds to a distinct kernel thread.
- The kernel maintains the full thread state (register set, stacks) and schedules them independently on multiple processors.

🐧 Linux Thread Implementation

> **Definition:** Linux treats both processes and threads as **tasks**, represented by struct task_struct.

- Threads are created with the clone() system call, which takes a set of flags to specify which resources are shared with the parent.

Flag	Shared Resource
CLONE_FS	File-system information (cwd, root)
CLONE_VM	Virtual memory space
CLONE_SIGHAND	Signal handlers
CLONE_FILES	Open file table

- Example flag combination (CLONE_FS | CLONE_VM | CLONE_SIGHAND | CLONE_FILES) creates a thread that shares the same address space, file descriptors, and signal handlers as its parent – the typical POSIX thread behavior.
- If **no** flags are set, clone() behaves like fork(), creating a separate process with its own copies of all resources.
- The kernel holds a unique task_struct for each task, containing pointers to the actual shared data structures (e.g., memory descriptors, file tables). This indirection enables fine-grained sharing as dictated by the clone flags.

🧩 Multithreaded Sudoku Validation

> *Definition:* A **worker thread** receives a portion of the Sudoku grid, checks the validity of its assigned rows, columns, or sub-grids, and reports the result back to the **parent thread**.

- **Parameter passing**
  - Allocate a structure with malloc(sizeof(parameters)) containing the coordinates (row, column, etc.).
  - Pass a pointer to this structure when creating the thread (pthread_create on POSIX, CreateThread on Windows).
- **Result communication**
  - Create a **global integer array** result[N] (where N equals the number of worker threads).
  - Index i of the array corresponds to worker thread *i*.
  - After checking, the worker stores 1 (valid) or 0 (invalid) in result[i].
  - The parent thread waits for all workers to finish, then scans the array; the Sudoku puzzle is valid only if every entry equals 1.

📊 Multithreaded Sorting Application

> *Definition:* A program that splits an unsorted array into two halves, sorts each half concurrently, and merges the results with a third thread.

- **Data layout**
  - **Global array** A[size] holds the original data.
  - **Second global array** B[size] receives the merged, sorted output.
- **Thread responsibilities**
  - **Sorting Thread 0** – sorts A[0 … size/2 - 1].
  - **Sorting Thread 1** – sorts A[size/2 … size-1].
  - **Merging Thread** – combines the two sorted sub-lists into B.
- **Parameter passing**
  - Each sorting thread receives the **starting index** and **length** of the segment it must sort (see the earlier "passing parameters to a thread" guidelines).
- **Final output**
  - After the merging thread finishes, the parent thread prints the sorted array from B.

🔄 Process Synchronization Overview

> *Definition:* **Cooperating processes** (or threads) share data and must coordinate their actions to avoid **data inconsistency**.

- **Typical problems** – race conditions, lost updates, and inconsistent views of shared resources.
- **Goal of synchronization** – enforce orderly access to critical resources while preserving progress and fairness.

⚙️ The Critical-Section Problem

> *Definition:* A **critical section** is a segment of code that accesses shared variables and must not be executed by more than one process (or thread) at the same time.

Requirement	Meaning
Mutual exclusion	No two processes may be in their critical sections simultaneously.
Progress	If no process is in a critical section, a requesting process will eventually enter.
Bounded waiting	There is an upper bound on how many times other processes may enter their critical sections before a requesting process is granted entry.

The typical process structure:

```
do {
    entry_section;
    critical_section;
    exit_section;
    remainder_section;
} while (true);
```

🤝 Peterson's Solution (Two-Process Algorithm)

> *Definition:* A classic software algorithm that satisfies mutual exclusion, progress, and bounded waiting for **exactly two** processes.

Key shared variables:

```
int turn;
bool flag[2];
```

- flag[i] → *true* when process *i* wishes to enter its critical section.
- turn → indicates whose turn it is to enter.

Algorithm (process i, where j = 1-i):

```
do {
    flag[i] = true;        // indicate interest
    turn = j;              // give priority to the other process
    while (flag[j] && turn == j) ;   // busy-wait
    /* critical section */
    flag[i] = false;       // exit protocol
    /* remainder section */
} while (true);
```

- Guarantees mutual exclusion because at least one of the two conditions (!flag[j] or turn != j) must be true before a process proceeds.
- Works on architectures where reads/writes to a single word are atomic; it may fail on modern CPUs with aggressive reordering.

🛠️ Synchronization Hardware Primitives

Primitive	Atomic behavior	Typical usage
Test-and-Set	Reads a Boolean, sets it to *true* in a single indivisible step.	Implements simple spin locks.
Compare-and-Swap (CAS)	Replaces a memory word with a new value only if it currently holds an expected value; returns the original word.	Basis for lock-free data structures and bounded-waiting algorithms.

Both primitives must execute **atomically** across all CPUs; the hardware guarantees that concurrent invocations are serialized in some order.

🧩 Test-and-Set Example

> *Definition:* test_and_set(&lock) returns the previous value of lock and sets lock to *true* atomically.

```
bool test_and_set(bool *target) {
    bool rv = *target;
    *target = true;
    return rv;
}
```

Spin-lock pattern:

```
while (test_and_set(&lock)) ;   // busy-wait until lock becomes free
/* critical section */
lock = false;                    // release
```

- Inefficient on multiprocessors because the busy loop consumes CPU cycles and may cause cache-line ping-pong.

🔄 Compare-and-Swap (CAS) Example

> *Definition:* compare_and_swap(&var, expected, new) atomically sets *var to new only if *var == expected; returns the original value.

```
int compare_and_swap(int *value, int expected, int new) {
    int temp = *value;
    if (*value == expected)
        *value = new;
    return temp;
}
```

CAS-based lock:

```
while (compare_and_swap(&lock, 0, 1) != 0) ;   // acquire
/* critical section */
lock = 0;                                      // release
```

- Eliminates the need for a separate “test-then-set” sequence, reducing the window for race conditions.

⏱️ Bounded-Waiting Algorithm Using Test-and-Set

> *Definition:* An extension of the basic test-and-set lock that guarantees each process will wait at most *n-1* turns (where *n* is the number of processes).

Shared data:

```
bool waiting[N];   // N = number of processes
bool lock = false;
```

Algorithm for process *i*:

```
do {
    waiting[i] = true;
    while (waiting[i]) {
        if (!lock && test_and_set(&lock) == false) {
            waiting[i] = false;          // acquire lock
        }
    }
    /* critical section */
    lock = false;                        // release
    waiting[i] = false;                  // allow next waiting process
    /* remainder section */
} while (true);
```

- The waiting array ensures that a process can be overtaken by at most *N-1* others before it gains the lock, satisfying the bounded-waiting requirement.

🔐 Mutex Locks (Software-Level)

> *Definition:* A **mutex** (mutual-exclusion lock) abstracts the hardware primitives into a simple acquire/release API.

Typical operations:

- acquire() – blocks the calling thread until the mutex becomes available, then marks it as held.
- release() – makes the mutex available for other threads.

Implementation sketch (using a Boolean lock and a test-and-set primitive):

```
bool lock = false;

void acquire(void) {
    while (test_and_set(&lock)) ;   // spin until lock is obtained
}

void release(void) {
    lock = false;                    // free the mutex
}
```

- In user-level libraries (e.g., POSIX pthread_mutex_t), the acquire/release functions may also block the thread (instead of spinning) to improve CPU utilization.
- A mutex satisfies **mutual exclusion**; additional fairness policies (e.g., FIFO ordering) are often added by the OS to meet **progress** and **bounded waiting**.

🔒 Mutexes & Spinlocks

> **Mutex** – a mutual-exclusion lock that guarantees only one process (or thread) can be in its critical section at a time.

- **acquire()** – attempts to obtain the lock; if the lock is unavailable the caller *spins* (busy-wait) until it becomes free.
- **release()** – atomically sets the lock to *available* so that another waiting process can acquire it.

```
/* simplified spin-lock */
bool lock = false;

void acquire(void) {
    while (test_and_set(&lock)) ;   // spin while lock is true
}

void release(void) {
    lock = false;                    // make lock available
}
```

- **Busy waiting** wastes CPU cycles because the waiting process repeatedly reads the lock variable.
- **Advantages** – no context switch while waiting, which is useful when the lock is held only a very short time (e.g., on a multiprocessor where one core can spin while another holds the lock).
- **Disadvantages** – on a single-CPU system the spinning process prevents any other process from making progress; the CPU is consumed without doing useful work.

Spinlocks are therefore preferred for short critical sections on multiprocessor machines.

🟢 Semaphores

> **Semaphore** – an integer variable accessed only through two atomic operations, **wait()** (P) and **signal()** (V).

Type	Value Range	Typical Use
Binary	0 or 1	Acts like a mutex for mutual exclusion
Counting	0 … ∞	Controls access to a pool of identical resources

Primitive Operations

```
/* wait (P) */
void wait(semaphore *S) {
    while (S->value <= 0) ;   // busy-wait
    S->value--;
}

/* signal (V) */
void signal(semaphore *S) {
    S->value++;
}
```

- Both operations must be **indivisible**: while one process is executing wait() or signal(), no other process may modify the same semaphore.
- The original names *P* (from Dutch "proberen", to test) and *V* (from "verhogen", to increment) reflect this history.

Binary vs. Counting

- A **binary semaphore** can replace a mutex when the system provides only semaphores.
- A **counting semaphore** is initialized to the number of available resources; each wait() consumes one resource, each signal() releases one.

⚙️ Semaphore Implementation (Blocking Queue)

> **Goal** – eliminate busy waiting by putting a process to sleep when the semaphore is unavailable.

```
typedef struct {
    int value;                  // current count
    queue *list;                // queue of blocked PCBs
} semaphore;
```

- **wait(semaphore *S)**
  1. Decrement S->value.
  2. If the result is negative, place the calling process on S->list and **block** it (state → *waiting*).
- **signal(semaphore *S)**
  1. Increment S->value.
  2. If the new value ≤ 0, remove one process from S->list and **wake up** that process (state → *ready*).

Atomicity is achieved by disabling interrupts (single-CPU) or by using hardware primitives such as **compare-and-swap** on multiprocessors.

⚠️ Deadlocks & Starvation

> **Deadlock** – a set of processes where each is waiting for an event that only another member of the set can cause.

- Classic example: two processes each hold one semaphore and wait for the other (wait(S); wait(Q); … signal(S); signal(Q);).
- No process can proceed → system is deadlocked.

> **Starvation** – a process waits indefinitely because the scheduling or synchronization policy never grants it the needed resource.

- Can arise from **LIFO** waiting queues (newer processes repeatedly pre-empt older ones).
- Using a **FIFO** queue for semaphore wait lists helps guarantee bounded waiting.

Issue	Symptom	Typical Remedy
Deadlock	All involved processes are blocked	Enforce a global ordering of resource acquisition, or use a timeout/recovery protocol

Starvation	One or more processes never obtain the resource	FIFO queues, priority inheritance, or fair scheduling algorithms

---

🔼 Priority Inversion

> **Priority inversion** – a low-priority process holds a resource needed by a high-priority process, while a medium-priority process pre-emptively runs, delaying the low-priority holder.

Real-world case: *Mars Pathfinder* (1997) suffered frequent resets because a high-priority task waited for a resource owned by a low-priority task that was pre-empted by medium-priority tasks.

Solution – Priority Inheritance:

- When a higher-priority task blocks on a lock held by a lower-priority task, the holder **temporarily inherits** the higher priority.
- The inherited priority prevents intermediate-priority tasks from pre-empting the holder, allowing it to release the lock sooner.

📦 Classic Synchronization Problems

🏭 Bounded-Buffer (Producer-Consumer)

Shared data structures:

```
int n;                    // buffer size
semaphore mutex = 1;      // protects buffer access
semaphore empty = n;      // counts empty slots
semaphore full  = 0;      // counts filled slots
```

Producer

```
do {
    wait(empty);
    wait(mutex);
    /* add item to buffer */
    signal(mutex);
    signal(full);
} while (true);
```

Consumer

```
do {
    wait(full);
    wait(mutex);
    /* remove item from buffer */
    signal(mutex);
    signal(empty);
} while (true);
```

- mutex guarantees mutual exclusion; empty and full enforce the buffer capacity constraints.

📖 Readers–Writers

- **First readers-writers problem** – readers may proceed unless a writer is already active.
- **Second readers-writers problem** – once a writer is waiting, no new readers may start.

Common data structures:

```
semaphore rw_mutex = 1;   // protects the shared resource for writers
semaphore mutex   = 1;    // protects read_count
int read_count = 0;
```

Reader

```
do {
    wait(mutex);
    read_count++;
    if (read_count == 1) wait(rw_mutex);   // first reader locks writers out
    signal(mutex);

    /* reading section */

    wait(mutex);
    read_count--;
    if (read_count == 0) signal(rw_mutex); // last reader releases writers
    signal(mutex);
} while (true);
```

Writer

```
do {
    wait(rw_mutex);
    /* writing section */
    signal(rw_mutex);
} while (true);
```

- The first reader acquires rw_mutex; subsequent readers proceed without further blocking.
- When the last reader exits, rw_mutex is released, allowing a waiting writer to proceed.

🍚 Dining Philosophers

- **Resources:** five chopsticks, each represented by a binary semaphore chopstick[i] initialized to 1.

```
do {
    wait(chopstick[i]);              // pick up left chopstick
    wait(chopstick[(i+1) % 5]);      // pick up right chopstick
    /* eat */
    signal(chopstick[i]);            // put down left
    signal(chopstick[(i+1) % 5]);    // put down right
    /* think */
} while (true);
```

Deadlock prevention strategies

Strategy	Idea
Limit concurrent philosophers	Allow at most four philosophers to sit simultaneously.
Resource ordering	Require each philosopher to pick up the lower-numbered chopstick first.
Asymmetric solution	Even-numbered philosophers pick left then right; odd-numbered pick right then left.
Acquire both chopsticks inside a *single* critical section (using an extra mutex).	

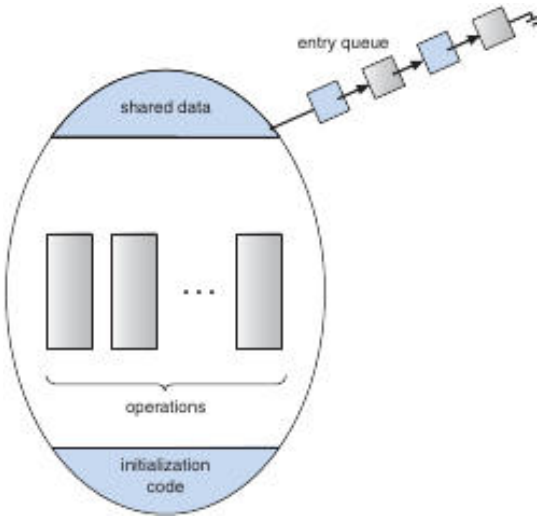
Even with deadlock avoidance, **starvation** must still be considered; fairness mechanisms (e.g., FIFO queues) are needed.

💻 Monitors & Condition Variables

> **Monitor** – a high-level synchronization construct that combines a lock with *condition variables*; only one process can execute inside the monitor at any time.

```
monitor Buffer {
    int count = 0;
    condition notFull, notEmpty;

    void put(item) {
        if (count == MAX) wait(notFull);
        /* add item */
        count++;
        signal(notEmpty);
    }

    item get() {
        if (count == 0) wait(notEmpty);
        /* remove item */
        count--;
        signal(notFull);
    }
}
```

- **Condition variable operations**
  - x.wait(); – the calling process is suspended and placed on x's waiting queue.
  - x.signal(); – wakes **exactly one** process waiting on x (if any).
- Unlike semaphores, a signal() has no effect when no process is waiting; the condition's state simply remains unchanged.



The figure shows a monitor's internal layout: shared data at the top, initialization code at the bottom, and three operation blocks in the middle. An entry queue on the right holds processes waiting to enter the monitor.

- The monitor's **implicit lock** ensures that only one process executes any of its functions at a time, removing the need for explicit wait(mutex)/signal(mutex) pairs.
- Condition variables enable processes to *block* inside the monitor while waiting for a particular state (e.g., buffer not full).

Monitors Overview

> **Definition:** A *monitor* is a high-level synchronization construct that encapsulates shared data, the procedures that operate on that data, and the condition variables used to coordinate access. Only one process (or thread) may be active inside the monitor at any time.

- **Signal semantics**
  1. **Signal-and-wait** – The signaling process **waits** until the waiting process leaves the monitor.
  2. **Signal-and-continue** – The signaling process **continues** immediately; the waiting process is resumed only after the signaling process exits the monitor.

  Both approaches have arguments in their favor. The former can guarantee that the condition still holds when the waiting process runs, while the latter reduces the time the signaling process is blocked.

  Compromise: Languages such as **Concurrent Pascal** adopt a hybrid rule: after executing signal, the signaling thread **immediately leaves** the monitor, allowing the waiting thread to be resumed.

🍽️ Dining-Philosophers Solution Using Monitors

- **Problem:** Prevent deadlock while allowing each of the five philosophers to eat only when both adjacent chopsticks are free.
- **State enumeration**

```
enum { THINKING, HUNGRY, EATING } state[5];
condition self[5];
```

- **Monitor operations**

```
void pickup(int i) {
    state[i] = HUNGRY;
    test(i);
    if (state[i] != EATING) self[i].wait();
}

void putdown(int i) {
    state[i] = THINKING;
    test((i+4) % 5);   // left neighbor
    test((i+1) % 5);   // right neighbor
}

void test(int i) {
    if (state[i] == HUNGRY &&
        state[(i+4)%5] != EATING &&
        state[(i+1)%5] != EATING) {
        state[i] = EATING;
        self[i].signal();
    }
}
```

- **Correctness properties**
  - No two neighboring philosophers can be in **EATING** simultaneously (mutual exclusion).
  - The solution is **deadlock-free** because a philosopher never holds a single chopstick while waiting for the other.
  - **Starvation** is not addressed; guaranteeing progress for every hungry philosopher is left as an exercise.

⚙️ Implementing a Monitor Using Semaphores

- **Core semaphores**
  - mutex (initialized to 1) protects entry to the monitor.

    - next (initialized to 0) and integer next_count manage the hand-off between a signaling thread and the resumed thread.
- **Entry/exit protocol** (for any monitor operation F):
    1. wait(mutex); → enter monitor.
    2. Execute the body of F.
    3. If next_count > 0 then signal(next); else signal(mutex); → leave monitor.
- **Condition variable implementation** (for condition x):
    - Semaphore x_sem (0) and integer x_count (0).
    - **wait(x)**

```
x_count++;
if (next_count > 0) signal(next);
else signal(mutex);
wait(x_sem);
x_count--;
```

    - **signal(x)**

```
if (x_count > 0) {
    next_count++;
    signal(x_sem);
    wait(next);
    next_count--;
} else {
    // no waiting process
}
```

- This scheme works for both **Hoare** (immediate hand-off) and **Mesa** (signal-and-continue) monitor models; the choice is reflected in whether the signaling thread waits on next.

🔁 Process-Resumption Order in Monitors

- **FCFS (first-come, first-served)** is the simplest policy: the longest-waiting process is resumed first.
- **Priority-based resumption** uses x.wait(c) where c is an integer priority evaluated at wait time. The x.signal() operation resumes the waiting process with the **smallest** c.

ResourceAllocator Example

```
monitor ResourceAllocator {
    boolean busy;
    condition x;
    void acquire(int time) {
        if (busy) x.wait(time);
        busy = true;
    }
    void release() {
        busy = false;
        x.signal();
    }
}
```

- Processes request a resource for a maximum duration time. The monitor grants the resource to the request with the **shortest** time (lowest priority number).
- Correctness requires:
    1. All user processes must invoke acquire/release in the proper order.

2. No process may bypass the monitor to access the shared resource directly.

☕ Java Monitors

> **Definition:** In Java, every object possesses an intrinsic monitor lock. Declaring a method synchronized makes the executing thread **acquire** that object's lock before entering the method and **release** it upon exit.

- **Lock acquisition** – If another thread already holds the lock, the calling thread blocks in the object's *entry set* (wait queue).
- **wait() / notify()** – Correspond to monitor wait and signal. A thread calling wait() releases the object's lock and is placed on the condition queue; notify() wakes one waiting thread, notifyAll() wakes all.
- **java.util.concurrent** – Provides higher-level constructs (semaphores, locks, condition variables, thread pools) that build on the same underlying monitor mechanisms.

🖥️ Synchronization Examples Across Operating Systems

🔷 Windows

- **Dispatcher objects** (mutexes, semaphores, events, timers) exist in either **signaled** or **nonsignaled** state.
- **Mutex** – Only one thread may own it; other threads block until it becomes signaled.
- **Critical-section object** – User-mode mutex that first spins, then falls back to a kernel mutex if contention persists, reducing kernel overhead for short critical sections.
- **Spinlocks** – Used inside the kernel; a thread holding a spinlock is never pre-empted.

🔷 Linux

- **Atomic integers** (atomic_t) provide lock-free operations (atomic_set, atomic_add, atomic_inc, atomic_read).
- **Mutexes** – mutex_lock() / mutex_unlock() protect kernel critical sections; a blocked thread sleeps and is awakened when the lock is released.
- **Spinlocks** – Preferred for very short sections on SMP systems; on single-processor systems the kernel disables preemption instead of spinning.
- **Preempt control** – preempt_disable() / preempt_enable() and a per-task preempt_count ensure that code holding a lock cannot be pre-empted.

🔷 Solaris

- **Adaptive mutexes** – Spin briefly if the lock holder is running on another CPU; otherwise the thread sleeps.
- **Turnstiles** – Queues attached to a lock that order waiting threads; each kernel thread owns its own turnstile, which can be reused for multiple locks.
- **Priority inheritance** – When a high-priority thread blocks on a lock held by a lower-priority thread, the holder temporarily inherits the higher priority, preventing priority inversion.
- **Reader–writer locks** – Allow concurrent reads while exclusive writes serialize access; more expensive than semaphores but beneficial for read-heavy workloads.

🔷 Pthreads (User-level)

- **Mutexes** – Created with pthread_mutex_init; acquired with pthread_mutex_lock, released with pthread_mutex_unlock.
- **Condition variables** – pthread_cond_wait and pthread_cond_signal implement the monitor wait/signal pattern.
- **Semaphores** (POSIX extension) – Unnamed (sem_init) and named (sem_open) variants; sem_wait / sem_post correspond to classic wait / signal.
- **Spinlocks** – Available via pthread_spin_init, pthread_spin_lock, pthread_spin_unlock; suitable for short critical sections on multiprocessor systems.

🔄 Alternative Approaches

💾 Transactional Memory

> **Definition:** *Transactional memory* allows a block of memory operations to execute atomically: either the entire block **commits** or **aborts** and rolls back, without explicit lock acquisition.

- **Programming model** – An atomic { … } block encloses the critical code. Example (conceptual):

```
void update() {
    atomic {
        /* modify shared data */
    }
}
```

- **Advantages**
  - Eliminates classic lock-based deadlocks.
  - Simplifies reasoning about concurrency; the runtime determines conflicts.
  - Can improve scalability under high contention.
- **Implementations**
  - **Software Transactional Memory (STM):** Instrumented by the compiler; manages read/write logs and validates at commit time.
  - **Hardware Transactional Memory (HTM):** Leverages cache-coherency protocols to detect conflicts with lower overhead, but requires processor support and modifications to the cache hierarchy.

🛠️ OpenMP Parallelism (brief)

- Provides compiler directives (e.g., #pragma omp parallel for) for shared-memory parallelism.
- Integrates with existing synchronization primitives (critical sections, atomic clauses, locks) to manage data races in multithreaded loops.

🟢 OpenMP Parallelism (Shared-Memory)

> **OpenMP** is a set of compiler directives, library routines, and environment variables that enable parallel programming in a shared-memory environment.

- The directive #pragma omp parallel defines a **parallel region**.
- By default the runtime creates a team of threads equal to the number of processing cores.
- Thread creation and management are handled by the OpenMP library, relieving the application developer of low-level bookkeeping.

Advantages

- Simplifies parallel code: only the region to be executed concurrently needs to be marked.
- Portable across many Unix-like and Windows compilers that support the OpenMP API.

🔐 OpenMP Critical Sections

> **Critical section** – a region of code that must be executed by only one thread at a time.

OpenMP provides the #pragma omp critical directive to protect shared data.
The construct behaves like a binary semaphore (or mutex) that guarantees mutual exclusion.

```
/* OpenMP critical-section example */
void update(int value) {
    #pragma omp critical
    {
        counter += value;   // only one thread modifies `counter` at a time
    }
}
```

- If a thread attempts to enter a critical section while another thread holds it, the caller **blocks** until the owner exits.
- Multiple named critical sections can be defined; each name has its own independent lock.

- **Pros:** Simple to use, no explicit lock handling.
- **Cons:** Still subject to deadlock if a thread acquires several critical sections in an inconsistent order.

🧩 Functional Programming Languages

> **Functional languages** emphasize immutable data and stateless computation.

Aspect	Imperative (e.g., C, C++, Java)	Functional (e.g., Erlang, Scala)
State	Mutable variables; state changes are explicit.	Variables are immutable; once bound they cannot change.
Concurrency concerns	Race conditions and deadlocks must be managed with locks, semaphores, etc.	Immutable data eliminates the need for mutual-exclusion primitives.
Typical use	General-purpose system and application programming.	Highly concurrent or distributed systems where safety is paramount.

- **Erlang** is renowned for lightweight processes and built-in message passing, making it suitable for large-scale concurrent applications.
- **Scala** blends object-oriented and functional paradigms; its syntax resembles Java while offering immutable collections and powerful concurrency abstractions.

⚔️ Deadlocks Overview

> **Deadlock** – a situation where a set of processes (or threads) are each waiting for an event that only another member of the set can cause, causing all of them to remain blocked forever.

Four Necessary Conditions (all must hold simultaneously)

#	Condition	Meaning
1	**Mutual exclusion**	At least one resource can be held by only one process at a time.
2	**Hold and wait**	A process holds one or more resources while waiting for additional ones.
3	**No preemption**	Resources cannot be forcibly taken away; they must be released voluntarily.
4	**Circular wait**	A circular chain of processes exists, each waiting for a resource held by the next.

If any condition is broken, deadlock cannot occur.

🗃️ System Model & Resource Types

- The system contains a **finite set of resource types**; each type may have multiple identical instances (e.g., 2 CPUs, 5 printers).
- A **resource class** groups instances that are interchangeable from the point of view of a process.
- Processes follow the **request → use → release** sequence for each resource they need.

Example Resource Types

Type	Instances	Notes
CPU cycles	2	Two instances → two CPUs.
Printers	5	May be treated as one class if any printer suffices; otherwise separate classes for

		location-specific printers.
Memory pages	many	Logical resources managed by the OS.

📊 Resource-Allocation Graph (RAG)

- A directed graph with **process vertices** (P) and **resource-type vertices** (R).
- **Assignment edge** R → P indicates an instance of R is allocated to P.
- **Request edge** P → R indicates P is waiting for an instance of R.

Interpretation Rules

- **Cycle with single-instance resources** ⇒ deadlock.
- **Cycle with multi-instance resources** ⇒ may indicate deadlock but is not sufficient; additional analysis required.

🧩 Deadlock Example Using Pthread Mutexes

The following C code demonstrates a classic deadlock caused by acquiring two mutexes in opposite order.

```
/* Pthread deadlock illustration */
#include
#include
#include

pthread_mutex_t first_mutex, second_mutex;

/* Thread 1 */
void *do_work_one(void *arg) {
    pthread_mutex_lock(&first_mutex);
    /* simulate work */
    pthread_mutex_lock(&second_mutex);
    /* critical section */
    pthread_mutex_unlock(&second_mutex);
    pthread_mutex_unlock(&first_mutex);
    return NULL;
}

/* Thread 2 */
void *do_work_two(void *arg) {
    pthread_mutex_lock(&second_mutex);
    /* simulate work */
    pthread_mutex_lock(&first_mutex);
    /* critical section */
    pthread_mutex_unlock(&first_mutex);
    pthread_mutex_unlock(&second_mutex);
    return NULL;
}
```

- Thread 1 locks first_mutex then second_mutex.
- Thread 2 locks second_mutex then first_mutex.
- If the two threads interleave such that each holds its first lock while waiting for the second, a circular wait forms → deadlock.

Avoidance tip: impose a **global lock acquisition order** (e.g., always lock first_mutex before second_mutex).

📉 Detecting & Handling Deadlocks

Detection Strategies

- **Resource-allocation-graph cycle detection**: periodic traversal to find cycles.
- **Wait-for graph** (process-level abstraction): edges represent “process i is waiting for process j”.

If a cycle is found, the system can invoke recovery actions.

Recovery Techniques

1. **Process termination** – abort one or more involved processes to break the cycle.
2. **Resource preemption** – forcibly reclaim a resource from a process (requires careful state handling).
3. **Rollback** – revert processes to a safe checkpoint before the deadlock formed.

🛡️ Deadlock Prevention & Avoidance

Approach	How it Works	Example
Prevention	Enforce a rule that invalidates at least one of the four necessary conditions.	Disallow hold-and-wait by requiring processes to request all needed resources atomically.
Avoidance	Dynamically examine future requests using additional information (e.g., maximum resource need) and only grant a request if the system remains in a safe state.	**Banker's algorithm**: simulates allocation before committing.
Combined	Apply prevention for high-risk resources and avoidance for others, balancing overhead and safety.	Use a strict ordering for mutexes (prevention) while employing the Banker's algorithm for I/O devices.

🔄 Recovery After Deadlock Detection

1. **Select victim(s)** – choose one or more processes to terminate or roll back (often the one with least work done).
2. **Release resources** held by victims.
3. **Restart affected processes** (if possible) after resources become available.

Operating systems such as Linux and Windows typically **do not implement automatic deadlock recovery**; they rely on the application to avoid or resolve deadlocks, making careful design essential.

📚 Summary of Key Points (Reference)

- **OpenMP** simplifies shared-memory parallelism; #pragma omp critical provides a high-level mutual-exclusion construct.
- **Functional languages** avoid traditional synchronization problems by enforcing immutability.
- **Deadlocks** require all four classic conditions; breaking any one eliminates the possibility.
- **Resource-allocation graphs** give a visual method to reason about potential deadlocks; cycles with single-instance resources guarantee deadlock.
- **Mutex-order violations** (as shown in the pthread example) are a common source of deadlock; consistent ordering prevents the circular-wait condition.
- **Prevention, avoidance, and detection/recovery** are the three fundamental strategies for handling deadlocks; most real-world systems combine them based on resource characteristics.

🌀 Spinlocks vs Mutexes

> **Spinlock:** a lock that repeatedly tests a variable (busy-wait) until it becomes free.
> **Mutex:** a lock that puts waiting threads to sleep, releasing the CPU.

- **When to prefer a mutex:**
  1. Critical section may hold the lock for an *unbounded* time.
  2. System wants to avoid wasting CPU cycles on busy-waiting.
- **When a spinlock can be advantageous:**
  - The lock is held only for a *very short* interval (e.g., updating a single word).
  - Upper-bound time is known, so the cost of spinning is less than the context-switch overhead of a mutex.

Property	Spinlock	Mutex
CPU usage while waiting	High (busy-wait)	Low (blocked)
Suitable for short critical sections	✅	✅ (but slower)
Guarantees FIFO ordering of waiters	❌	✅ (implementation-dependent)

📈 Counter Synchronization Strategies

Two ways to protect the **hits** variable in a multithreaded web server:

Strategy	Code sketch	Synchronization primitive	Expected efficiency
Mutex lock	hit_lock.acquire(); hits++; hit_lock.release();	pthread_mutex (or OS mutex)	Guarantees mutual exclusion; incurs lock/unlock overhead.
Atomic integer	atomic_inc(&hits);	Hardware-supported atomic operation (fetch_add)	No explicit lock; typically faster for simple increments.

> **Observation:** For a single-word increment, the *atomic* version is usually more efficient because it avoids the kernel-mode transition and context switches required by a mutex.

🔧 Race Conditions in Process Allocation (Figure 5.27)

Problem:

```
int number_of_processes = 0;
int allocate_process() {
    if (number_of_processes == MAX_PROCESSES) return -1;
    ++number_of_processes;
    return new_pid;
}
void release_process() {
    --number_of_processes;
}
```

a. Identified race conditions

- Two threads may simultaneously test number_of_processes == MAX_PROCESSES and both proceed, exceeding the limit.
- Increment/decrement of number_of_processes is not atomic → lost updates.

b. Fix with a mutex

```
pthread_mutex_t proc_lock = PTHREAD_MUTEX_INITIALIZER;

int allocate_process() {
    pthread_mutex_lock(&proc_lock);
    if (number_of_processes == MAX_PROCESSES) {
        pthread_mutex_unlock(&proc_lock);
        return -1;
    }
    ++number_of_processes;
    pthread_mutex_unlock(&proc_lock);
    return new_pid;
}
void release_process() {
    pthread_mutex_lock(&proc_lock);
    --number_of_processes;
```

```
    pthread_mutex_unlock(&proc_lock);
}
```

c. Using an atomic integer

- Replacing int number_of_processes with atomic_int **eliminates the race** for the increment/decrement, but the *capacity check* (== MAX_PROCESSES) still requires a critical section to be safe.
- Therefore a full replacement is **not sufficient**; a lock (or compare-and-swap loop) is still needed for the bound check.

🌐 Controlling Concurrent Connections with Semaphores

> **Semaphore:** an integer counter with atomic wait() (P) and signal() (V) operations.

Server scenario: Allow at most **N** simultaneous socket connections.

```
sem connection_sem = N;   // initial count = N

/* On new connection request */
wait(connection_sem);    // blocks if N connections already active
accept_connection();     // proceed with socket handling

/* When a client disconnects */
close_connection();
signal(connection_sem);  // frees a slot for another client
```

- The semaphore **blocks** excess clients until a slot becomes free, guaranteeing the limit without busy-waiting.

🪟 Windows Vista Slim Reader-Writer (SRW) Locks

- **SRW lock** is a lightweight synchronization primitive introduced in Windows Vista.
- **Key properties:**
  - Supports *shared* (reader) and *exclusive* (writer) modes.
  - Does **not** enforce FIFO ordering; threads are scheduled by the kernel scheduler.
  - No explicit “fairness” guarantee, but provides better performance than traditional CRITICAL_SECTION when many readers coexist.

🛠️ Implementing wait() / signal() with Test-and-Set (Multiprocessor)

Goal: minimal instruction sequence.

```
/* lock is a Boolean initialized to false */
void wait(semaphore *S) {
    while (true) {
        while (test_and_set(&S->lock)) ;   // acquire the spinlock
        if (S->value > 0) {
            --S->value;
            S->lock = false;               // release spinlock
            break;
        }
        S->lock = false;                   // release spinlock
        // optional pause or back-off before retry
    }
}

void signal(semaphore *S) {
    while (test_and_set(&S->lock)) ;       // acquire spinlock
    ++S->value;
```

```
    S->lock = false;                    // release
}
```

- The **test-and-set** primitive guarantees atomic acquisition of the internal spinlock; the outer loops implement the semaphore semantics without kernel calls.

👶 Parent-Child Thread Coordination (Fibonacci)

Original requirement: parent waits (pthread_join) before printing the Fibonacci numbers.

Modified design to allow early access:

- Use a **shared buffer** protected by a **mutex** and a **condition variable**.
- Child thread writes each newly computed Fibonacci value, then pthread_cond_signal.
- Parent thread loops, waiting on the condition variable, prints values as soon as they become available, and continues until the child signals completion (e.g., with a special sentinel).

```
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  cv  = PTHREAD_COND_INITIALIZER;
int fib_val;
bool done = false;
```

- This eliminates the need for the parent to block until the child terminates.

🔁 Monitors vs Semaphores Equivalence

- **Monitor:** encapsulates shared data, procedures, and condition variables; only one process may be active inside the monitor at a time.
- **Semaphore:** provides wait() and signal() primitives that can be used to build the same mutual-exclusion and waiting behavior.

Construction sketch:

- A binary semaphore (mutex = 1) implements the monitor's implicit lock.
- Each monitor condition variable is represented by a **queue** of waiting processes plus a **counting semaphore** (cond_sem = 0).
- wait(C) → signal(mutex); wait(cond_sem);
- signal(C) → signal(cond_sem); (optionally re-acquire mutex depending on Hoare vs. Mesa semantics).

Thus any monitor can be expressed with semaphores, and conversely semaphores can be wrapped in a monitor interface.

📦 Bounded-Buffer Monitor Design (Embedded Portions)

Standard monitor version (small critical sections):

```
monitor Buffer {
    int count = 0;
    condition notFull, notEmpty;

    void put(item x) {
        if (count == MAX) wait(notFull);
        /* insert x */
        ++count;
        signal(notEmpty);
    }

    item get() {
        if (count == 0) wait(notEmpty);
        /* remove item */
        --count;
        signal(notFull);
```

```
    }
}
```

- Each wait/signal pair protects only the *tiny* buffer manipulation, making busy-waiting negligible.

a. Why unsuitable for large portions

- The monitor forces **exclusive entry** for the whole function body, so a long computation inside put or get would block all other threads, leading to poor throughput and unnecessary waiting.

b. Scheme for larger portions

- Split the monitor into **two layers**:
  1. **Fine-grained lock** (e.g., a mutex) protecting just the buffer indices.
  2. **Separate condition variables** for notFull/notEmpty that are signaled **outside** the critical section after the buffer update.
- This allows the producer/consumer to perform non-critical work (e.g., data preparation) without holding the monitor lock.

📚 Readers-Writers Fairness vs Throughput

- **Fairness-oriented solution** (e.g., giving priority to waiting writers) reduces writer starvation but can lower overall throughput because readers are frequently blocked.
- **Throughput-oriented solution** (reader-preferring) maximizes concurrent reads but may starve writers.

Starvation-free method

- Maintain a **queue counter** (waiting_writers).
- Readers check waiting_writers; if > 0 they wait, otherwise they proceed.
- Writers acquire the lock only when read_count == 0.
- This policy gives writers eventual access while still allowing bursts of concurrent reads.

🔔 Signal Operation Differences (Monitor vs Semaphore)

Aspect	Monitor signal	Semaphore signal
Effect on waiting processes	May **transfer control** immediately (Hoare) or simply **wake** a waiter (Mesa).	Increments the semaphore count; a waiting process will be awakened by the scheduler, not by the signaling thread.
Ordering guarantee	Often **FIFO** within the condition variable's queue.	No intrinsic ordering; the kernel may wake any waiting thread.
Blocking behavior	In Hoare semantics, the signaling thread **waits** until the awakened thread exits the monitor.	The signaling thread never blocks; it continues execution.

🗃️ Simplified Monitor Implementation (Signal Only Inside Functions)

When signal() appears **exclusively** inside monitor procedures, the monitor's **entry protocol** can be simplified:

1. **Enter monitor** (wait(mutex)).
2. Execute procedure body; any signal(C) simply sets a flag need_wakeup = true.
3. **Exit monitor**:
   - If need_wakeup is true, perform signal(C) *after* releasing the mutex (Mesa style).
   - No extra hand-off (next) is required because no external code can signal the condition.

This reduces the monitor's internal state (no next semaphore) and eases implementation on platforms that only support basic mutexes and condition variables.

🖨️ Printer Allocation Monitor (Priority)

> **Goal:** Allocate three identical printers to processes, respecting each process's priority number (lower number = higher priority).

```
monitor PrinterAllocator {
    int free_printers = 3;
    condition printer_available;
    queue waiting;   // ordered by priority (min-heap)

    void request(int priority) {
        if (free_printers == 0) {
            /* insert into waiting queue sorted by priority */
            waiting.insert(this_process, priority);
            wait(printer_available);
        }
        --free_printers;
    }

    void release() {
        ++free_printers;
        if (!waiting.empty()) {
            /* wake highest-priority waiter */
            signal(printer_available);
        }
    }
}
```

- The monitor guarantees **mutual exclusion** on the printer count and respects the priority ordering via the waiting queue.

📂 File Access Monitor with Sum Constraint

Constraint: Sum of the *process numbers* of all currently accessing processes must stay **< n**.

```
monitor FileAccess {
    int current_sum = 0;
    condition ok_to_enter;
    const int LIMIT = n;

    void enter(int proc_id) {
        while (current_sum + proc_id >= LIMIT)
            wait(ok_to_enter);
        current_sum += proc_id;
    }

    void leave(int proc_id) {
        current_sum -= proc_id;
        signal(ok_to_enter);
    }
}
```

- Each process supplies its identifier when entering; the monitor blocks the process until the summed constraint is satisfied.

🔄 Signaling Semantics in Monitors

When a process executes signal(C) inside a monitor, two possible continuations exist:

1. **Signal-and-continue** (Mesa): the signaling process **continues** execution; the waiting process is merely placed on the ready queue and will run after the monitor is exited.

2. **Signal-and-wait** (Hoare): the signaling process **blocks** immediately, transferring control to the awakened process. The monitor is then re-entered by the awakened process, guaranteeing that the condition holds when it runs.

Effect on solutions:

- Hoare semantics simplify reasoning about the condition (it is true when the waiter runs).
- Mesa semantics require the waiter to **re-check** the condition after being awakened (commonly done with a while loop).

⏱️ Await Construct and Writer Problem

a. Monitor using await(B) for writers

```
monitor Writers {
    int active_readers = 0;
    bool writer_active = false;
    condition ok;

    void start_write() {
        await(!writer_active && active_readers == 0);
        writer_active = true;
    }

    void end_write() {
        writer_active = false;
        signal(ok);
    }
}
```

- await(expr) blocks until expr evaluates to true, then proceeds atomically.

b. Inefficiency reason

- General await may require **re-evaluation of an arbitrary Boolean expression**, potentially causing repeated scans of complex state and leading to *spurious wake-ups*.

c. Restrictions for efficient implementation

- Limit await to **simple predicates** involving a **single condition variable** and a **counter** (e.g., count == 0).
- This enables the runtime to use a **single semaphore** per condition, avoiding costly expression evaluation.

⏰ Alarm Monitor Using tick()

> **Goal:** Provide delay(t) that blocks the calling thread for *t* timer ticks.

```
monitor Alarm {
    int current_tick = 0;
    condition wakeup;

    void tick() {               // invoked periodically by hardware timer
        ++current_tick;
        signal(wakeup);
    }

    void delay(int t) {
        int target = current_tick + t;
        while (current_tick < target)
            wait(wakeup);
    }
}
```

- The monitor keeps a global tick count; delay waits on the wakeup condition until the required number of ticks have elapsed.

🚦 Traffic Deadlock (Figure 5.28)

a. Four necessary conditions

1. **Mutual exclusion:** each road segment can be occupied by only one vehicle.
2. **Hold and wait:** a vehicle may occupy one segment while waiting for the next.
3. **No preemption:** a vehicle cannot be forced to release a segment.
4. **Circular wait:** the four vehicles form a cycle, each waiting for the segment held by the next.

b. Simple avoidance rule

- **Enforce a global ordering** of road segments (e.g., north-south before east-west).
- Require every vehicle to request segments **in that order**, breaking the circular-wait condition.

🍽️ Dining Philosophers Deadlock Discussion

- Classic deadlock occurs when each philosopher picks up its **left** chopstick and then waits for the **right** (or vice-versa), forming a circular wait.
- The four conditions apply as in the traffic example.

Prevention strategies (refer to earlier monitor solutions):

- **Resource hierarchy:** each philosopher picks the lower-numbered chopstick first.
- **Asymmetric ordering:** even-numbered philosophers pick left then right; odd-numbered pick right then left.
- **Limit concurrent philosophers** (e.g., allow at most four of five to sit).

🆔 Thread-Safe PID Manager

Original design (single-threaded):

```
int number_of_processes = 0;
int allocate_process() { ++number_of_processes; return new_pid; }
void release_process() { --number_of_processes; }
```

Race condition: simultaneous allocate_process calls may return the same PID or corrupt the counter.

Thread-safe fix (Pthreads):

```
pthread_mutex_t pid_lock = PTHREAD_MUTEX_INITIALIZER;
int allocate_process() {
    pthread_mutex_lock(&pid_lock);
    if (number_of_processes == MAX_PROCESSES) {
        pthread_mutex_unlock(&pid_lock);
        return -1;
    }
    ++number_of_processes;
    int pid = compute_pid();
    pthread_mutex_unlock(&pid_lock);
    return pid;
}
void release_process() {
    pthread_mutex_lock(&pid_lock);
    --number_of_processes;
    pthread_mutex_unlock(&pid_lock);
}
```

- The mutex guarantees **atomicity** of allocation and release.

🧱 Resource Manager Race Condition & Condition Variable Solution

a. Data involved

- Shared variable available_resources (initially MAX_RESOURCES).

b. Race-prone locations

- The test if (available_resources < count) and the subsequent decrement are **non-atomic**; two threads may both see enough resources and both decrement, leading to a negative count.

c. Fix with a binary semaphore (mutex) and a condition variable to block until enough resources exist.

```
pthread_mutex_t  mtx = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t   enough = PTHREAD_COND_INITIALIZER;
int available = MAX_RESOURCES;

int decrease_count(int cnt) {
    pthread_mutex_lock(&mtx);
    while (available < cnt)               // wait until sufficient
        pthread_cond_wait(&enough, &mtx);
    available -= cnt;
    pthread_mutex_unlock(&mtx);
    return 0;
}

void increase_count(int cnt) {
    pthread_mutex_lock(&mtx);
    available += cnt;
    pthread_cond_broadcast(&enough);      // wake all waiting threads
    pthread_mutex_unlock(&mtx);
}
```

- Callers of decrease_count are **blocked** (no busy-wait) until enough resources become free.

🎲 Monte Carlo Parallel Estimation (Mutex Protection)

- Global counters: total_points, inside_circle.
- Each thread generates random points, tests inclusion, then updates the counters.

Required protection:

```
pthread_mutex_t cnt_lock = PTHREAD_MUTEX_INITIALIZER;
/* inside thread */
pthread_mutex_lock(&cnt_lock);
++total_points;
if (inside) ++inside_circle;
pthread_mutex_unlock(&cnt_lock);
```

- This eliminates the race on the shared counters; the final estimate $\pi \approx 4 \cdot \text{inside\_circle} / \text{total\_points}$ is then correct.

⛔ Barrier Synchronization API

> **Barrier:** a synchronization point where *N* threads must all arrive before any may proceed.

API (as described):

- int init(int n); → initialize barrier for *n* participants.
- int barrier_point(void); → called by each thread when it reaches the barrier.

Typical implementation (pseudocode):

```
int count = 0;
pthread_mutex_t bmtx = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  bcv  = PTHREAD_COND_INITIALIZER;

int barrier_point(void) {
    pthread_mutex_lock(&bmtx);
    ++count;
    if (count == N) {
        count = 0;              // reset for reuse
        pthread_cond_broadcast(&bcv);
    } else {
        while (count != 0)
            pthread_cond_wait(&bcv, &bmtx);
    }
    pthread_mutex_unlock(&bmtx);
    return 0;
}
```

- The last arriving thread **releases** all others, ensuring that no thread proceeds past the barrier until the group is complete.

💤 Sleeping TA Project Overview

Scenario: One TA, *n* student threads, three hallway chairs.

Synchronization primitives needed:

- **Semaphore ta_sleep** (initial value 0) – students signal the TA when they arrive while the TA is sleeping.
- **Semaphore chairs** (initial value 3) – controls access to hallway chairs.
- **Mutex help_lock** – protects the critical section where the TA assists a student.

High-level algorithm:

1. **Student thread:**
   - If chairs > 0, decrement chairs (sit).
   - If TA is sleeping, signal(ta_sleep).
   - Wait on help_lock to get help.
   - After help, signal(chairs) to free a chair.
2. **TA thread:**
   - Loop: wait(ta_sleep) (blocks when no students).
   - Acquire help_lock to assist the next waiting student.
   - Release help_lock after help; if no waiting students, go back to waiting on ta_sleep.

- This design guarantees **mutual exclusion** during help, **bounded waiting** for chairs, and wakes the TA only when needed.

🍴 Dining Philosophers with Monitors

Monitor implementation (Mesa style):

```
monitor Dining {
    enum { THINKING, HUNGRY, EATING } state[5];
    condition self[5];

    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING) wait(self[i]);
    }
```

```
    void putdown(int i) {
        state[i] = THINKING;
        test((i+4)%5);   // left neighbor
        test((i+1)%5);   // right neighbor
    }

    void test(int i) {
        if (state[i] == HUNGRY &&
            state[(i+4)%5] != EATING &&
            state[(i+1)%5] != EATING) {
            state[i] = EATING;
            signal(self[i]);   // wake if waiting
        }
    }
}
```

- Guarantees **mutual exclusion** of adjacent chopsticks, avoids deadlock because a philosopher never holds a single chopstick while waiting.

🧵 Pthreads Condition Variable Pattern

> **Condition variable:** used with a mutex to block a thread until a predicate becomes true.

Typical usage pattern (C/Pthreads):

```
pthread_mutex_lock(&mtx);
while (!predicate)              // re-check after wake-up
    pthread_cond_wait(&cv, &mtx);
... // critical section where predicate holds
pthread_mutex_unlock(&mtx);
```

- **Signaling:**

  ```
  pthread_mutex_lock(&mtx);
  predicate = true;
  pthread_cond_signal(&cv);   // or pthread_cond_broadcast
  pthread_mutex_unlock(&mtx);
  ```

The loop guards against **spurious wake-ups** and ensures the condition is still satisfied when the thread proceeds.

📦 Producer-Consumer with Semaphores & Mutex

Resources:

- sem empty – counts empty slots (initial = BUFFER_SIZE).
- sem full – counts filled slots (initial = 0).
- pthread_mutex_t buf_lock – protects buffer indices.

Producer thread:

```
wait(empty);
pthread_mutex_lock(&buf_lock);
insert_item(item);
pthread_mutex_unlock(&buf_lock);
signal(full);
```

Consumer thread:

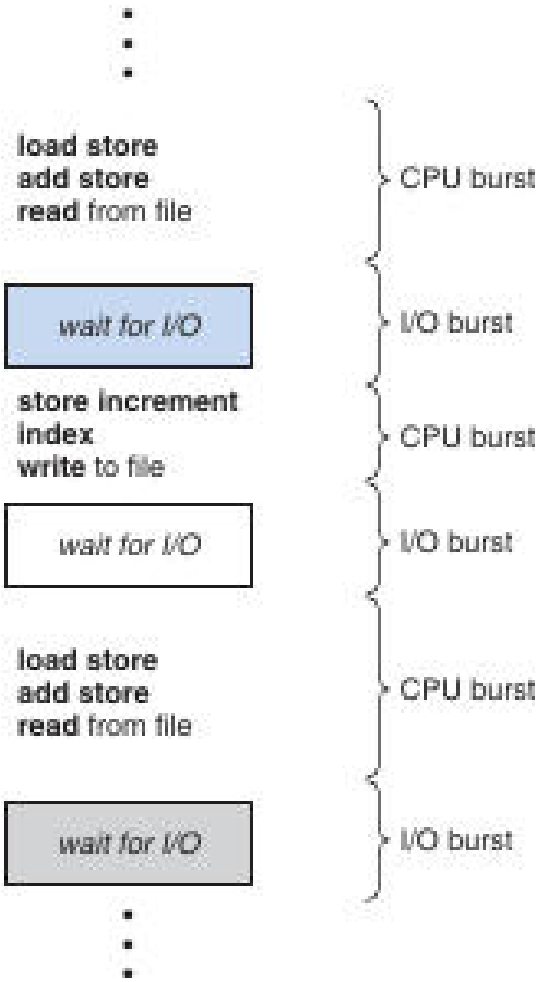
```
wait(full);
pthread_mutex_lock(&buf_lock);
remove_item(&item);
pthread_mutex_unlock(&buf_lock);
signal(empty);
```

- The **semaphores** enforce the bounded-buffer capacity, while the **mutex** prevents concurrent modifications of the circular-queue indices.

CPU Scheduling Overview

> **Definition:** *CPU scheduling* is the operating-system mechanism that selects which of the ready processes will run on the CPU and for how long. It is the core of multiprogrammed system design, aiming to keep the processor busy while processes alternate between computation and I/O.

Multiprogramming keeps several processes resident in memory so that when one process blocks for I/O, another can use the CPU. The goal is to **maximize CPU utilization** and reduce idle time.



The diagram shows a sequence of CPU bursts (black text) and I/O bursts (blue/grey rectangles), demonstrating the alternating execution pattern of a typical process.

🧩 Basic Concepts

- **CPU-I/O burst cycle** – A process executes a *CPU burst*, then performs an *I/O operation* (the I/O burst), and repeats until termination.
- **CPU-bound vs. I/O-bound** –
  - *CPU-bound* processes have many long CPU bursts and few I/O operations.
  - *I/O-bound* processes have short CPU bursts and frequent I/O, causing the CPU to be idle often.
- Measured CPU-burst durations typically follow an **exponential or hyper-exponential** distribution: many short bursts, few long ones.

📋 CPU Scheduler Components

Component	Role
Short-term scheduler	Chooses a process from the *ready queue* for the next CPU allocation.
Dispatcher	Transfers control to the selected process: performs a context switch, switches to user mode, and jumps to the saved program counter.
Dispatch latency	Time taken by the dispatcher to stop one process and start another; should be as small as possible because it occurs on every scheduling event.

⏱️ Scheduling Criteria

Criterion	What it measures	Desired direction
CPU utilization	Percentage of time the CPU is doing useful work.	↑ (approach 100 % on a loaded system)
Throughput	Number of processes completed per unit time.	↑
Turnaround time	Time from process submission to completion (including waiting, execution, and I/O).	↓
Waiting time	Total time a process spends in the ready queue.	↓
Response time	Time from request submission to first output (important for interactive systems).	↓
Variance of response time	Predictability of response; lower variance is preferred for interactive workloads.	↓

🧩 Preemptive vs. Non-preemptive Scheduling

- **Preemptive scheduling** allows the operating system to interrupt a running process (e.g., on timer expiration or higher-priority arrival) and assign the CPU to another process.
  - Used by modern Windows (post-95), macOS, and most UNIX variants.
  - Can introduce **race conditions** because a process may be preempted while modifying shared data.
- **Non-preemptive (cooperative) scheduling** lets a process retain the CPU until it voluntarily yields (by terminating or requesting I/O).
  - Historically used in early Windows (3.x) and classic Mac OS.
  - Simpler but unsuitable for time-sharing because a misbehaving process can monopolize the CPU.

When preemption occurs

1. A running process moves to **waiting** (e.g., initiates I/O).

2. A running process is **preempted** by an interrupt and placed back in the ready queue.
3. A waiting process becomes **ready** (I/O completes).
4. A process **terminates**.

If the scheduler makes decisions only in cases 1 and 3, the scheme is **non-preemptive**; otherwise it is **preemptive**.

🏁 Scheduling Algorithms

🕑 First-Come, First-Served (FCFS)

> **Definition:** The *FCFS* algorithm assigns the CPU to the process that has been waiting the longest, using a simple FIFO queue.

- **Implementation:** Insert a PCB at the tail of the ready queue when a process becomes ready; dispatch the head of the queue when the CPU is free.
- **Pros:** Simple to implement; fair in the sense of arrival order.
- **Cons:** Can yield long **average waiting times** (convoy effect) and performs poorly for time-sharing systems.
- **Non-preemptive:** Once a process gets the CPU, it runs until it blocks or terminates.

Example:
Processes P1 = 24 ms, P2 = 3 ms, P3 = 3 ms arrive at time 0.
FCFS order → P1, P2, P3 → average waiting time = (0 + 24 + 27) / 3 = 17 ms.
Reordering to P2, P3, P1 reduces the average to 3 ms, illustrating the impact of burst length ordering.

⏱️ Shortest-Job-First (SJF)

> **Definition:** *SJF* selects the process with the **shortest next CPU burst** for execution.

- **Optimality:** Proven to minimize average waiting time for a given set of processes.
- **Non-preemptive SJF** runs the chosen process to completion of its current burst.
- **Preemptive SJF** (also called *Shortest-Remaining-Time-First*, SRTF) preempts the running process if a newly arrived process has a shorter remaining burst.

Burst-time prediction

Because the exact length of the next burst is unknown, the OS often predicts it using exponential averaging:

$ \tau_{n+1}= \alpha , t_n + (1-\alpha) , \tau_n $

- (t_n) = actual length of the nth burst
- (\tau_{n+1}) = predicted length of the next burst
- (\alpha) ($0 \le \alpha \le 1$) controls the weighting of recent vs. older history (common choice $\alpha = ½$).

Example: Processes P1 = 8 ms, P2 = 4 ms, P3 = 9 ms, P4 = 5 ms arriving at times 0, 1, 2, 3 respectively.
Preemptive SJF produces a Gantt chart where P1 runs first, then is preempted by P2, etc., yielding an average waiting time of 6.5 ms, better than the non-preemptive case.

⚡ Priority Scheduling

> **Definition:** *Priority scheduling* assigns the CPU to the ready process with the **highest priority** (numerically lowest in this text).

- **Relation to SJF:** When priority is defined as the inverse of the predicted next CPU burst, SJF becomes a special case of priority scheduling.
- **Tie-breaking:** Processes with equal priority are scheduled in FCFS order.
- **Priority ranges:** Often 0 – 15 or 0 – 4095; here **low numbers denote high priority**.

Example:

Process	Burst (ms)	Priority

P1	10	3
P2	1	0
P3	2	4
P4	1	1
P5	5	2

Scheduling by priority → P2, P4, P5, P1, P3 → average waiting time ≈ 8.2 ms.

- **Internal vs. external priorities** – Internal priorities are computed from process characteristics (e.g., CPU-burst length, I/O-burst ratio). External priorities are set by the OS or administrator (e.g., user class, payment level).
- **Aging** – To avoid indefinite postponement of low-priority jobs, a process's priority can be gradually increased the longer it waits, guaranteeing eventual execution.

🔁 Round-Robin (RR)

> **Definition:** *Round-Robin* is a preemptive time-sharing algorithm that gives each process a fixed **time quantum** (or time slice) in a cyclic order.

- **Ready queue** is treated as a circular FIFO.
- After a process consumes its quantum, the scheduler **preempts** it, places it at the tail, and dispatches the next process.
- **Quantum size** (typically 10–100 ms) strongly influences performance:
  - **Large quantum** → behavior approaches FCFS, fewer context switches, but longer response times.
  - **Small quantum** → many context switches, higher overhead, but better responsiveness.

Example: Quantum = 4 ms, processes: P1 = 24 ms, P2 = 3 ms, P3 = 3 ms (all arrive at 0).
RR schedule → P1(0-4), P2(4-7), P3(7-10), P1(10-14), … resulting in an average waiting time of ≈ 5.66 ms.

🛠️ Dispatcher Details

> **Definition:** The *dispatcher* is the OS module that gives control of the CPU to the process selected by the short-term scheduler.

Steps performed by the dispatcher:

1. **Context switch** – Save the state of the currently running process and load the state of the chosen process.
2. **Switch to user mode** – Change CPU privilege level.
3. **Jump to the saved program counter** – Resume execution at the point where the process previously stopped.

The **dispatch latency** is the total time taken for these steps; minimizing it is critical because the dispatcher is invoked on every scheduling event.

📉 Real-World Considerations

- **CPU-bound vs. I/O-bound mix:** A single long CPU-bound process can dominate the CPU, causing I/O devices to sit idle while many short I/O-bound processes wait. Algorithms that favor short jobs (e.g., SJF, RR with small quantum) improve overall utilization.
- **Starvation:** Priority and SJF can starve low-priority or long-burst processes; **aging** mitigates this by gradually raising waiting processes' priorities.
- **Response-time variance:** For interactive systems, predictable response time (low variance) may be more important than minimizing the average. Few scheduling algorithms explicitly target variance, leaving it an open research area.

⏱️ Time Quantum & Context-Switch Overhead

> **Time quantum** – the maximum amount of CPU time a process may run before the scheduler preempts it and performs a context switch.

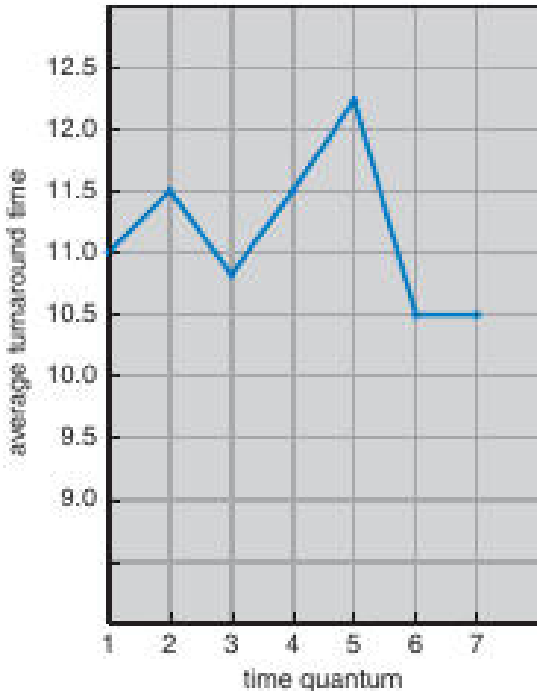
- A context switch costs roughly **10 %** of a quantum when the quantum is small; the larger the quantum, the smaller the fraction of CPU time spent on switching.
- **Average turnaround time** improves when most CPU bursts fit within a single quantum.

Example – three processes each require a CPU burst of 10 time units:

Quantum	Avg. turnaround (no context-switch cost)
1	29
10	20

When the overhead of a context switch (≈ 10 % of the quantum) is added, smaller quanta suffer larger increases in turnaround because more switches are required.

Guideline – aim for a quantum such that **≈ 80 %** of CPU bursts are shorter than the quantum. Modern systems typically use quanta of **10–100 ms**; a context switch takes about **10 µs**, a negligible fraction of the quantum.



The blue line shows how average turnaround time varies as the time quantum increases from 1 to 7. Turnaround drops quickly at first, then stabilizes, illustrating the diminishing returns of a very large quantum.

📊 Multilevel Queue Scheduling

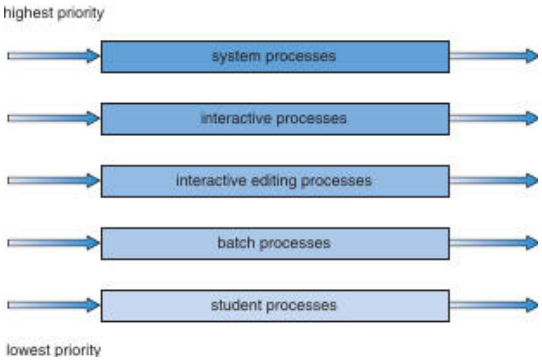
> **Multilevel queue** – partitions the ready queue into several *permanent* queues, each associated with a distinct class of processes (e.g., foreground vs. background).

- **Queue assignment** is based on a static property such as **process type**, **priority**, or **size**.
- Each queue can employ a different scheduling algorithm (e.g., RR for interactive, FCFS for batch).
- **Inter-queue scheduling** is usually **fixed-priority preemptive**: a higher-priority queue can preempt any lower-priority queue.

Typical hierarchy (high → low priority)

Priority	Queue content

1	System processes
2	Interactive editing processes
3	Interactive processes
4	Batch processes
5	Student processes



Bar chart showing the five priority classes from highest (system) to lowest (student).

Scheduling nuances

- A **foreground** queue may receive **80%** of CPU time (RR among its processes), while the **background** queue gets the remaining **20%** (FCFS).
- Lower-priority queues run only when all higher queues are empty; no process ever moves between queues, which yields low overhead but limited flexibility.

🔁 Multilevel Feedback Queue (MLFQ)

> **MLFQ** – a dynamic variant of multilevel queue where processes can **move between queues** based on their observed CPU-burst behavior.

- Short, I/O-bound bursts stay in **higher-priority queues**; long, CPU-bound bursts are **demoted** to lower queues.
- **Aging**: a process that waits too long in a low-priority queue is eventually **promoted** to avoid starvation.

Illustrative three-queue example

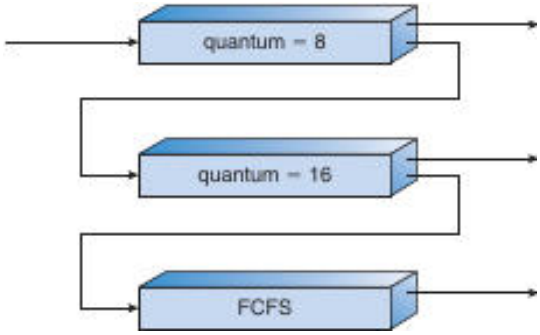
Queue	Time quantum	Promotion/Demotion rule
0	8 ms	If a process finishes ≤ 8 ms → stays; otherwise → move to tail of queue 1
1	16 ms	Same rule; if not finished → move to queue 2
2	FCFS (no quantum)	Runs only when queues 0 and 1 are empty

A process always enters at **queue 0**. The scheduler always selects the **highest-priority non-empty queue**. This scheme gives priority to any burst ≤ 8 ms, allowing quick I/O-bound jobs to finish promptly while longer jobs gradually sink to queue 2.

Key parameters for an MLFQ scheduler

- Number of queues.
- Scheduling algorithm used in each queue.
- Criteria for **promotion** (e.g., after waiting *t* milliseconds).
- Criteria for **demotion** (e.g., after exhausting its quantum).

- Method for **initial queue selection** (often the highest-priority queue).



Flowchart contrasts three policies: quantum = 8 ms, quantum = 16 ms, and FCFS.

🧵 Thread Scheduling & Contention Scope

> **Thread scheduling** determines which runnable thread receives a CPU core at any moment.

Contention Scope

Scope	Meaning
Process-Contention Scope (PCS)	Threads compete *only* with other threads of the **same process**. Typically used in many-to-many models where a user-level library maps threads onto a pool of lightweight processes (LWPs).
System-Contention Scope (SCS)	Threads compete with **all** threads in the system; the kernel schedules them directly (one-to-one model).

- In **PCS**, the library may implement its own scheduler; the kernel sees only the underlying LWPs.
- In **SCS**, the kernel's scheduler decides, usually based on thread priority.

Pthread Scheduling API

```
/* Pthread scheduling example – shows contention-scope handling */
#include
#define NUM_THREADS 5

int main(int argc, char *argv[]) {
    int i, scope;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;

    pthread_attr_init(&attr);
    if (pthread_attr_getscope(&attr, &scope) != 0)
        fprintf(stderr, "Unable to get scheduling scope\n");
    else if (scope == PTHREAD_SCOPE_PROCESS)
        printf("PTHREAD_SCOPE_PROCESS\n");
    else if (scope == PTHREAD_SCOPE_SYSTEM)
        printf("PTHREAD_SCOPE_SYSTEM\n");
    else
        fprintf(stderr, "Illegal scope value.\n");

    /* Force system-scope (SCS) */
    pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);

    for (i = 0; i < NUM_THREADS; i++)
```

```
        pthread_create(&tid[i], &attr, runner, NULL);

    for (i = 0; i < NUM_THREADS; i++)
        pthread_join(tid[i], NULL);
    return 0;
}
```

- pthread_attr_getscope / pthread_attr_setscope query or set the contention scope.
- Many modern systems (e.g., Linux) support **only** PTHREAD_SCOPE_SYSTEM.

Multiprocessor Scheduling

Scheduling Architectures

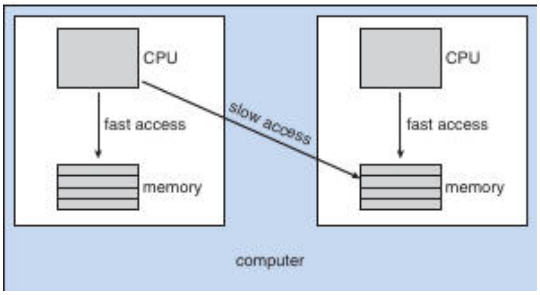
Architecture	Description
Asymmetric Multiprocessing (AMP)	One *master* processor runs the scheduler; other CPUs execute only user code. Simpler because only one processor updates scheduling data structures.
Symmetric Multiprocessing (SMP)	Every processor runs its own scheduler and may have a **private ready queue** or share a **global queue**. Requires synchronization to avoid double-scheduling.

Processor Affinity

> **Processor affinity** – the tendency to keep a thread on the same CPU core to preserve cache locality.

- **Soft affinity**: OS *tries* to keep a thread on its previous core but may migrate it when load-balancing.
- **Hard affinity**: Application explicitly restricts a thread to a set of CPUs (e.g., sched_setaffinity on Linux).

NUMA considerations – In a *non-uniform memory access* system, a CPU accesses its local memory faster than remote memory. Aligning a thread's affinity with memory placement improves performance.



Left side: fast local memory access; right side: slower remote memory access. Affinity helps keep a thread near its local memory.

Load Balancing

Two complementary strategies:

- **Push migration** – a busy processor periodically *pushes* tasks to less-loaded CPUs.
- **Pull migration** – an idle processor *pulls* tasks from a busy counterpart.

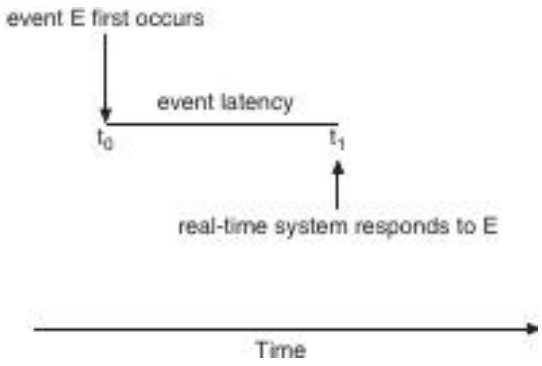
Both approaches appear in modern kernels (e.g., Linux, FreeBSD). Load balancing can conflict with affinity because moving a thread discards its cached data, so systems often employ thresholds to decide when migration is worthwhile.

🧩 Multicore & Multithreading

Memory Stalls

> **Memory stall** – a period during which the processor idles while waiting for data from memory (e.g., after a cache miss).

- Stalls can consume **up to 50%** of execution time on memory-bound workloads.



The diagram shows a processor spending a large fraction of its cycle waiting for data.

Hardware Multithreading

- **Coarse-grained**: a thread runs until it encounters a long-latency event (e.g., cache miss), then the core switches to another thread. Switching cost is relatively high.
- **Fine-grained**: the core interleaves instructions from multiple threads every cycle; the hardware incurs a small switching overhead.

Modern chips expose multiple **logical processors**:

- Dual-threaded cores → each core appears as two logical CPUs.
- Example: **UltraSPARC T3** – 16 cores ×8 hardware threads = **128 logical processors**.

The OS schedules software threads onto these logical processors using any of the algorithms discussed earlier (RR, priority, etc.). The hardware then selects which *hardware thread* to run (e.g., simple round-robin on each core, or urgency-based selection as on the Itanium).

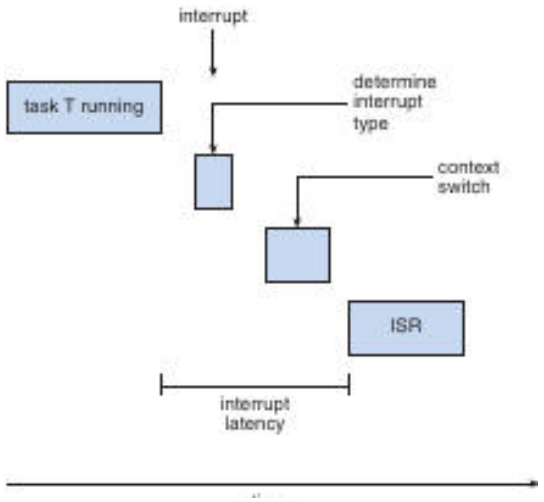
⏱️ Real-Time CPU Scheduling

Soft vs. Hard Real-Time

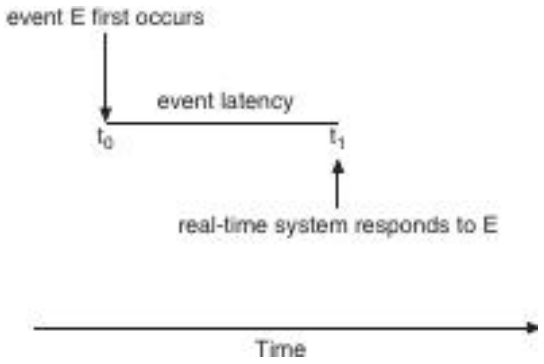
Category	Guarantees
Soft real-time	No strict deadline guarantee; tasks receive higher priority but may miss deadlines occasionally.
Hard real-time	**Deadline must be met**; missing it is considered a system failure.

Latency Types

1. **Interrupt latency** – time from an interrupt's arrival to the start of its **Interrupt Service Routine (ISR)**.
2. **Dispatch latency** – time from the moment a real-time task becomes ready to the moment it actually starts executing on the CPU.



The flowchart traces the steps from an interrupt occurring while a task runs, through context switching, to the ISR execution, highlighting interrupt latency.



Timeline shows an event occurring at (t_0) and the system's response at (t_1); the interval is the event latency.

Minimizing Latency

- **Preemptive kernels** allow an urgent real-time task to interrupt a lower-priority task instantly, reducing dispatch latency.
- Keep the period during which interrupts are disabled **as short as possible**; lengthy critical sections increase both interrupt and dispatch latency.
- Use **priority inheritance** (or similar mechanisms) to avoid priority inversion that can inflate latency.

⏱️ Dispatch Latency Diagram

> *Dispatch latency* is the total time from when a process becomes ready to when it actually starts executing on the CPU.

The latency consists of two phases:

1. **Preemption of the currently running kernel process** – the scheduler must interrupt the kernel task and save its state.
2. **Release of resources held by low-priority processes** – any locks, memory pages, or I/O buffers required by the newly ready real-time process must be freed.

> In Solaris, enabling preemption reduces dispatch latency from **> 100 ms** to **< 1 ms**, illustrating the importance of the preemption phase for hard real-time responsiveness.

⚙️ Priority-Based Scheduling

> *Priority-based scheduling* assigns each process a numerical priority; higher-priority processes are chosen before lower-priority ones.

- When **preemption** is supported, a running lower-priority process is immediately displaced if a higher-priority process becomes ready.
- Real-time operating systems typically reserve the highest priority levels for real-time tasks (e.g., Windows reserves levels 16–31).
- Preemptive, priority-based schedulers guarantee **soft real-time** behavior; **hard real-time** guarantees require additional mechanisms (e.g., admission control, deadline enforcement).

📅 Periodic Real-Time Processes

> A *periodic* real-time process repeats every fixed interval **p** and must finish its CPU burst **t** before its deadline **d** (where (0 \le t \le d \le p)).

Key parameters:

- **Processing time** (t) – CPU time needed each period.
- **Period** (p) – interval between successive releases.
- **Deadline** (d) – latest time the burst must complete (often (d = p)).

These attributes enable the scheduler to reason about feasibility and assign priorities based on rate or deadline.

🕒 Rate-Monotonic Scheduling (RMS)

> *RMS* is a static-priority algorithm that gives a higher priority to the task with the **shorter period**.

How RMS Works

1. Upon admission, each periodic task receives a priority inversely proportional to its period.
2. The scheduler always runs the highest-priority ready task; lower-priority tasks are preempted as soon as a higher-priority task arrives.

Utilization Test

For **N** periodic tasks, RMS guarantees that all deadlines are met if

$ U = \sum_{i=1}^{N}\frac{t_i}{p_i} \le N\left(2^{1/N}-1\right) $

- As (N \to \infty), the bound approaches **≈ 69 %**.
- With two tasks, the bound is **≈ 83 %**; with three tasks, **≈ 78 %**, etc.

Example

Task	(p) (ms)	(t) (ms)	(t/p)
P1	50	20	0.40
P2	100	35	0.35

Total utilization (U = 0.75) < 83 % → RMS can schedule them.
Assigning P1 the higher priority (shorter period) yields the schedule shown in Figure 6.17, where both deadlines are met.

When the utilization exceeds the bound (e.g., Figure 6.18 with (U ≈ 0.94)), RMS cannot guarantee deadline satisfaction; P1 may miss its deadline despite the CPU being less than fully loaded.

📆 Earliest-Deadline-First (EDF) Scheduling

> *EDF* is a dynamic-priority algorithm that always selects the ready task whose **absolute deadline** is soonest.

- Priorities are recomputed whenever a task becomes runnable.
- Unlike RMS, EDF does **not** require fixed periods; any task that announces its deadline can be scheduled.

Optimality

- EDF can achieve **100 % CPU utilization** in theory (all deadlines met if (U \le 1)).
- In practice, context-switch overhead and interrupt handling reduce achievable utilization.

Example (same tasks as RMS)

- Initially P1 (deadline 50) runs, then P2 (deadline 80) runs.
- When P2's deadline becomes earlier than P1's next deadline, EDF preempts P1, guaranteeing both tasks meet their deadlines (see Figure 6.19).

📊 Proportional-Share Scheduling

> *Proportional-share* allocates CPU time proportionally to **shares** requested by each client.

- Total shares **T** are divided among applications; an application receiving **S** shares gets (S/T) of the processor.
- An **admission-control** policy ensures that the sum of granted shares never exceeds **T**.

Example

- (T = 100) shares.
- A receives 50 shares → 50 % CPU, B receives 15 % → 15 % CPU, C receives 20 % → 20 % CPU.
- A new request for 30 shares (process D) would be denied because only 15 shares remain.

🛠️ POSIX Real-Time Scheduling (SCHED_FIFO & SCHED_RR)

> POSIX.1b defines two real-time scheduling classes for threads:

Class	Policy	Characteristics
SCHED_FIFO	First-In-First-Out	No time slicing; threads run to completion or block.
SCHED_RR	Round-Robin	Time-sliced among threads of equal priority (similar to traditional RR).
SCHED_OTHER	Implementation-defined (usually the default time-sharing class).	Behavior varies across OSes.

Key API functions:

- pthread_attr_getschedpolicy(pthread_attr_t *attr, int *policy) – retrieve current policy.
- pthread_attr_setschedpolicy(pthread_attr_t *attr, int policy) – set desired policy (e.g., SCHED_FIFO).

Error handling: both functions return non-zero on failure.

🐧 Linux Scheduling Evolution

O(1) Scheduler (pre-2.6)

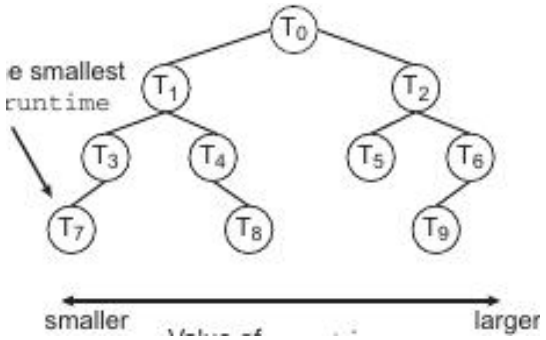
- Ran in **constant time** regardless of task count.
- Provided good SMP load balancing but exhibited poor interactive response.

Completely Fair Scheduler (CFS) – default since 2.6.23

- **Scheduling classes**: a default CFS class and a real-time class.
- **Nice values** (-20 \dots +19) map to relative priority; lower nice → higher share of CPU.
- **Virtual runtime (vruntime)** tracks how much "fair" CPU time each task has received; the task with the **smallest vruntime** is chosen next.

Red-Black Tree Structure

The runnable tasks are stored in a balanced red-black tree keyed by vruntime. The leftmost node holds the task with the smallest vruntime (i.e., highest priority).



The diagram shows nodes (T_0)–(T_9) ordered from smallest (left) to largest (right) vruntime. The leftmost node is the next task to run, enabling O(log N) selection, cached as rb_leftmost for O(1) access.

- **I/O-bound vs. CPU-bound**: I/O-bound tasks accumulate less vruntime and are favored, leading to natural preemption of CPU-bound tasks when I/O becomes ready.
- Real-time threads (SCHED_FIFO / SCHED_RR) run at higher priority than normal CFS tasks.

Windows Scheduling Overview

- Uses a **preemptive, priority-based** scheduler with **32 priority levels** (1–15 for normal, 16–31 for real-time; 0 reserved for the system).
- **Dispatcher** traverses priority queues from highest to lowest, selecting the first ready thread. If none are ready, the idle thread runs.

Priority Classes & Relative Priorities

Priority Class	Range	Typical Use
REALTIME	16-31	Critical system or multimedia tasks.
HIGH	8-13	Time-sensitive user processes.
NORMAL	4-7	Standard interactive applications.
IDLE	1-3	Background maintenance.

- Within a class, threads have a **relative priority** (e.g., THREAD_PRIORITY_NORMAL).
- Windows boosts the priority of the **foreground process** (typically ×3) to improve responsiveness.
- **Time quantum** expiration, I/O completion, or higher-priority arrival cause preemption.

User-Mode Scheduling (UMS)

- Introduced in Windows 7, allowing applications to manage their own threads without kernel intervention, reducing overhead for massive thread counts.
- Fibers (pre-Windows 7) lacked independent thread contexts; UMS provides full context for each user-mode thread.

Solaris Scheduling Highlights

- Employs **priority-based** thread scheduling similar to other UNIX-like systems.

- Real-time threads receive the highest static priorities; normal threads share a lower range.
- Supports **multiprocessor** environments with load-balancing and affinity mechanisms.

📈 Comparative Snapshot of Real-Time Scheduling Strategies

Strategy	Priority Assignment	Preemptive?	Utilization Bound	Typical Use
Rate-Monotonic (RMS)	Fixed, shorter period → higher priority	Yes	≤ (N(2^{1/N}-1)) (≈ 69 % → 83 % for few tasks)	Hard real-time systems with periodic workloads
Earliest-Deadline-First (EDF)	Dynamic, earliest absolute deadline → higher priority	Yes	≤ 1 (theoretically 100 %)	Soft real-time and mixed workloads
Proportional-Share	Shares → CPU proportion	Usually non-preemptive but can be combined with RR	≤ 1 (subject to overhead)	Fair resource allocation among competing applications
POSIX SCHED_FIFO / SCHED_RR	Static priority levels	Yes (FIFO: no slicing; RR: time-slicing)	No explicit bound	Real-time threads on POSIX-compatible OSes
Linux CFS	Nice-based virtual runtime	Yes (preemptive)	No hard bound; strives for fairness	General-purpose Linux systems
Windows Priority-Based	32-level numeric priority + class boost	Yes	No explicit bound	Desktop and server Windows environments

---

🛡️ Ensuring Hard Real-Time Guarantees

- **Admission control**: before admitting a task, the scheduler verifies that the cumulative utilization satisfies the algorithm's bound (e.g., RMS bound).
- **Deadline enforcement**: tasks that miss deadlines trigger corrective actions (e.g., task abort, mode switch).
- **Resource reservation**: CPU time, memory, and I/O bandwidth may be statically allocated to guarantee availability.

🗂️ Scheduling Classes Overview

> **Definition:** A *scheduling class* groups threads that share a common priority scheme and scheduling policy.

Solaris defines six classes:

Class	Abbreviation	Typical Use	Priority Behaviour
Time-Sharing	TS	General interactive and batch processes	Dynamic priorities; higher priority → shorter time quantum
Interactive	IA	GUI and window-manager tasks (e.g., KDE)	Treated as a special TS subset with higher base priority
Real-Time	RT	Hard-real-time threads requiring bounded response	Highest absolute priority; never pre-empted by lower classes
System	SYS	Kernel threads (paging daemon, interrupt handlers)	Fixed priorities; not altered by the scheduler
Fixed-Priority	FP	Legacy or specialized workloads that need static ordering	Priorities are static within the class
Fair-Share	FSS	Projects or groups of processes that receive a proportion of CPU	Priorities derived from assigned CPU shares

The **default** class for a newly created process is **Time-Sharing**.

⏱️ Time-Sharing & Interactive Scheduling

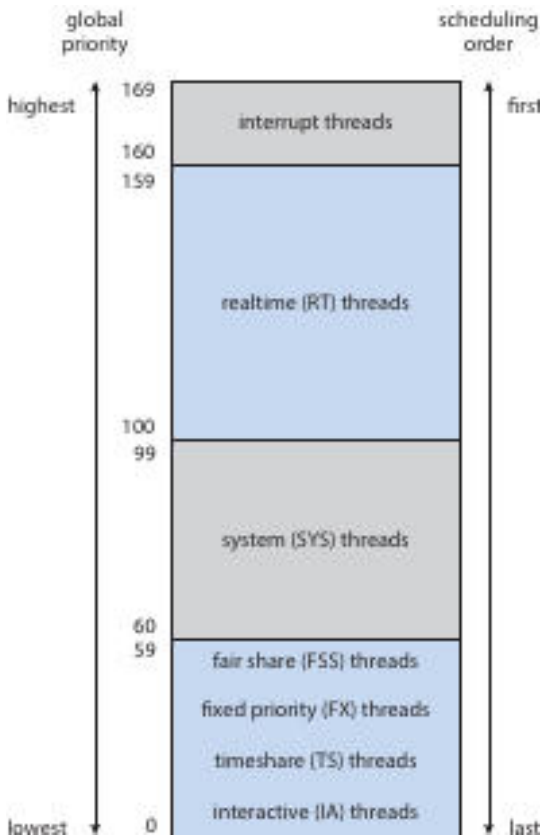
> **Definition:** The *time-sharing* class uses a **multilevel feedback queue (MLFQ)** that changes both a thread's priority and its time-slice length while it runs.

- **Priority ↔ Time Quantum:**
  - Higher numeric priority → **smaller** time quantum.
  - Lower numeric priority → **larger** time quantum.
- **Interactive processes** (e.g., window managers) receive higher base priorities, giving them better response and throughput.
- The **dispatch table** (Solaris) maps each priority to a specific quantum; lower-priority entries have larger quanta (up to 200 ms), while the highest priority (59) gets a 20 ms slice.

Solaris Dispatch Table (excerpt)

Priority	Time Quantum (ms)
0	200
5	160
10	160
15	120
20	120
25	80
30	80
35	40
40	40
45	40
50	40
55	40
59	20

- **Quantum Expired:** The thread's priority is *lowered* (moves to a lower-priority queue).
- **Return from Sleep / I/O:** The thread's priority is *boosted* (moved toward the higher-priority end) to improve response time for I/O-bound work.



The table shows priority numbers (0–59) on the left and the associated time-slice lengths on the right. Higher priorities have shorter slices, illustrating the inverse relationship discussed above.

🚀 Real-Time, System, Fixed-Priority, and Fair-Share Classes

- **Real-Time (RT):** Threads run before any other class. A real-time thread is guaranteed to execute for a **bounded period** once scheduled.
- **System (SYS):** Reserved for kernel-mode threads; priorities are set once at creation and never change.
- **Fixed-Priority (FP):** Behaves like the time-sharing class but **does not** adjust priorities dynamically; useful for legacy applications.
- **Fair-Share (FSS):** Instead of numeric priorities, each *project* receives a **CPU share** (e.g., 20 % of the processor). The scheduler converts these shares into **global priorities** that compete with other classes. Threads with equal global priority are placed in a **round-robin** queue.

🧩 Global Priority Mapping

> **Definition:** *Global priority* is a single numeric scale that the dispatcher uses to choose the next runnable thread, regardless of its original class.

- Each class-specific priority is **translated** into a global value.
- The highest global priorities (169 – 160) belong to **interrupt threads**, which always run first.
- The ordering from highest to lowest global priority is:

  Interrupt → Real-Time → System → Fair-Share → Fixed-Priority → Time-Sharing → Interactive

- Solaris switched from a **many-to-many** thread model to a **one-to-one** model beginning with version 9, simplifying the mapping and improving predictability.

📊 CPU Scheduling Algorithm Evaluation

Selecting a scheduling algorithm requires balancing several **criteria** (response time, throughput, CPU utilization, etc.) and applying an appropriate **evaluation method**.

📈 Deterministic Modeling

- Uses a **fixed workload** with known arrival times and burst lengths.
- Provides **exact** results for the chosen scenario, allowing direct comparison of algorithms.

Example: Five processes arrive at time 0 with bursts $\{10, 29, 3, 7, 12\}$.

Algorithm	Average Waiting Time (ms)
FCFS	28
SJF (non-preemptive)	13
RR (quantum = 10)	23

- Deterministic modeling shows SJF yields the lowest waiting time for this workload, but conclusions apply only to the specific input set.

📉 Queueing Models

- Treat the CPU as a **service server** and processes as customers.
- Two key stochastic inputs:
  1. **Arrival-time distribution** (often exponential).
  2. **Service-time distribution** (CPU-burst lengths).
- **Little's Law:** (n = \lambda \times W)
  - (n): average number of jobs in the queue.
  - (\lambda): average arrival rate (jobs/second).
  - (W): average waiting time.

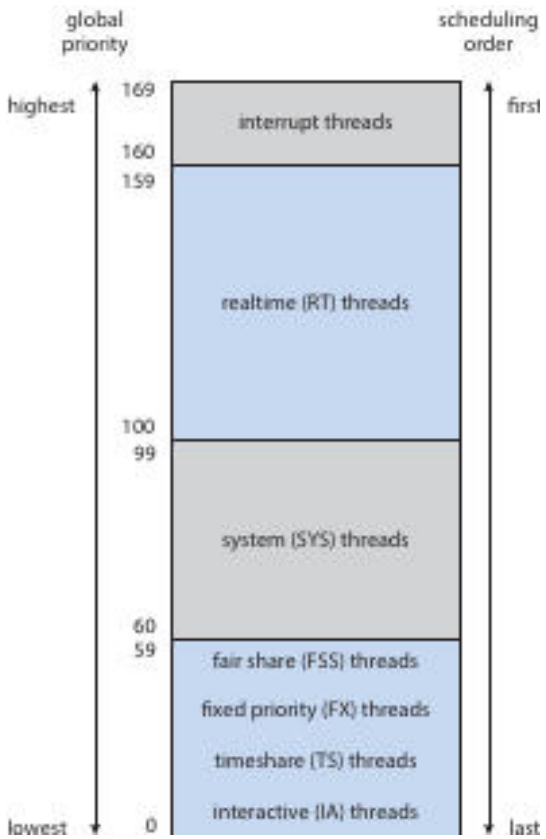
  If (\lambda = 7) jobs/s and (n = 14), then (W = 2) s.
- Queueing analysis yields **utilization**, **average queue length**, and **response time**, but it is limited to simple distributions and may not capture complex system behaviour.

🎲 Simulations

- Build a **software model** of the OS components (CPU, I/O devices, scheduler).
- The simulator advances a **virtual clock**, processes events, and collects performance statistics.

Two approaches:

1. **Distribution-driven simulation:** Randomly generate arrivals and bursts from chosen probability distributions (uniform, exponential, Poisson).
2. **Trace-tape simulation:** Record the exact sequence of events from a real system and replay them, guaranteeing identical input for each algorithm.



The diagram illustrates how a trace tape captures real events, which are then fed to a simulator to compare multiple scheduling policies on identical workloads.

- Simulations are more realistic than deterministic models but require **significant coding effort** and **CPU time**; trace tapes can be large.

🛠️ Implementation Evaluation (Live-System Testing)

- The **most accurate** method is to integrate the new scheduler into the operating system and run real workloads.
- Drawbacks: high development cost, potential disruption to users, and the need to maintain a stable production environment.

📚 Summary of Major Scheduling Algorithms (reference to earlier sections)

Algorithm	Preemptive?	Typical Use	Key Parameter
FCFS	No	Simple batch workloads	–
SJF (optimal for average waiting)	Optional	CPU-bound jobs where burst prediction is possible	Burst-time estimate
Priority	Optional	Mix of interactive and background tasks	Numeric priority (lower = higher)
RR	Yes	Time-shared interactive systems	Time quantum
Multilevel Queue	Yes (within each queue)	Separate classes (e.g., interactive RR, batch FCFS)	Queue-specific policy
MLFQ	Yes	Adaptive systems that reward I/O-bound behaviour	Number of queues, promotion/demotion rules

Rate-Monotonic (RM)	Yes	Hard real-time periodic tasks	Period (shorter → higher priority)
Earliest-Deadline-First (EDF)	Yes	Soft/hard real-time tasks with explicit deadlines	Absolute deadline
Proportional-Share (FSS)	Yes	Projects needing guaranteed CPU fractions	Share weight

- **Aging** can be added to any priority-based scheme to prevent starvation.
- On multiprocessor systems, each CPU may maintain its own **run queue** (per-processor) or share a **global queue**; the former reduces contention, the latter simplifies load balancing.
- **Processor affinity** and **load-balancing** policies influence where threads execute, especially in NUMA environments.

📚 Practice Exercise Highlights (selected concepts)

- **Deterministic example** (FCFS vs. SJF vs. RR) shows how average waiting time varies with algorithm choice.
- **Little's Law** provides a quick way to compute one queue metric when the other two are known.
- **MLFQ advantage:** Different time-quantum lengths per level give I/O-bound threads higher priority without explicit classification.
- **Real-time priority inversion** is avoided by the RT class's absolute priority and, if needed, by **priority inheritance** mechanisms.

These notes capture the essential mechanisms, evaluation techniques, and comparative properties of the scheduling classes and algorithms covered in the source material.

🗂️ Multilevel Scheduling & Dynamic Priorities

> **Multilevel scheduling** partitions the ready queue into several sub-queues, each possibly using a different scheduling algorithm.
> A **preemptive priority scheduler** may adjust a process's priority while it is waiting or running.

Let

- α – rate of priority change while a process is **waiting** in the ready queue.
- β – rate of priority change while a process is **running** on the CPU.

All processes enter the ready queue with priority 0.

Condition on α, β	Resulting behaviour
$\alpha > \beta > 0$	Priority of a waiting process grows faster than it declines during execution → **priority-aging** (processes eventually rise to the top of the queue).
$\alpha < \beta < 0$	Both rates are negative, but the running decay is steeper → **priority-decay** (processes that have recently run keep lower priority, favouring newer arrivals).

These two extremes illustrate how a scheduler can be tuned to either prevent starvation (aging) or to penalise CPU-bound jobs (decay).

⚖️ Discrimination Against Short Processes

Algorithm	How it favours short jobs
FCFS (First-Come, First-Served)	No discrimination; short and long jobs are treated identically, often causing the *convoy effect*.
RR (Round-Robin)	Gives each job a fixed quantum; short jobs typically finish within one quantum, reducing their waiting time compared with FCFS.

Feedback queues (multilevel)	Jobs that exhaust their quantum are **demoted** to a lower-priority queue, while those that relinquish the CPU early stay higher. This naturally favours short, I/O-bound bursts.
Multilevel (static)	If high-priority queues use a short-quantum RR and lower queues use longer quanta, short jobs placed in the high queue receive faster service.

🪟 Windows Scheduling – Priority Classes & Relative Priorities

Windows defines **priority classes** (system-wide) and **relative priorities** (per-thread).
The effective priority is the sum of the class base and the relative offset.

Thread description	Effective priority (class + relative)
REALTIME class, **NORMAL** relative	Highest possible priority (real-time base + 0).
ABOVE NORMAL class, **HIGHEST** relative	Slightly lower than REALTIME but higher than any NORMAL class thread.
BELOW NORMAL class, **ABOVE NORMAL** relative	Falls between the NORMAL and ABOVE NORMAL classes.

When no thread belongs to the **REALTIME** class and **TIME-CRITICAL** is unavailable, the **ABOVE NORMAL** class with **HIGHEST** relative priority becomes the highest-priority runnable thread in the system.

☀️ Solaris Time-Sharing Scheduler Details

The Solaris time-sharing scheduler uses a multilevel feedback queue where priority and time-slice are linked.

Priority	Time quantum (ms)
15	120 ms
40	40 ms
59 (highest)	20 ms

- **If a thread exhausts its quantum** without blocking, the scheduler **lowers** its priority (moves it to a lower queue).
- **If a thread blocks for I/O** before its quantum expires (e.g., priority 20), the scheduler **boosts** its priority when it becomes ready again, reflecting the I/O-bound nature.

🐧 Linux Completely Fair Scheduler (CFS) & Nice Values

CFS tracks each runnable task with a **virtual runtime (vruntime)**; the task with the smallest vruntime runs next.

Nice value	Effect on vruntime growth
-5 (higher-priority)	vruntime advances **more slowly**, granting the task more actual CPU time.
+5 (lower-priority)	vruntime advances **faster**, reducing the task's share of the CPU.

Scenarios

1. **Both A (-5) and B (+5) are CPU-bound** – A's vruntime stays behind B's, so A receives a larger share of the processor.
2. **A is I/O-bound, B is CPU-bound** – When A blocks, its vruntime stops advancing; upon return it will have a very low vruntime, allowing it to run before B.
3. **A is CPU-bound, B is I/O-bound** – B repeatedly re-enters the run queue with a low vruntime, gaining frequent short bursts, while A runs only when B is blocked.

⏱️ Real-Time Priority Inversion & Proportional-Share Schedulers

- **Priority inversion** occurs when a low-priority task holds a resource needed by a high-priority task, and a medium-priority task pre-empts the low-priority holder.
- **Solution 1 – Priority inheritance:** The low-priority holder temporarily inherits the highest blocked priority, preventing the medium task from pre-empting it.
- **Solution 2 – Immediate resource pre-emption:** The scheduler can forcibly take the resource from the low-priority task (used rarely because of state-consistency concerns).

In a **proportional-share scheduler**, each client is assigned a *share* of the CPU.

- By granting the high-priority real-time task a larger share, its effective execution rate rises, mitigating inversion without explicit inheritance.
- However, if the low-priority task still holds a non-preemptible lock, inversion can persist; thus a combination of share-based weighting **and** priority inheritance is often employed.

🕛 When RMS & EDF Meet Their Deadlines

Condition	Rate-Monotonic Scheduling (RMS)	Earliest-Deadline-First (EDF)
Utilization ≤ 69 % (as $N \to \infty$)	Guarantees all deadlines are met.	Guarantees all deadlines are met **if** total utilization ≤ 100 %.
Small number of tasks (e.g., 2–3)	Bound is higher (≈ 83 % for 2 tasks, ≈ 78 % for 3).	Still only requires ≤ 100 % utilization.
Dynamic task sets	Not suitable – static priorities require known periods.	Suitable – priorities are recomputed each arrival.

Thus, **RMS** is appropriate for strictly static periodic workloads with low overall utilization, while **EDF** provides optimal feasibility for any set of periodic tasks whose total CPU demand does not exceed the processor capacity.

📊 Scheduling Example – Processes P_1 and P_2

- **P_1:** period $p_1 = 50$ ms, execution time $t_1 = 25$ ms.
- **P_2:** period $p_2 = 75$ ms, execution time $t_2 = 30$ ms.

a. Rate-Monotonic Scheduling (RMS)

Because $p_1 < p_2$, **P_1** receives the higher static priority.

```
0-25   : P1 runs (first instance)
25-55  : P2 runs (first instance, 30 ms)
50-75  : P1 runs (second instance, pre-empting P2 at 50 ms)
75-80  : P2 resumes (remaining 5 ms)
80-105 : P1 runs (third instance)
105-115: P2 runs (second instance)
```

The schedule meets all deadlines (P_1 completes by 50, 100, 150 ms; P_2 by 75, 150 ms).

b. Earliest-Deadline-First (EDF)

Deadlines are compared at each scheduling point.

```
0-25   : P1 runs (deadline 50)
25-55  : P2 runs (deadline 75)
55-80  : P1 runs (second instance, deadline 100) – pre-empted P2 at 55
80-105 : P2 runs (remaining 5 ms, deadline 75 missed → EDF would have scheduled P2 earlier, so the feasible E
```

A feasible EDF schedule (meeting all deadlines) would be:

```
0-25   : P1
25-55  : P2
55-80  : P1 (second instance)
80-105 : P2 (second instance)
```

All deadlines are satisfied, illustrating EDF's flexibility compared with RMS.

⏱️ Interrupt & Dispatch Latency in Hard Real-Time Systems

> **Interrupt latency** – time from the arrival of an external interrupt to the start of its ISR.
> **Dispatch latency** – time from a task becoming ready (e.g., after I/O) to the moment it actually begins executing on the CPU.

In hard real-time environments these latencies **must be bounded** because a missed deadline is considered a system failure.

- **Preemptive kernels** reduce dispatch latency by allowing the scheduler to immediately switch to a newly ready high-priority task.
- **Minimising interrupt latency** requires keeping ISRs short and often deferring work to a high-priority thread.
- Solaris, for example, reduced dispatch latency from > 100 ms to < 1 ms by enabling preemption.

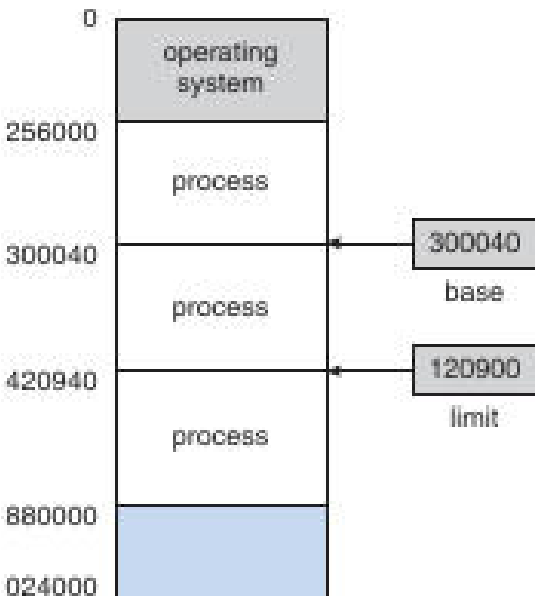
📚 Bibliographical Highlights (selected)

- **Feedback queues** originated in CTSS (Schrage, 1967).
- **Rate-monotonic and EDF** were introduced by Liu & Layland (1973).
- **Multicore scheduling** surveyed by Bhatia (2005) and Kongetira et al. (2005).
- **Linux CFS** design discussed by Love (2010) and Faggioli et al. (2009) (EDF extension).
- **Windows scheduling internals** detailed by Russinovich & Solomon (2009).
- **Solaris ULE & fair-share** described by Roberson (2003) and Henry (1984).

(Full citations are listed in the source transcript.)

🧠 Main Memory Management Overview

🖥️ Memory Layout Diagram



The diagram illustrates the operating-system area at low addresses, three separate process regions, and the base/limit registers (base = 300040, limit = 120900) that define each process's legal address range.

- **Base register** → smallest legal physical address for a process.
- **Limit register** → size of the allowed region; together they bound the process's logical address space.

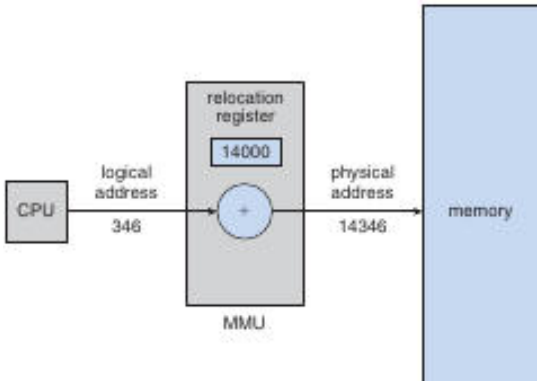
🔪 Address Binding & Spaces

> **Logical (virtual) address** – generated by the CPU during execution.
> **Physical address** – actual location in main memory after translation.

Binding can occur at three points:

1. **Compile time** – absolute addresses are known; no relocation needed.
2. **Load time** – relocatable code is bound to a specific memory region when the loader places the program.
3. **Execution (run) time** – a **Memory-Management Unit (MMU)** adds a relocation value to each logical address.

🧩 MMU Translation Example



The CPU issues logical address 346; the MMU adds the relocation register (14000) to produce physical address 14346, which is then used for the memory access.

🔄 Dynamic Loading

- Routines are **not loaded** until first called.
- The loader reads the relocatable module from disk, updates the program's address tables, and patches the call site.
- Benefits: reduces initial memory footprint and start-up time; useful for rarely-used error handlers or optional features.

🔗 Dynamic Linking & Shared Libraries

- **Dynamic linking** postpones resolution of library symbols until execution.
- A **stub** in the executable checks whether the library routine is resident; if not, the loader brings the library into memory and patches the stub.
- **Shared libraries** allow many processes to map the same code pages, saving RAM and enabling system-wide updates without recompiling applications.
- Versioning information lets the runtime select the appropriate library copy for each program.

🔁 Swapping

- When physical memory is insufficient, a **process image** can be moved to a **backing store** (typically a fast disk).
- **Standard swapping** swaps entire processes in and out; context-switch time is dominated by transfer latency (e.g., moving a 100 MiB process at 50 MiB/s ≈ 2 s).
- Swapping enables a degree of multiprogramming greater than the physical RAM size but incurs high overhead, so modern systems prefer paging or demand-paging techniques.

📄 Key Takeaways

- Multilevel and feedback scheduling provide mechanisms to **age** or **penalise** processes based on their CPU-burst behavior.
- Windows, Solaris, and Linux each expose distinct priority schemes; understanding the mapping between **class** and **relative** priority is essential for real-time tuning.
- Real-time schedulers (RMS, EDF) have clear utilization bounds; EDF is optimal but requires dynamic deadline tracking.
- Interrupt and dispatch latencies must be bounded in hard real-time systems; preemptive kernels are a primary tool.
- Memory protection via **base/limit** registers, **MMU** translation, **dynamic loading**, **dynamic linking**, and **swapping** collectively enable safe, flexible, and efficient use of main memory.

🗄️ Swapping and Its Limitations

> **Swapping** – moving an entire process's memory image between main memory and a backing store (typically disk) to free RAM for other processes.

- Swapping large processes is slow; a 100 MiB process can be swapped in ≈ 2 s, while a 3 GiB process may need ~60 s.
- To reduce swap time the OS would need to know the exact amount of memory a process actually uses and swap only that portion. This requires the process to **request** and **release** memory dynamically (e.g., request_memory(), release_memory()).
- Swapping is unsafe for a process that has pending I/O because the kernel may need to access the process's buffers. Two common solutions:
  1. **Never swap a process with pending I/O.**
  2. **Copy I/O data to kernel buffers only while the process is resident** (double-buffering), which adds extra copy overhead.
- Modern desktop/server OSes keep swapping disabled by default and enable it only when free memory falls below a configurable **threshold**; swapping stops once the threshold is cleared.
- **Mobile devices** (iOS, Android) avoid swapping because they rely on flash storage, which has limited write endurance and low throughput.
  - iOS asks applications to voluntarily free memory; read-only code pages are discarded and re-loaded from flash when needed.
  - Android may terminate a low-memory process after flushing its state to flash, allowing a quick restart.
  - Both platforms still support **paging** for virtual-memory management, but not full-process swapping.

📦 Contiguous Memory Allocation

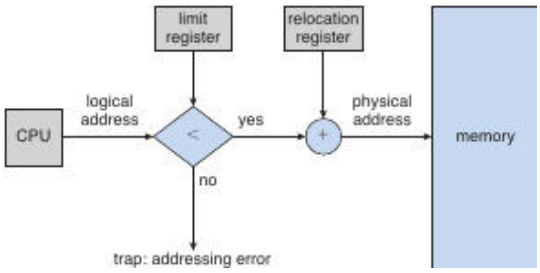
> **Contiguous allocation** – each process occupies one uninterrupted block of physical memory.

- The address space is split into two regions: **OS memory** (usually placed in low memory to keep the interrupt vector nearby) and **user memory**.
- Two classic schemes:
  1. **Fixed-size partitions** – a small, predetermined number of equal-sized slots; limits multiprogramming to the number of partitions.
  2. **Variable-size partitions** – the OS maintains a list of **holes** (free blocks) of assorted sizes and fits processes into the smallest suitable hole.
- **Allocation algorithms** for variable partitions:

Algorithm	Strategy	Typical outcome
First-fit	Scan from the beginning of the hole list; allocate the first hole that is large enough.	Fast, may leave relatively large leftover fragments.
Best-fit	Scan the entire list (or a size-ordered list) and allocate the *smallest* hole that fits.	Minimises leftover size, but requires more search.
Worst-fit	Allocate the *largest* hole found.	Leaves a large residual hole that can satisfy future large requests.

- When a process terminates, its hole is **merged** with any adjacent free holes, reducing external fragmentation.

🖼️ Memory-Management Flowchart



The CPU generates a logical address. The limit register checks that the address is within the allowed range; if it is, the relocation register is added to produce the physical address. An out-of-range address triggers a trap.

🛡️ Memory Protection with Relocation & Limit Registers

> **Memory protection** – preventing a process from accessing memory it does not own, enforced by the MMU using a **relocation register** (base) and a **limit register**.

- The **relocation register** holds the smallest physical address that the process may use.
- The **limit register** defines the maximum offset the process can address; any logical address larger than the limit causes a trap.
- During a context switch the OS loads the appropriate base and limit values for the scheduled process. Because **every** address generated by the CPU is checked, both the OS and user programs are shielded from accidental or malicious overwrites.
- This scheme also enables **dynamic OS sizing**: transient kernel code (e.g., rarely used device drivers) can be loaded on demand, expanding the OS region, and later unloaded to free memory.

📊 Variable-Partition Allocation – Dynamic Storage Allocation

- Initially the entire memory is a single large hole.
- When a process arrives, the OS searches the hole list using one of the strategies above; the chosen hole may be **split** into:
  1. The portion allocated to the process.
  2. The remaining free fragment, which stays (or is merged later).
- Upon process termination, its block becomes a hole; if it is contiguous with neighboring holes, they are **coalesced** into a larger hole.
- After each deallocation the OS may scan the waiting queue to see whether any blocked processes can now be satisfied.

🔧 Fragmentation

External Fragmentation

- Occurs when enough total free memory exists to satisfy a request, but the memory is divided into many small holes, none of which is individually large enough.
- The classic **50-percent rule** approximates that, with *N* allocated blocks, about *0.5 N* blocks are lost to external fragmentation.

  $\text{Lost blocks} \approx 0.5,N$

- Choice of allocation algorithm influences the severity: first-fit often leaves larger leftover fragments, while best-fit tends to produce many tiny holes.

Internal Fragmentation

- Happens when a fixed-size allocation (e.g., a partition or a page) is larger than the process's actual request, leaving unused space **inside** the allocated region.
- Example: with a 2 KB page size, a process needing 72 766 B requires 36 pages, wasting 962 B in the last page.
- On average, a process suffers roughly **½ page** of internal waste, assuming request sizes are independent of page size.

Compaction

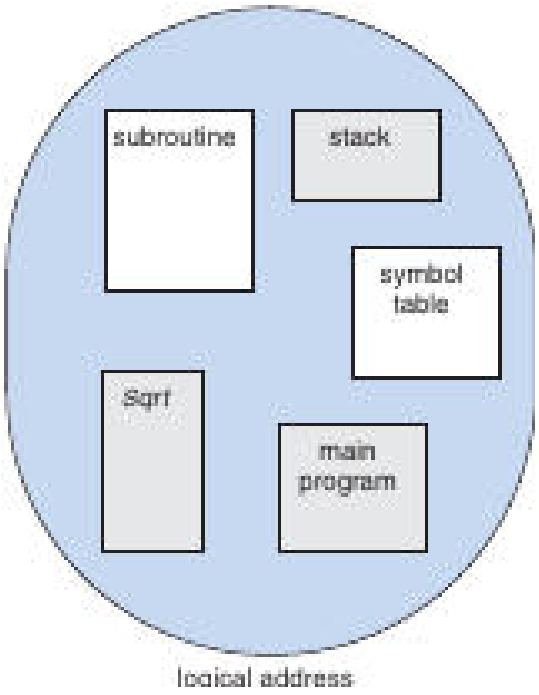
- To eliminate external fragmentation, the OS may **compact** memory: move all occupied blocks toward one end, gathering all free space into a single large hole.
- Compaction is only feasible when **dynamic relocation** is supported (i.e., the base address can be changed at run time).
- The cost includes copying data and updating base registers, which can be substantial; many systems instead rely on non-contiguous allocation techniques (segmentation, paging).

🧩 Segmentation

> **Segmentation** – a logical-address scheme where each address is a two-tuple , allowing variable-sized, named program modules.

- A **segment table** maps each segment number to a **base** (starting physical address) and a **limit** (segment length).
- On every memory reference the hardware:
  1. Uses the segment number to index the segment table.
  2. Checks that the offset ≤ limit; otherwise raises a trap.
  3. Adds the base to the offset to obtain the physical address.
- Segments typically correspond to:
  1. **Code** (text) segment.
  2. **Global data** segment.
  3. **Heap** (dynamically allocated memory).
  4. **Stack** (per-thread).
  5. **Shared libraries**.

🖼️ Segmentation Illustration



Each rectangle represents a distinct segment; a logical address is given by the segment identifier plus an offset within that segment.

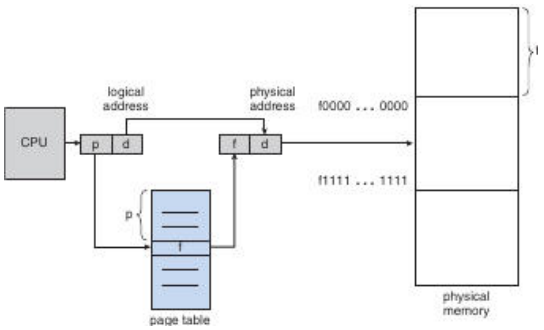
- Because segments can be placed anywhere in physical memory, **external fragmentation** is avoided; however, the segment table itself can become large.

📄 Paging – Fixed-Size Non-Contiguous Allocation

> **Paging** – divides both physical memory (into **frames**) and logical address space (into **pages**) of identical size; a page table maps virtual page numbers to physical frame numbers.

- **Page size** is a power of two (commonly 4 KB or 8 KB); this simplifies address translation:
  - High-order bits → **page number** (*p*).
  - Low-order bits → **offset** (*d*).
- The **page table** entry for page *p* contains the **frame number** (*f*) and status bits (present/absent, protection). The physical address is then <*f*, *d*>.

🖼️ Paging Translation Diagram



The CPU's logical address (page p, offset d) is looked up in the page table; the resulting frame f is combined with the same offset to form the physical address.

- **Advantages**: eliminates external fragmentation (any free frame can satisfy any page request) and simplifies allocation.
- **Internal fragmentation** remains because the last page allocated to a process may be only partially filled; on average this waste is ≤ ½ page.
- Larger pages reduce page-table size (fewer entries) but increase internal fragmentation and I/O latency; modern systems often support multiple page sizes (e.g., 4 KB and 2 MB "huge pages").
- **Page-table overhead**: on a 32-bit machine with 4-byte entries, a full 4-GB address space requires 4 MiB of page-table memory. Variable-length page tables, inverted page tables, or multi-level hierarchies mitigate this cost.

📱 Memory Management on Mobile Systems

- Flash storage's limited write endurance and low bandwidth discourage traditional swapping.
- **iOS**: when free RAM drops below a threshold, the OS requests applications to free caches; read-only code pages may be evicted and later re-loaded from flash. Modified pages (e.g., stacks) are never discarded. If an app cannot release enough memory, it is terminated.
- **Android**: follows a similar policy but, before killing a process, writes its state to flash so it can be quickly restarted.
- Both platforms rely on **paging** (kernel-managed page tables) for virtual-memory abstraction, but they do not swap entire processes to disk.

📚 Summary of Key Concepts

- Swapping whole processes is costly and unsafe for I/O-bound tasks; modern OSes enable it only under severe memory pressure, while mobile OSes avoid it entirely.
- **Contiguous allocation** is simple but suffers from external fragmentation; first-fit, best-fit, and worst-fit provide trade-offs between speed and waste.
- **Relocation and limit registers** give hardware-enforced protection and enable dynamic OS sizing.
- **Segmentation** offers a programmer-friendly logical view (named, variable-size segments) while still mapping to physical memory via a segment table.
- **Paging** provides a uniform, non-contiguous allocation method that removes external fragmentation; internal fragmentation is bounded by half a page on average.
- Mobile devices favor **memory-pressure callbacks** and selective page eviction over traditional swapping, due to flash-memory constraints.

📚 Paging Overview

> *Paging* divides both logical (virtual) memory and physical memory into fixed-size blocks called **pages** and **frames**. A page table maps each virtual page number to a physical frame number.

- When a process arrives, the OS determines how many pages n the process requires.
- If at least n free frames exist, they are allocated; each page is loaded into one of the frames and the corresponding frame number is entered into the process's page table.
- The programmer sees a single contiguous address space; the actual physical locations are hidden by the **address-translation** performed by the MMU.
- Because a process can only address pages listed in its page table, it cannot access memory belonging to another process.

📦 Frame Table

> *The frame table* is a data structure maintained by the OS, with one entry per physical frame indicating whether the frame is **free** or **allocated**, and if allocated, which process/page occupies it.

The frame table is consulted whenever the OS needs to locate an available frame for a new page or to free a frame when a process terminates.

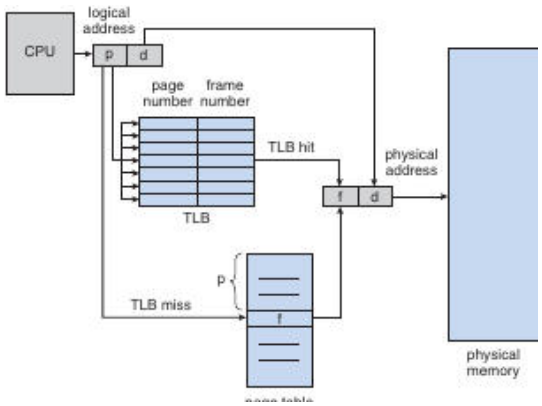
🖥️ Hardware Support for Paging

> *Hardware paging support* provides the mechanism by which the CPU translates a logical address (page + offset) into a physical address (frame + offset).

- **Page-table storage options**
  - **Registers** – Very fast but limited to a few dozen entries (e.g., PDP-11's eight-register page table). Suitable only for tiny address spaces.
  - **Main-memory page table** – The page-table base register (PTBR) points to a page-table stored in RAM. Changing the PTBR on a context switch updates the active page table with a single register write, reducing switch overhead.
- **Translation cost** – If the page table resides in memory, each memory reference may require two accesses (one for the page-table entry, one for the data). This overhead is mitigated by the TLB (see below).

⚡ Translation Lookaside Buffer (TLB) 🏎️

> *A TLB* is a small, fully associative cache that stores recent **page-number → frame-number** mappings, allowing the CPU to obtain the frame number without a full page-table walk.



The CPU sends a logical address (page p, offset d) to the TLB. On a hit, the TLB returns the frame number, which together with d forms the physical address. On a miss, the CPU must fetch the page-table entry from memory, then updates the TLB.

Hit Ratio & Effective Access Time

- **Hit ratio (h)** = probability that the needed page number is found in the TLB.
- **Effective access time (EAT)** = $h \cdot t_1 + (1\text{-}h) \cdot (t_1 + t_2)$, where t_1 is the time to access memory with a TLB hit and t_2 is the additional page-table lookup time.

Example	Hit ratio	t_1 (ns)	t_2 (ns)	EAT (ns)
Moderate TLB	0.80	100	100	120
Excellent TLB	0.99	100	100	101

Multi-Level TLBs & Replacement

- Modern CPUs may have separate **instruction** and **data** TLBs, each typically 32–64 entries.
- Replacement policies include **LRU**, **round-robin**, and **random**.
- Some entries can be **wired down** (e.g., kernel code) so they never get evicted.

- **Address-Space Identifiers (ASIDs)** tag each TLB entry with the owning process. If the ASID does not match the current process, the entry is treated as a miss, allowing the same TLB to hold translations for multiple processes without flushing on every context switch.

🛡️ Memory Protection in a Paged System

> *Protection bits* stored in each page-table entry specify the allowed access type for the corresponding frame (read, write, execute) and a **valid/invalid** flag.

- When a reference is made, the MMU checks the protection bits while translating the address.
- An attempt to write to a **read-only** page or to access an **invalid** page triggers a hardware trap (page-fault or protection-violation).
- The OS can combine multiple bits to create fine-grained policies such as **read-only**, **read-write**, **execute-only**, or **no-access**.

Illustrative example: In a 14-bit address space with 2 KB pages, pages 0-5 are marked valid. Any reference to pages 6-7 (addresses 12 288-16 383) causes a trap because the valid-invalid bit is cleared.

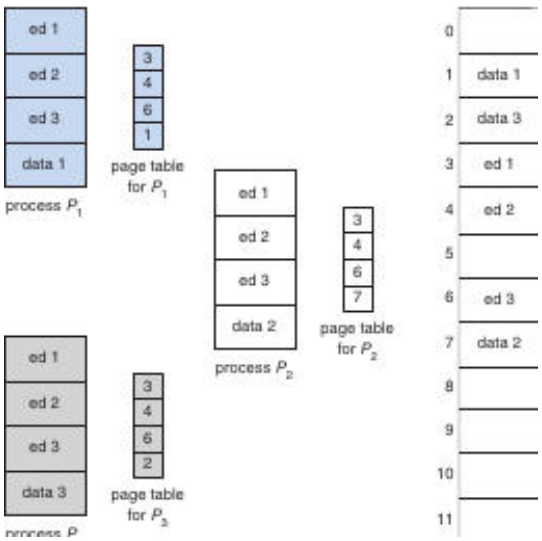
📂 Internal Fragmentation & Page-Table Size

- **Internal fragmentation** arises because the last page of a process may be only partially used; on average up to **½ page** of memory is wasted per process.
- Large logical address spaces produce huge page tables (e.g., a 32-bit address space with 4 KB pages → ≈ 1 M entries → 4 MB per process).
- Some architectures provide a **page-table length register (PTLR)** that records the actual size of a process's page table, allowing the OS to detect out-of-range addresses early and avoid allocating unused entries.

🤝 Shared Pages

> *Shared pages* allow multiple processes to map the same physical frame into their own virtual address spaces, reducing memory consumption for common, read-only code.

- The code must be **reentrant** (non-self-modifying) so that concurrent execution does not cause data races.
- Each process keeps its own page table, but the entries for the shared code point to the **same physical frame**.



Three processes (P1-P3) each have a private data page but share a single code page (the editor). The total memory needed drops from 8 000 KB (40 × (150 KB + 50 KB)) to about 2 150 KB because the 150 KB code page is stored once.

Benefits include:

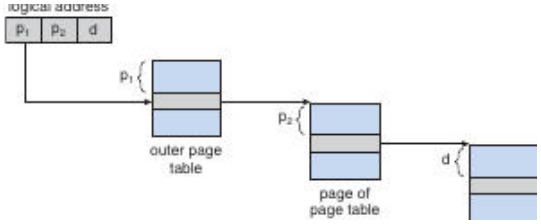
- **Memory savings** for frequently used libraries (compilers, runtime libraries, database engines).
- **Cache efficiency** – shared code stays resident in CPU caches across processes.

🏗️ Page-Table Structures

📊 Hierarchical (Multilevel) Paging

> *Hierarchical paging* breaks a large page table into smaller tables, each of which can itself be paged, dramatically reducing memory consumption.

- **Two-level scheme** (typical for a 32-bit address space, 4 KB pages):
  - Logical address split: **20-bit outer page number (p_1)**, **10-bit inner page number (p_2)**, **12-bit offset (d)**.
  - The outer page table holds pointers to inner page tables; each inner table holds the final frame numbers.



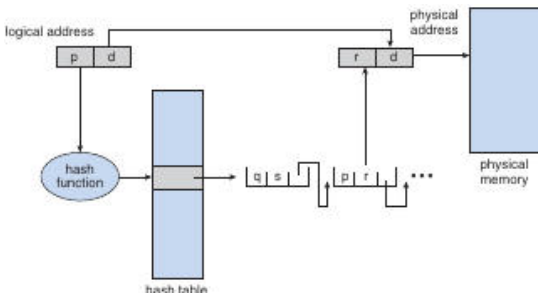
The outer page table entry points to a page containing the inner page table. The inner entry finally yields the physical frame number.

- **Three- or four-level paging** is used for 64-bit address spaces, where each additional level reduces the size of the top-level table (e.g., a four-level scheme can map a 2^{38}-byte space with manageable table sizes).
- Unused regions of the virtual address space simply have no second-level (or deeper) tables allocated, saving physical memory.

🔍 Hashed Page Tables

> *Hashed page tables* replace the hierarchical index with a hash of the virtual page number, suitable for very large or sparse address spaces.

- The hash function maps a virtual page number to a bucket in a hash table.
- Each bucket stores a **linked list** of entries: *(virtual page number, frame number, next-pointer)*.
- On a lookup, the CPU hashes the page number, scans the list for a matching virtual page, and retrieves the frame.



The logical address (p, d) is hashed to an index; the selected hash-table entry supplies the frame number, which together with d forms the physical address.

- **Clustered page tables** extend this idea by allowing a single hash-bucket entry to describe multiple contiguous pages, further reducing table size for sparse mappings.

🔄 Inverted Page Tables

> *An inverted page table* contains **one entry per physical frame** rather than per virtual page.

- Each entry stores:
  - The **virtual page number** currently occupying the frame.
  - An identifier for the **process** that owns the page.
- Because there is only one table for the whole system, memory overhead is proportional to the number of frames, not the size of the virtual address space.

Advantages

- Scales to very large address spaces without exploding memory usage.
- Simplifies detection of which frame a process is using.

Drawbacks

- Lookup requires searching the table (often via a hash on the virtual page number) which can be slower than a direct indexed page table.

📄 Key Takeaways

- Paging provides a clean abstraction between a process's logical view of memory and the underlying physical frames, enforced by the MMU and protected by per-page bits.
- The **TLB** is essential for performance; its hit ratio directly impacts effective memory-access time.
- **Shared pages** and **reentrant code** dramatically reduce memory footprints in multi-user systems.
- As address spaces grow, **hierarchical**, **hashed**, and **inverted** page-table designs keep page-table storage manageable while supporting efficient address translation.

🧩 Inverted Page Tables (IPT)💾

> *An inverted page table stores one entry per **physical frame** rather than per virtual page, using a tuple (**process-id**, **page-number**) to identify the owner of each frame.*

- **Typical entry**: – the **process-id** serves as the address-space identifier (ASID) that distinguishes which logical page the frame belongs to.
- **Lookup process**
  1. A virtual address reaches the memory subsystem.
  2. The IPT is **searched** for a matching pair.
  3. If found, the physical frame number is combined with the offset to form the final physical address.
  4. If no match exists, a page-fault is generated.
- **Search cost** – because the table is indexed by **physical frame** rather than virtual page, a naïve linear scan would be expensive. Modern systems therefore **hash** the tuple, limiting the search to one or a few entries.
- **Performance impact** – each address translation now requires at least two memory reads: one for the hash-bucket entry and another for the IPT entry (after a TLB miss).
- **Shared-memory limitation** – with a single IPT entry per frame, the same physical page cannot be mapped to **different virtual addresses** in multiple processes. Implementing shared memory therefore requires each process to have its own

virtual page that maps to the same physical frame, complicating the classic “multiple virtual addresses → one physical page” scheme used with ordinary page tables.

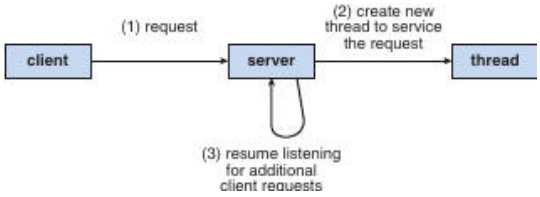
- **Historical examples** – IBM’s **RT** (and later RS/6000, Power CPUs) were early adopters of inverted tables; UltraSPARC also employs this technique.

⚡ Hashed Page Tables & Translation Storage Buffer (TSB) – Solaris Example 🌐

> *A hashed page table stores page-table entries in a hash-based structure; the TSB is a small hardware cache that holds recently used TTEs (translation-table entries).*

- **Two separate hash tables** – one for the kernel, one for all user processes. Each entry may represent a **contiguous span of pages**, reducing the number of hash buckets required.
- **Address translation flow**
  1. CPU generates a virtual address.
  2. **TLB** is consulted first. On a miss, the hardware **walks the TSB** (a fast in-memory cache).
  3. If the TSB also misses, the CPU raises an interrupt. The kernel searches the appropriate hashed page table, creates a new TTE, and inserts it into the TSB.
  4. The TSB entry is then used for subsequent translations of the same virtual page.

  This flow is illustrated in the Solaris diagram below, where the hash-table lookup is triggered only after the TLB and TSB miss.



  The diagram shows a client request being handled by a server thread, analogous to the kernel handling a TSB miss and consulting the hashed page table.
- **Advantages** – dramatically reduces the memory overhead of a full page table while keeping lookup times acceptable through hashing and caching.
- **Drawbacks** – still requires an extra memory reference for the hash-bucket and can suffer from collisions in heavily loaded systems.

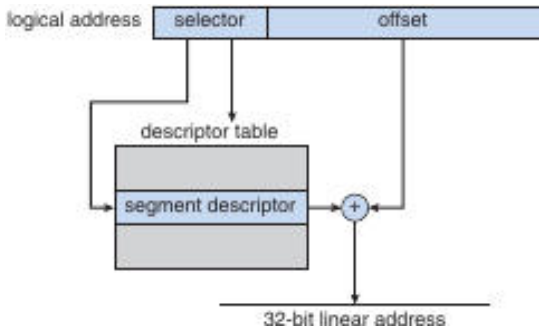
🏁 IA-32 Architecture: Segmentation + Paging 🔧

> *IA-32 combines a **segmentation unit** that produces a linear address from a logical selector/offset pair, followed by a **paging unit** that maps the linear address to a physical frame.*

Segmentation fundamentals

- **Logical address** = *(selector, offset)*
  - **selector (16 bits)** identifies a segment descriptor in either the **Local Descriptor Table (LDT)** or the **Global Descriptor Table (GDT)**.
  - **offset (32 bits)** specifies the byte offset within the selected segment.
- **Segment descriptor (8 bytes)** contains: base address, limit, type, and protection bits.

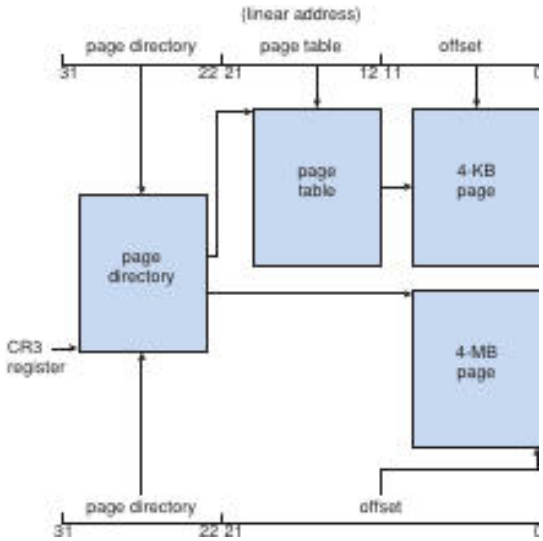
- **Segment registers** (six total) hold the selector values; a small **descriptor cache** in the CPU avoids fetching the descriptor from memory on every access.



The selector points to a descriptor; the base from the descriptor is added to the offset, yielding a 32-bit linear address. A limit check precedes the addition; a violation triggers a fault.

Paging on IA-32

- **Two-level page translation** (for 4-KB pages)
  - **CR3 register** holds the physical base of the **page directory**.
  - The high-order 10 bits of the linear address index the **page-directory entry** → points to a **page table**.
  - The next 10 bits index the **page-table entry** → yields the **frame number**.
  - The low-order 12 bits are the **page offset**.



The diagram shows the split of a linear address into directory, table, and offset fields, with the optional 4-MB page shortcut.

- **Page-size flag** in a directory entry can select a **4-MB page**, bypassing the inner page table.
- **Physical-memory-management bits** (present, dirty, accessed, etc.) allow the OS to page directory or page-table entries themselves in and out of RAM.

Physical-Address-Extension (PAE)

- Extends the physical address field from **20 bits → 24 bits**, enabling up to **64 GB** of RAM.
- **Three-level hierarchy**:
  1. **Page-Directory-Pointer Table (PDPT)** – top two bits of the linear address.
  2. **Page Directory** – next 9 bits.
  3. **Page Table** – following 9 bits.

- Supports **4-KB** and **2-MB** pages.
- Requires OS support (Linux/macOS support PAE; many Windows desktop versions limit user processes to 4 GB even when PAE is enabled).

🖥️ x86-64 Architecture (AMD64 / Intel 64) 🚀

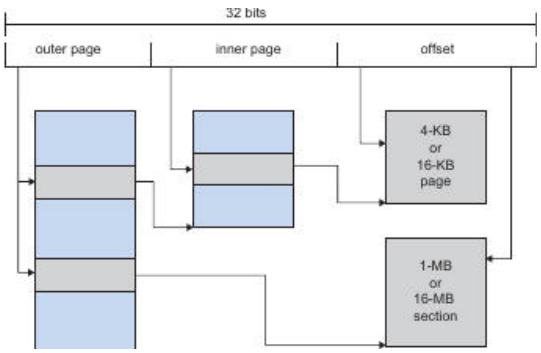
> *x86-64 expands IA-32 with a 48-bit virtual address space and a 52-bit physical address space, using a four-level paging hierarchy.*

- **Virtual address layout (48 bits used)**
  - **Level-4 index (9 bits)** → **PML4** (Page-Map Level 4) entry.
  - **Level-3 index (9 bits)** → **PDPT** entry.
  - **Level-2 index (9 bits)** → **Page Directory** entry.
  - **Level-1 index (9 bits)** → **Page Table** entry.
  - **Offset (12 bits)** → byte within the 4-KB page.
- **Supported page sizes** – 4 KB, 2 MB, and 1 GB. Larger pages reduce TLB pressure.
- **Physical address** – up to **52 bits** (≈ 4 PB).
- **ASIDs** are still used to tag TLB entries, allowing fast context switches without full TLB flushes.
- The hierarchy mirrors the PAE scheme but adds an extra level (PML4) to accommodate the larger virtual space.

📱 ARM Architecture: Paging & TLBs 📡

> *ARM processors use a flexible page-size scheme and a two-level TLB hierarchy to translate 32-bit virtual addresses.*

- **Supported page/section sizes**
  - **Pages**: 4 KB, 16 KB
  - **Sections** (larger "super-pages"): 1 MB, 16 MB
- **Two-level paging** for pages and sections
  - **Outer page** entry points to an **inner page** table (or directly to a section).
  - The **offset** selects the final byte.



The diagram shows outer and inner page fields, followed by offsets for both 4-KB/16-KB pages and 1-MB/16-MB sections.

- **TLB hierarchy**
  - **Micro-TLB**: separate instruction and data TLBs, each supporting **ASIDs**; very fast, checked first.
  - **Main TLB**: larger, shared between instruction and data.
  - On a miss in both TLBs, hardware performs a **page-table walk** to locate the translation, then caches the result in the TLBs.
- **Use cases** – mobile devices (smartphones, tablets) favor ARM's variable page sizes to balance memory usage and TLB pressure, while maintaining low power consumption.

📊 Comparison of Major Memory-Management Schemes

Scheme	Mapping granularity	Main data structure	Memory overhead	Typical lookup cost*	Fragmentation type
Contiguous allocation	Whole process	Base/limit registers	Very low	O(1) (simple compare)	Internal (wasted space inside partition)
Segmentation	Variable-size segments	Segment descriptor tables (LDT/GDT)	Moderate (per-segment descriptor)	O(1) if descriptor cached; else memory fetch	External (holes between segments)
Paging	Fixed-size pages	Hierarchical page tables (2-level, 3-level with PAE, 4-level x86-64)	Proportional to number of pages (can be large)	O(1) with TLB; otherwise 2-3 memory accesses	Internal (partial page waste)
Inverted page table	One entry per **physical frame**	Single table indexed by (hashed)	Small (one entry per frame)	Hash lookup + possible TLB miss (≈ 2 accesses)	No external; internal depends on page size
Hashed page table (Solaris)	Variable-size spans	Hash buckets + TSB cache	Moderate (hash + TSB)	TLB → TSB → hash (usually 1–2 memory refs)	Internal (partial-span waste)
Combined segmentation + paging	Segments → pages	Segment table + per-segment page tables	Higher (both tables)	TLB for page, plus segment lookup (often cached)	Both internal & external (depends on segment layout)

*Lookup cost assumes a modern CPU with a TLB; without a TLB the cost grows with the depth of the page-table hierarchy.

🛡️ Key Takeaways (Cross-Sectional)

- **Address-space identifiers (ASIDs)** are essential wherever multiple processes share a single translation structure (hashed tables, inverted tables, TLBs) to prevent cross-process aliasing.
- **Hashing** transforms the potentially linear search of an IPT into an O(1) expected lookup, at the expense of an extra memory reference and possible collisions.
- **TLB / TSB** layers act as fast-path caches; their effectiveness determines whether the overhead of complex page-table schemes is tolerable.
- **Shared memory** is straightforward with ordinary page tables (multiple virtual pages → same frame) but requires explicit handling in inverted schemes (e.g., multiple IPT entries mapping to the same frame or using separate page tables per process).
- **PAE** and **x86-64** illustrate how extending the physical address width (from 20 bits to 24 bits, then to 52 bits) is achieved by adding hierarchy levels rather than redesigning the whole translation logic.
- **ARM's flexible page/section sizes** demonstrate a design trade-off for power-constrained devices: larger sections reduce TLB pressure but increase internal fragmentation; smaller pages improve memory utilization at the cost of more page-table entries.

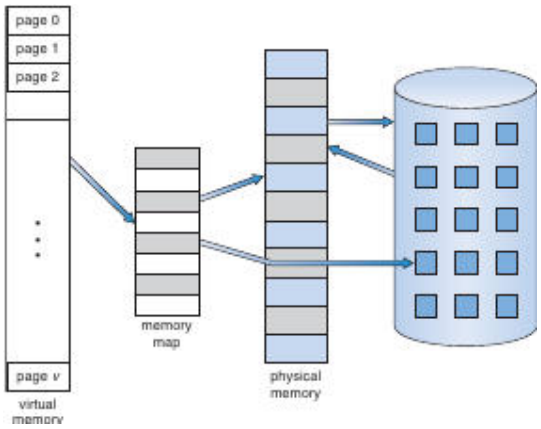
These concepts interlink with earlier discussions of **TLBs, page-table walks, and fragmentation**, completing the picture of how modern systems balance memory efficiency, translation speed, and hardware constraints.

📚 Virtual Memory Overview

> **Virtual memory** is a technique that abstracts main memory into a large, uniform logical address space, separating the *logical* view seen by a process from the *physical* memory actually present.

- Allows programs to be larger than physical RAM.
- Enables multiple processes to share code and data pages.
- Provides a convenient mechanism for efficient process creation (e.g., via fork).

🖼️ Virtual-Memory Mapping Diagram



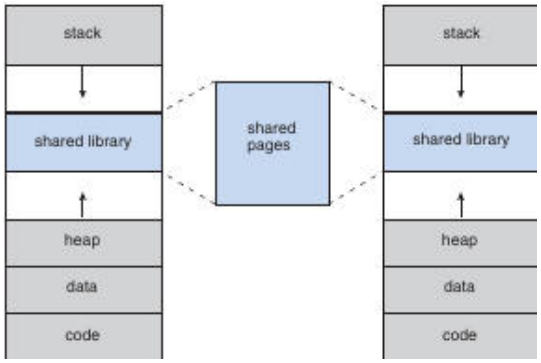
Virtual pages (left) are mapped through a memory-map table to a set of physical frames (right). Only the pages that are actually needed reside in physical memory at any moment.

🏗️ Address-Space Layout

> **Logical address space** of a process typically starts at address 0 and is divided into distinct regions that grow in opposite directions.

Region	Growth direction	Typical contents
Code / Text	Downward (low addresses)	Executable instructions (read-only)
Data	Downward	Initialized global/static variables
Heap	Upward	Dynamically allocated memory (via malloc, new)
Shared libraries	Placed wherever convenient; often mapped into a *hole* between heap and stack	
Stack	Upward from high addresses	Call frames, local variables, return addresses

🖼️ Process Memory Layout



Two processes each have stack, heap, data, code, and a shared-library segment. The shared library maps to a common set of physical pages, illustrated by the blue "shared pages" block.

- The gap between heap and stack is a **hole** that may remain unused (a *sparse* address space).
- Sparse spaces are useful because they can later host dynamically linked objects or additional segments without relocating existing pages.

⚙️ Demand Paging

> **Demand paging** loads a page into physical memory only when the executing process first accesses it.

Why Demand Paging?

- Reduces the amount of RAM needed at startup.
- Exploits *locality of reference*: most programs touch only a small fraction of their address space during any given interval.

Page-Fault Handling Steps

1. **Trap**: CPU detects an invalid-bit in the page-table entry and generates a page-fault interrupt.
2. **Save state**: Registers, program counter, and other process context are saved.
3. **Validate reference**: OS checks whether the faulting address is legal (e.g., not an outright segmentation violation).
4. **Locate page**: Determine the location of the required page on secondary storage (swap space or file backing).
5. **Allocate frame**: Choose a free physical frame (or evict a victim frame via a replacement algorithm).
6. **Initiate I/O**: Issue a disk read to transfer the page into the allocated frame; the CPU may be scheduled to another ready process while waiting.
7. **Update page table**: Mark the entry as *valid* and store the frame number.
8. **Restart instruction**: Restore the saved context and re-execute the faulting instruction.

🖼️ Page-Fault Flowchart

Steps in handling a page fault

The diagram shows the sequence from fault detection through disk I/O, page-table update, and instruction restart.

Performance Impact

Effective access time (EAT) for a demand-paged system:

$ \text{EAT}= (1-p)\times m_a ;+; p \times t_{\text{fault}} $

- (p) = probability of a page fault.
- (m_a) = memory-access time (typically 10–200 ns).
- (t_{\text{fault}}) = time to service a fault (disk seek + latency + transfer + OS overhead), often several **milliseconds**.

Example calculation (using typical values):

Parameter	Value
(m_a)	200 ns
Disk service time	8 ms (≈ 8 000 000 ns)
Desired slowdown ≤ 10 %	0.1 × 200 ns = 20 ns

Solving

$ 20\text{ ns} \ge p \times 7{,}999{,}800\text{ ns} ;\Rightarrow; p < 2.5\times10^{-6} $

Thus, **fewer than 1 fault per 400 000 memory references** is needed to keep slowdown under 10 %.

Key insight: demand paging is viable only when the page-fault rate is extremely low, which is usually achieved thanks to program locality.

📂 Page-Table Support for Demand Paging

- **Valid/Invalid bit** (or a special protection value) distinguishes pages that are resident (valid = 1) from those that are not (valid = 0).
- When valid = 0, the page-table entry may also store the **disk block address** of the page's backing store.
- On a fault, the OS reads this address to issue the I/O request.

📦 Swap Space & Backing Store

- **Swap space** is a reserved region on disk used as the backing store for pages that have been evicted from RAM.
- Some systems treat **file-backed pages** (e.g., executable code, read-only data) as their own backing store, reading directly from the file system instead of swap.
- **Anonymous pages** (heap, stack, and other pages without a file) must be backed by swap.

Page type	Typical backing store
File-backed (code, read-only data)	Original file on disk
Anonymous (heap, stack)	Swap space
Shared library mappings (read-only)	Shared file or memory-mapped region

Mobile operating systems (iOS, Android) often **omit swap**, relying on aggressive reclamation of file-backed pages and on-the-fly compression rather than a dedicated swap partition.

🔁 Copy-On-Write (COW)

> **Copy-on-write** is a technique that lets a parent and child process share the same physical pages after a fork(). Pages are marked **read-only**; the first write by either process triggers a page fault, at which point the OS allocates a new private copy for the writer.

- **Advantages**
  - Immediate process creation without copying all pages.
  - Only pages that are actually modified become duplicated, saving memory and I/O.
- **Typical workflow**
  1. fork() creates child; both address spaces point to the same set of frames.
  2. All shared frames are marked **COW** (read-only).
  3. When either process writes to a COW page, a fault occurs.
  4. OS allocates a free frame, copies the contents, updates the page-table entry for the faulting process, and clears the COW flag.
- **Illustration** (conceptual):

Before write: Parent P and Child C → same physical frame **F** (marked COW).

After child writes: C gets a new frame **F′**, P continues to use **F**.

COW is employed by modern OSes such as **Linux**, **Windows XP**, and **Solaris**.

📂 Shared Libraries & Page Sharing

- System libraries are **mapped read-only** into each process that links them.
- Because the mapping is identical, all processes reference the **same physical pages**.
- This reduces overall memory consumption and speeds up process startup (no need to load duplicate copies).

Benefit table

Benefit	Explanation
Memory savings	One copy of library code serves many processes.
Cache efficiency	Frequently used library code stays in CPU caches across context switches.
Fast fork()	The child inherits the same library mappings; COW handles any later modifications (rare for read-only code).

📊 Sparse Address Spaces

> A **sparse address space** contains large unmapped "holes" that do not correspond to any physical pages.

- Useful for **dynamic linking**: a shared object can be mapped into a hole without moving existing segments.
- Holes also accommodate future growth of the heap or stack without requiring relocation of other regions.

🛠️ Hardware Support for Paging & Demand Paging

- **Translation Lookaside Buffer (TLB)**: caches recent virtual-to-physical translations; a TLB miss triggers a page-table walk, which may uncover an invalid entry → page fault.
- **Valid/Invalid bits** in page-table entries are examined by the MMU during address translation.
- **Protection bits** (read/write/execute) help detect illegal accesses before a fault is raised.

> As discussed earlier, a **TLB hit** yields effective access time ≈ memory-access time, while a miss that leads to a page fault incurs the full fault service overhead.

📈 Summary of Demand-Paging Performance Factors

Factor	Influence on EAT
Page-fault rate (p)	Linear increase; must be kept extremely low.
Disk service time	Dominates fault cost; faster disks (SSD) can reduce t_{fault}.
Replacement algorithm	Affects how often faults occur; good algorithms lower (p).
Prefetching / clustering	Can reduce faults for sequential accesses.
TLB hit ratio	Higher hit ratio reduces the chance of encountering an invalid entry early.

📚 Selected Historical References

- Early segmentation concepts: J. B. Dennis (1965).
- First paged systems: GE 645 implementation (Organick 1972).
- Inverted page-table research: Chang & Mergen (1988).
- 64-bit address-space page-table designs: Talluri et al. (1995).
- Large-page hardware support studies: Fang et al. (2001).
- PAE support in Windows: Microsoft MSDN documentation.
- IA-32 and x86-64 architecture manuals: Intel® 64 and IA-32 Architectures Software Developer's Manual.
- ARM Cortex-A9 architecture overview: ARM.com.

📂 Copy-On-Write and vfork

> **Copy-on-Write (COW)** – a memory-management optimization where parent and child processes initially share the same physical pages; a page is duplicated only when one of the processes attempts to modify it.

- After a fork(), pages **A**, **B**, and **C** are marked read-only and mapped into both processes.
- When **process 1** writes to page **C**, the hardware raises a protection fault; the OS copies page C to a new frame, updates the page table of the writer, and clears the read-only flag for that process.
- Pages that remain unchanged stay shared, saving RAM and avoiding unnecessary I/O.

Zero-fill-on-demand pages are created by the OS as all-zero frames that are allocated only when a process first accesses them; the frame is cleared before it becomes visible to the process.

vfork()

- vfork() creates a child that **shares the entire address space** of the parent without using COW.
- The parent is **suspended** until the child either calls exec() or exits.
- Because the child can modify the parent's pages, **the programmer must ensure the child does not alter the address space** (e.g., only calls exec() immediately).
- vfork() is therefore an efficient way to launch a new program from a shell, avoiding the page-copy overhead of a regular fork().

📊 Page-Replacement Fundamentals

When a page fault occurs and **no free frame** exists, the OS must **evict** a resident page to make room for the needed one.

Standard replacement steps

1. Locate the required page on secondary storage.
2. Choose a *victim* frame (via a replacement algorithm).
3. If the victim's **modify/dirty bit** is set, write the frame back to disk; otherwise, simply discard it.
4. Read the desired page into the now-free frame and update all relevant page-table entries.
5. Restart the faulting process.

> **Modify (dirty) bit** – set by hardware when a page is written; indicates that the in-memory copy differs from the copy on disk.

Using the dirty bit cuts I/O roughly in half for read-only or unmodified pages.

📈 Page-Fault Rate & Reference Strings

A **reference string** is the ordered list of page numbers referenced by a process.
Only the **page number** matters (the offset is ignored) and consecutive references to the same page cannot cause additional faults.

Example reduction

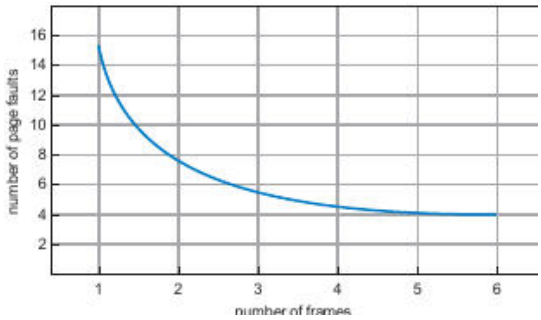
Original addresses (page size = 100 bytes):

0100, 0432, 0101, 0612, … → Page numbers: 1, 4, 1, 6, …

The number of frames influences the fault count:

- More frames ⇒ fewer faults (the fault-rate curve is monotonic decreasing for most algorithms).
- With very few frames, a fault may occur on *every* reference.

The graph below shows how the number of **edge faults** drops as the number of frames grows:



The blue line illustrates that increasing the frame count from 1 to 6 steadily reduces the number of faults.

📉 Page-Replacement Algorithms

🗃️ FIFO (First-In, First-Out)

- Pages are kept in a **queue** ordered by arrival time.
- On a fault, the page at the **head** of the queue is evicted; the newly loaded page is appended to the **tail**.

Illustrative reference string (3 frames)

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 …

Step	Frames (left→right)	Fault?
7	7 – –	✓
0	7 0 –	✓
1	7 0 1	✓
2	2 0 1	✓ (evict 7)
0	2 0 1	–
3	2 3 1	✓ (evict 0)
…	…	…

Total faults: **15**.

Belady's anomaly – for FIFO, increasing the number of frames can *increase* the fault count (e.g., 3 frames → 15 faults, 4 frames → 17 faults). This counter-intuitive behavior is specific to certain non-stack algorithms.

🏆 Optimal (OPT / MIN)

- Replace the page whose **next use is farthest in the future** (or never used again).
- Guarantees the **lowest possible fault rate** for a fixed number of frames.

Using the same reference string, OPT yields **9 faults**—the theoretical minimum.
Because it requires knowledge of future references, OPT is used only for **benchmarking**.

⏳ LRU (Least-Recently-Used)

- Approximate optimality by evicting the page that has **not been used for the longest time** (look-back).
- **Implementations**

1. **Counters** – each page-table entry stores a timestamp; the OS updates the timestamp on every reference.
2. **Stack** – a doubly-linked list where the most-recently-referenced page is moved to the front; the tail holds the LRU page.

LRU never exhibits Belady's anomaly and typically outperforms FIFO (12 faults on the example string).

🌀 Approximate LRU

Additional-Reference-Bits Algorithm

- Each page has an **8-bit shift register** (or configurable length).
- At regular intervals (e.g., every 100 ms), the OS shifts the register right, inserting the current **reference bit** (set by hardware on any access) into the high-order position.
- The register's integer value reflects recent usage; the page with the **smallest value** is chosen as the victim.

Second-Chance (Clock) Algorithm

- Pages are arranged in a **circular queue** with a pointer ("clock hand").
- When a victim is needed, the algorithm scans pages:
  - If the **reference bit = 0**, the page is evicted.
  - If the **reference bit = 1**, the bit is cleared and the pointer moves on, giving the page a "second chance."
- If all bits are set, the algorithm cycles once, clearing bits, and then selects the first cleared page.
- In the worst case the scan touches every frame twice, but it requires only the single reference bit per page.

Enhanced Second-Chance (Class Algorithm)

- Combines **reference** and **modify (dirty)** bits to define four classes:

Class	Ref-bit	Dirty-bit	Replacement priority
(0,0)	0	0	**Best** – not recently used, clean
(0,1)	0	1	Next – clean-to-dirty (needs write-back)
(1,0)	1	0	Later – recently used, clean
(1,1)	1	1	**Worst** – recently used, dirty

The clock hand scans for the **lowest-nonempty class**; this reduces unnecessary I/O because clean pages are preferred over dirty ones.

📊 Counting-Based Algorithms

Algorithm	Metric	Typical behavior
LFU (Least Frequently Used)	Access count per page	Evicts the page with the smallest count; suffers from "cache-pollution" if a page is heavily used early then becomes idle.
MFU (Most Frequently Used)	Access count per page	Evicts the page with the largest count, assuming it has already been used enough; rarely used in practice.
Decay-based LFU	Periodically right-shift counters (exponential decay)	Allows counts to "age," preventing stale high-frequency pages from monopolizing memory.

📄 Quick Comparison of Common Page-Replacement Policies

Policy	Knowledge required	Typical fault rate (example)	Anomaly?	Hardware support
FIFO	None	15 faults (3 frames)	**Yes** (Belady)	Simple queue
OPT	Future references	9 faults (minimum)	No	None (theoretical)

LRU	Past usage timestamps	12 faults	No	Counters or stack (software)
Approx. LRU (Clock)	Reference bit	≈ 13 faults	No	Single reference bit per page
LFU / MFU	Access counts	Varies; may be high	No	Counters (more storage)

📚 Key Takeaways (integrated with earlier sections)

- **Copy-on-Write** and **vfork** are complementary techniques that reduce the cost of process creation; they rely on the same **page-table and dirty-bit mechanisms** discussed in the demand-paging section.
- **Page replacement** completes the logical-to-physical mapping loop introduced when describing the MMU and TLB: after a page is loaded, it must eventually be evicted to keep the frame pool bounded.
- The **modify/dirty bit** ties together page-fault handling, copy-on-write, and replacement decisions, enabling the OS to avoid unnecessary writes.
- Understanding **reference strings** and **frame allocation** is essential for evaluating any replacement algorithm; the graph above visualizes the generic trade-off between frame count and fault frequency.
- Approximate LRU schemes (second-chance, clock, additional-reference-bits) are widely used in real systems because they require only the minimal hardware support (a single reference bit) already present for **page-access tracking**.

📚 Page-Buffering Algorithms

> *Page-buffering* augments a conventional page-replacement algorithm by keeping a **pool of free frames** that can be used to read a faulted page **before** the victim frame is written back to disk.

- When a page fault occurs:
  1. The needed page is read into a **free frame** from the buffer pool.
  2. The process can be resumed immediately; the victim page is written back **later**, when the buffer pool has idle capacity.
- Variants keep a **list of modified pages** (dirty pages). While the paging device is idle, a dirty page is selected, written to disk, and its modify bit is cleared. This increases the chance that a victim selected later will already be clean, avoiding an extra write-out.

Advantages

- Reduces the latency of the faulting process (no immediate I/O for the victim).
- Allows the OS to schedule victim writes during periods of low paging activity.

Implementation notes

- The VAX/VMS system uses this technique together with FIFO; a page that is still in active use is quickly retrieved from the buffer pool, eliminating unnecessary I/O.
- Some UNIX variants combine the buffer pool with the **second-chance** algorithm to mitigate the penalty of an incorrectly chosen victim.

⚙️ Applications and Page Replacement

> Certain applications (e.g., database servers, data-warehouses) perform their own **I/O buffering** and may achieve better performance when they bypass the OS's generic paging mechanisms.

- **Database engines** often keep their own cache of frequently accessed pages, reducing the number of page faults that reach the OS.
- In **sequential-read workloads** (typical of data-warehouses), the OS's LRU policy can evict pages that will be needed soon, while an **MFU** (most-frequently-used) policy would keep those pages longer and improve throughput.
- Some systems expose a **raw I/O** interface: a large, contiguous disk partition used as a "raw" array without filesystem metadata. This eliminates overhead from file-system services (locking, allocation, directory lookup) and can be advantageous for special-purpose applications that manage their own layout.

> *Note*: For most general-purpose programs, using the regular filesystem yields better overall performance because the OS can apply sophisticated caching and prefetching strategies.

📦 Allocation of Frames

> Frame allocation determines how the **fixed pool of physical frames** is divided among active processes.

Minimum Number of Frames

> A process must receive **enough frames** to hold all pages that can be simultaneously referenced by a single instruction.

- For simple architectures where an instruction references **one address**, the minimum is **2 frames** (one for the instruction, one for the operand).
- With **single-level indirect addressing**, at least **3 frames** are required (instruction, pointer page, operand page).
- Complex instructions (e.g., PDP-11 move, IBM 370 MVC) can need **6–8 frames** because the instruction itself, its operands, and possible indirect references may each straddle page boundaries.
- To avoid pathological cases, many systems impose a **limit on indirection depth** (e.g., 16 levels). The limit caps the worst-case number of frames needed for a single instruction.

Allocation Algorithms

Algorithm	Principle	Example (62 free frames)
Equal allocation	Each process receives ⌊m/n⌋ frames (m = total frames, n = processes).	Two processes → 31 frames each; leftover frames form a buffer pool.
Proportional allocation	Frames are distributed in proportion to each process's **virtual size** s_i. $a_i = \left\lceil \frac{s_i}{\sum_j s_j}; m \right\rceil$	Process A = 10 KB, Process B = 127 KB, m = 62 → A gets 4 frames, B gets 58 frames (rounded).

- Both schemes may reserve a small **buffer pool** of free frames to guarantee that a fault can be serviced without immediate eviction.
- High-priority processes can be granted extra frames beyond their proportional share, either by **priority-weighted proportional allocation** or by borrowing from lower-priority processes.

Global vs. Local Replacement

- **Global replacement**: the victim can be any frame in the system, even those belonging to other processes.
  - Allows a high-priority process to "steal" frames, improving its own performance.
  - Drawback: a process's fault rate becomes dependent on the behavior of *all* other processes, leading to unpredictable performance.
- **Local replacement**: each process may only replace frames that it already owns.
  - Guarantees that a process's fault rate reflects only its own paging pattern.
  - May reduce overall throughput because idle frames owned by a low-priority process cannot be used by a high-priority one.

> In practice, many OSes combine the two: a **global pool** supplies frames when a process's local allocation is exhausted, but otherwise each process works with its own local set.

Non-Uniform Memory Access (NUMA)

> **NUMA** systems have memory modules with **different access latencies** depending on the CPU that issues the request.

- Goal: allocate pages **as close as possible** to the CPU that will use them, minimizing remote-memory latency.
- Typical strategy:
  1. The scheduler records the **last CPU** on which a process (or thread) ran.
  2. The memory manager attempts to place new pages in the node that contains that CPU's local memory.

- **Solaris** implements this with **lgroups** (latency groups). An lgroup groups together CPUs and memory with low inter-node latency; the kernel schedules all threads of a process within a single lgroup when possible.
- When a process's threads span multiple nodes, the OS may allocate pages across several lgroups, preferring the group that hosts the majority of the threads.

🚨 Thrashing

> **Thrashing** occurs when a process does not have enough frames to keep its active pages resident, causing it to fault repeatedly on the same set of pages.

Cause of Thrashing

1. **Degree of multiprogramming** rises → more processes compete for a limited frame pool.
2. A **global replacement** algorithm selects victims without regard to process ownership; a newly started process can steal frames from an already-running process.
3. The victim process now faults on pages it just lost, stealing frames from yet another process, and so on.
4. The **ready queue empties** because processes spend most of their time waiting for the paging device.
5. **CPU utilization drops**, prompting the scheduler to increase the degree of multiprogramming even further, which exacerbates the problem.

The characteristic curve (Figure 8.18) shows CPU utilization rising with multiprogramming until a peak, after which it collapses due to thrashing.

Working-Set Model

> The **working-set** of a process at time t is the set of pages referenced during the most recent Δ references (the **working-set window**).

- A process's **working-set size** WSS_i approximates the number of frames it needs to avoid thrashing.
- If the sum of all processes' working-set sizes $D = \Sigma\, WSS_i$ exceeds the total number of frames, at least one process will thrash.

Operating-system strategy

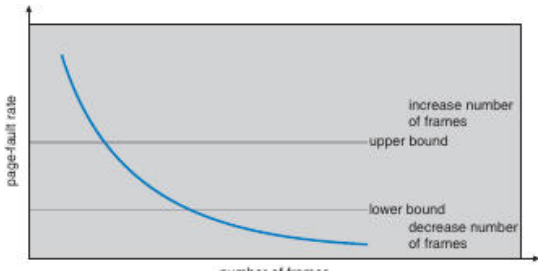
- Periodically (e.g., every 5 000 references) the OS samples each page's **reference bit**.
- Pages with the bit set are considered part of the working set; those without are candidates for eviction.
- The OS may **suspend** a process whose working set cannot be satisfied, swap its pages out, and reallocate the frames to other processes.

Page-Fault Frequency (PFF) Control

> Instead of tracking exact working-set windows, the **PFF** approach monitors the **page-fault rate** of each process.

- Define an **upper bound** and a **lower bound** on acceptable fault rates.
- If a process's fault rate exceeds the upper bound, the OS **adds a frame** to that process.
- If the fault rate falls below the lower bound, the OS **removes a frame** (if possible).

The graph below illustrates the relationship between **number of frames** and **page-fault rate**; the shaded region between the upper and lower bounds indicates the desirable operating range.



The curve shows that as more frames are allocated, the page-fault rate drops sharply. Horizontal grey lines mark the upper and lower bounds; the OS aims to keep the system within this corridor to avoid thrashing.

Practical Remarks

- Providing **sufficient physical memory** is the most effective way to prevent thrashing.
- Modern systems combine **local replacement** (to protect high-priority processes) with **working-set monitoring** to keep multiprogramming levels high without sacrificing throughput.

📂 Memory-Mapped Files

> **Memory-mapping** a file associates a region of the process's virtual address space with a file stored on disk.

- The first access to a mapped page generates a **demand-page fault**, after which the page is loaded into a physical frame just like any other page.
- Subsequent accesses to the same page behave as ordinary memory reads/writes, eliminating the need for explicit read()/write() system calls.

Benefits

- Reduces system-call overhead for large sequential reads or writes.
- Allows the OS to apply its **page-replacement** and **caching** policies uniformly to file data and program data.
- Enables **shared memory** between processes simply by mapping the same file region into each process's address space.

Typical workflow

1. Process invokes mmap() (or the OS-specific equivalent) specifying the file and desired protection flags.
2. The OS creates a **page-table entry** that points to the file's disk blocks.
3. On first reference, a page fault brings the block into a frame; the page-table entry is updated to reference the frame.
4. Writes to a mapped page set the **dirty bit**; the page will be written back to the file on eviction or when msync()/munmap() is called.

Memory-mapped I/O is especially advantageous for **large data sets** (e.g., multimedia processing, scientific simulations) where the cost of repeated read() calls would dominate performance.

📚 Working Set Dynamics & Locality

> **Working set** – the set of pages a process actively uses during a given interval of execution.

- When a process moves to a **new working set**, the page-fault rate initially spikes, then falls as the new pages are loaded.
- The interval between the start of one fault-rate peak and the next marks the transition from one working set to another.
- Frequent switches between large working sets can cause **thrashing**, where the system spends most of its time handling page faults.

📁 Memory-Mapped Files

> **Memory-mapped file** – a file whose contents are mapped into a process's virtual address space, allowing the program to access file data through ordinary memory reads and writes.

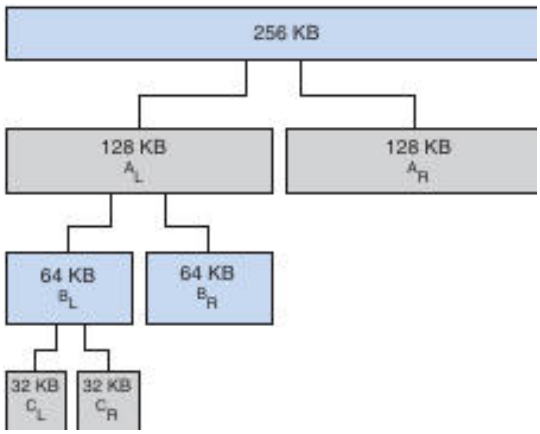
- Reads and writes become simple memory operations; no explicit read()/write() system calls are needed.
- **Writes are not immediately synchronous** – the OS may defer updating the physical file until the page is flushed (e.g., by the page-replacement daemon).
- When a mapped page is evicted, any modifications are written back to the file and the page is removed from the process's address space.

Key properties

Property	Effect

Transparency	File I/O is performed via normal memory accesses.
Lazy updating	Modified pages are written back only when the OS decides (periodic flushing or on unmap).
Sharing	Multiple processes can map the same file, seeing a single physical copy of each page.
Copy-on-Write (COW)	Read-only mappings can be shared; a write triggers a private copy for the writer.

Illustration of shared mapping



The diagram (Figure 8.22) shows two processes whose virtual address spaces map the same file. Both virtual pages point to the same physical page, so a write by one process is immediately visible to the other.

🤝 Shared Memory via Memory-Mapped Files

> **Shared memory** – a region of memory that multiple processes can map, enabling direct communication without extra system calls.

- Implemented by creating a **memory-mapped file** that all participating processes open and map into their address spaces.
- The file can be a regular on-disk file or a special **POSIX shared-memory object** (shm_open).
- Because the underlying pages are the same physical pages, writes by any process are instantly visible to all others.

Typical workflow

1. **Create** a shared-memory object (or ordinary file).
2. **Map** it into the address space of each process (mmap).
3. **Read/write** the shared region as ordinary memory.
4. **Unmap** (munmap) and **close** the object when done.

🪟 Windows API Memory-Mapped Files

> **CreateFileMapping** – Windows API call that creates a kernel object representing a file mapping; the object can be named for sharing.

> **MapViewOfFile** – establishes a view of the mapping in the calling process's virtual address space and returns a pointer to the mapped region.

Producer-consumer example (simplified)

1. **Producer**
    - CreateFile opens the backing file.
    - CreateFileMapping creates a named mapping ("SharedObject").
    - MapViewOfFile maps the entire file; the returned pointer is used to write a message.
2. **Consumer**
    - OpenFileMapping opens the existing named mapping.
    - MapViewOfFile obtains a pointer to the same physical pages.
    - Reads the message from the shared region.

Both processes later call UnmapViewOfFile and close their handles.

Note: The mapping is **demand-paged**; pages are brought into physical memory only when accessed, not at mapping creation time.

🖥️ Memory-Mapped I/O

> **Memory-mapped I/O** – a technique where device registers are mapped into the processor's address space, allowing the CPU to read/write them with ordinary load/store instructions.

- Ideal for **fast-response devices** (e.g., video controllers, serial ports).
- The CPU accesses a specific address range; the hardware translates the access to a device register read or write.
- **Polling** (busy-waiting) and **interrupt-driven** I/O are both possible; the difference lies in whether the CPU repeatedly checks a control bit or is notified by an interrupt.

Advantages

- Simpler code – no special I/O instructions.
- Can leverage the same caching and protection mechanisms used for regular memory (though device pages are typically marked non-cacheable).

🧩 Kernel Memory Allocation

🧱 Buddy System

> **Buddy system** – a power-of-2 allocator that splits a fixed-size memory pool into pairs of "buddies" to satisfy allocation requests.

- Memory is divided recursively into halves (buddies) until a block of size ≥ request is found.
- When a block is freed, its buddy is checked; if both are free they are **coalesced** back into a larger block.

Advantages

- Fast allocation and deallocation (simple bit-maps).
- Easy coalescing of adjacent free blocks.

Drawbacks

- Internal fragmentation: allocation size is rounded up to the next power of 2, potentially wasting up to ~50 % of a block.

Example – allocating 21 KB from a 256 KB pool results in successive splits to 32 KB blocks; the 21 KB request occupies one 32 KB buddy, leaving the remainder available for future splits.

🧱 Slab Allocation

> **Slab allocator** – a cache-based allocator that maintains per-object-type slabs, each consisting of one or more contiguous pages holding multiple objects of the same size.

- **Cache**: one per kernel data structure (e.g., semaphores, task structs).
- **Slab**: a set of pages belonging to a cache.
- **Object**: an individual instance stored inside a slab.

Slab states

State	Description
Full	All objects in the slab are in use.
Partial	Some objects are free, some allocated.
Empty	All objects are free; the slab can be reclaimed.

Benefits

- No fragmentation – objects are allocated exactly to their size.
- Fast allocation: a free object is taken from a partial slab; if none exist, a new slab is created.
- Immediate reuse: when an object is freed, it returns to its cache, ready for the next request.

Variations

- **SLAB** (original Solaris implementation).
- **SLUB** (Linux default since 2.6.24) – stores metadata in the page structures and removes per-CPU queues, improving scalability on many-core systems.
- **SLOB** – simple allocator for memory-constrained embedded systems, using three size classes (small, medium, large) with a first-fit policy.

📈 Other Considerations

🔧 Prepaging

> **Prepaging** – proactively loading a set of pages into memory before they are actually referenced, aiming to reduce the initial burst of page faults.

- Often used when a process is about to resume after being swapped out or when a small file is opened.
- The trade-off: if a large fraction of the prefetched pages are never used, prepaging adds unnecessary I/O.
- Effectiveness can be expressed as s * (1-α) where s is the number of prefetched pages and α is the fraction actually used.

🔪 Page Size Selection

> **Page size** – the granularity of memory paging, always a power of 2 (commonly 4 KB to 4 MB).

Factors influencing the choice

Factor	Impact of Larger Pages	Impact of Smaller Pages
Page-table size	Fewer entries → smaller tables, less memory overhead.	More entries → larger tables, higher overhead per process.
Internal fragmentation	Higher – the final partially used page can waste many bytes.	Lower – less unused space per process.
I/O transfer time	Fewer I/O operations (each transfers more data).	More I/O operations, but each transfer is quicker; overall effect limited because seek/latency dominate.
TLB coverage	Fewer TLB entries needed to cover the same address space.	More TLB entries required; may increase miss rate.

Typical design balances **memory-utilization efficiency** (favoring small pages) against **overhead** (favoring larger pages). Modern architectures often support multiple page sizes (e.g., 4 KB “base” pages and 2 MB “huge” pages) to allow the OS to choose per-allocation.